PYTHON MACHINE LEARNING · ADVANCED MACHINE LEARNING · COMPLETE WORKBOOK

Advanced Machine Learning

Your working notebook for all 13 sessions of Advanced Machine Learning. Read the key ideas, study the worked examples, do the guided labs, predict before you run, and check yourself with the self-check quizzes.

13 sessions · 23 lessons | Write-in exercises & self-checks | Name: _____

CONTENTS

S01 **Interpreting Models and SHAP**

- 7.1 Interpreting Models
- 7.2 SHAP Values

S02 **Local Explanations, Fairness and K-Means**

- 7.3 LIME and Fairness Checks
- 8.1 K-Means Clustering

S03 **Choosing k and Hierarchical Clustering**

- 8.2 Choosing k and Customer Segments
- 8.3 Hierarchical Clustering

S04 **Density-Based Clustering and Gaussian Mixtures**

- 8.4 Density-Based Clustering
- 8.5 Gaussian Mixture Models

S05 **Anomaly Detection and PCA**

- 8.6 Anomaly Detection
- 9.1 Principal Component Analysis

S06 **LDA, t-SNE and UMAP**

- 9.2 Linear Discriminant Analysis
- 9.3 t-SNE and UMAP

S07 **Neural Networks and PyTorch**

- 10.1 Neural Networks from Scratch
- 10.2 PyTorch Fundamentals

S08 **CNNs and Sequence Models**

10.3 Convolutional Neural Networks

10.4 Sequence Models

S09 **Transfer Learning and Text Classification**

10.5 Transfer Learning

11.1 Text Classification

S10 **Recommendations and Reproducibility**

11.2 Recommendation Systems

12.1 Reproducibility and Experiment Tracking

S11 **Serving, Monitoring and the Churn Case Study**

12.2 Serving and Monitoring Models

13.1 Case Study: Predicting Churn

S12 **Capstone Kickoff and Studio I**

13.2 Capstone Project

S13 **Capstone Studio II and Showcase**

13.2 Capstone Project

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 1: Interpreting Models and SHAP

Your workbook for Session 1: the environment check, what this kit assumes from Foundations and a refresher, the key ideas of Lessons 7.1 and 7.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons **7.1–7.2** | Session **1** · 3 hours | Name: _____

SESSION GOALS

- Prove your machine is ready for the whole kit: the environment check ends with READY (or READY for Sessions 1 to 4, with the image download planned).
- Check the Foundations habits this kit builds on: the canonical split, named pipelines, cross-validation, metrics and the $15 / $60 threshold.
- Open a model up three ways: coefficients as odds ratios (the recipe), permutation importance (the taste test), partial dependence (the dial).
- Split one prediction into fair shares with SHAP, check that they add up, and speak about them in the right units.
- Say every finding as a statement about the model, never about causes.

START HERE: WHAT THIS KIT ASSUMES

YOUR ENVIRONMENT

- ☐ `python labs/module-01/l7_0-orientation/check_environment.py` ends with `READY for Advanced Machine Learning`, or with `READY for Sessions 1 to 4`
- ☐ If it said `READY for Sessions 1 to 4`: I will run it again with `--download` at home before Session 5 (about 175 MB, once)
- ☐ If it said NOT READY, I copied the numbered fix list here:

SIX CONVENTIONS FROM MACHINE LEARNING FOUNDATIONS

Every Kit B lesson uses these without explaining them again. If one is new to you, the refresher below and `labs/module-01/l7_0-orientation/diagnostic_starter.py` practice it.

CONVENTION	CODE	NUMBERS TO RECOGNIZE
The canonical churn split	`train_test_split(customers, test_size=0.25, stratify=customers["churned"], random_state=42)`	4,500 / 1,500; first test customer C102112
A baseline first	`DummyClassifier()`	accuracy 0.835, ROC AUC 0.500 on churn
Named pipelines	`Pipeline([("prep", preprocess), ("model", model)])`; `pipe["model"]`, `pipe[-1]`	Lesson 2.2 pipeline: logistic regression test AUC 0.828, CV 0.833 +/- 0.021
Cross-validation on training rows only	`StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`, `cross_val_score(..., scoring="roc_auc")`	gradient boosting CV 0.861 +/- 0.013; tuned (Lesson 6.2) CV 0.8795, test 0.874
Metrics that match the decision	ROC AUC to rank, average precision for rare positives, recall and precision at a threshold	fraud: 0.88% positives, so accuracy 0.9913 means nothing
Campaign economics	$15 per offer, $60 per real churner reached: profit = 60 x TP - 15 x flagged	break-even probability 0.25; threshold chosen on out-of-fold training probabilities (Lesson 1.4: 0.245, $3,555 on test)

FOUNDATIONS REFRESHER: PREDICT, THEN CHECK

Six questions, one per convention. Circle your answer before the class reveals it. Nobody collects this. Missed one? The skill and what to review: **D1** the canonical churn split (Foundations Lessons 1.1 and 1.2) · **D2** named pipelines (Foundations Lesson 2.2) · **D3** cross-validation on training rows (Foundations Lesson 1.3) · **D4** recall and precision (Foundations Lesson 1.4) · **D5** ROC AUC as ranking (Foundations Lesson 1.4) · **D6** campaign economics and a threshold (Foundations Lesson 1.4).

D1

What does this print?

```
import pandas as pd
from sklearn.model_selection import train_test_split
customers = pd.read_csv("data/customers_churn.csv")
train, test = train_test_split(customers, test_size=0.25, stratify=customers["churn
print(len(train), len(test), test["customer_id"].iloc[0])
```

A) `4500 1500 C100001`

B) `4500 1500 C102112`

C) `1500 4500 C102112`

D) `4800 1200 C102112`

D2

What does this print?

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe = Pipeline([("prep", StandardScaler()), ("model", LogisticRegression())])
print(list(pipe.named_steps), type(pipe["model"]).__name__)
```

A) `['standardscaler', 'logisticregression'] LogisticRegression`

B) `['prep', 'model'] Pipeline`

C) `KeyError: 'model'`

D) `['prep', 'model'] LogisticRegression`

D3

What does this print?

```
import pandas as pd
from sklearn.model_selection import StratifiedKFold, train_test_split
customers = pd.read_csv("data/customers_churn.csv")
train, test = train_test_split(customers, test_size=0.25, stratify=customers["churn
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
y_train = train["churned"]
print([len(val) for _, val in cv.split(y_train, y_train)])
```

A) `[900, 900, 900, 900, 900]`

B) `[3600, 3600, 3600, 3600, 3600]`

C) `[1125, 1125, 1125, 1125]`

D) `[300, 300, 300, 300, 300]`

D4

What does this print?

```
from sklearn.metrics import recall_score, precision_score
y_true = [1, 1, 1, 0, 0, 0, 0, 0]
y_pred = [1, 0, 0, 1, 0, 0, 0, 0]
print(round(recall_score(y_true, y_pred), 3), precision_score(y_true, y_pred))
```

A) `0.5 0.333`

B) `0.75 0.75`

C) `0.333 0.5`

D) `0.125 0.5`

D5

What does this print?

```
from sklearn.metrics import roc_auc_score
print(roc_auc_score([0, 0, 1, 1], [0.1, 0.4, 0.35, 0.8]))
```

A) `0.5`

B) `0.75`

C) `1.0`

D) `0.25`

D6

What does this print?

```
import numpy as np
y = np.array([1, 0, 1, 0, 0, 1])
p = np.array([0.9, 0.3, 0.2, 0.6, 0.1, 0.4])
flags = p >= 0.25                                # $15 per offer, $60 per churner rea
tp = (flags & (y == 1)).sum()
print(flags.sum(), tp, 60 * tp - 15 * flags.sum())
```

A) `4 2 120`

B) `3 3 135`

C) `6 3 90`

D) `4 2 60`

LESSONS

7.1 Interpreting Models

BY THE END YOU CAN

- Read the coefficients of a scaled logistic regression as odds ratios, and say when they mislead
- Compute permutation importance on the test set with `permutation_importance` and explain why impurity-based importance can fool you
- Draw partial dependence and ICE plots with `PartialDependenceDisplay` and read thresholds, curves and interactions from them
- State what feature importance does and does not mean: it describes the model, not the causes of churn

KEY IDEAS

- **Global and local.** "What drives the model?" and "is it using anything it should not?" are global questions; "why this customer?" is local.
- **The recipe: coefficients.** In a scaled logistic regression, `np.exp(coef)` is an odds ratio per standard deviation. Divide by the scaler's `scale_` to speak in natural units (x1.36 per 30 days without an order). Coefficients cannot bend, overlapping features share credit, and "held fixed" may describe customers who do not exist.
- **The trap:** `feature_importances_`. Impurity importance is measured on training data and favors many-valued columns: it ranked pure noise sixth.
- **The taste test: permutation importance.** Shuffle one test column, measure the drop in a held-out score; repeat and report mean and std. Works for any model. Correlated features split the credit: shuffle related columns together.
- **The dial: partial dependence.** Set one feature to each grid value for everyone, predict, average. ICE curves are the same per customer; fanning ICE lines signal an interaction. `method="brute"` averages real probabilities; PDP needs float columns.
- **Describe the model, not the world.** "The model relies on recency", never "recency causes churn".

WORKED EXAMPLE: THREE MORE ODDS RATIOS, AND A CURVE

The lesson turned two coefficients into business sentences. Do the same for three more, then check one of them against a curve: does a constant odds ratio tell the whole story?

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "days_s
            "returns_12m", "return_rate", "support_tickets_12m", "discount_share",
            "email_opens_90d"]
X = customers[features].astype(float)
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify
logreg = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), Logisti

per_unit = pd.Series(logreg[-1].coef_[0] / logreg[1].scale_, index=features)
print(f"one more support ticket:          odds x{np.exp(per_unit['support_tickets_
print(f"10 points more discount share:    odds x{np.exp(0.10 * per_unit['discount_
print(f"5 more email opens in 90 days:    odds x{np.exp(5 * per_unit['email_opens_
```

```
one more support ticket:          odds x1.48
10 points more discount share:    odds x1.46
5 more email opens in 90 days:    odds x0.76
```

Each line is "other features held fixed": one more support ticket multiplies the odds of churn by about 1.5; ten percentage points more of spend on discounts by about 1.5; five more email opens in 90 days by about 0.76. A logistic regression gives every step the same multiplier, wherever the customer starts.

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.inspection import partial_dependence

hgb = HistGradientBoostingClassifier(random_state=42).fit(X_train, y_train)
grid = [0.1, 0.2, 0.3, 0.4, 0.5]
curve = partial_dependence(hgb, X_test, ["discount_share"], method="brute",
                           custom_values={"discount_share": grid})
print("boosting, average predicted churn:", curve["average"][0].round(3).tolist())
real = customers.groupby(pd.cut(customers["discount_share"], [0, 0.15, 0.25, 0.35,
                         observed=True)["churned"].mean()
print("real churn rate by discount band: ", real.round(3).tolist())
```

```
boosting, average predicted churn: [0.095, 0.095, 0.119, 0.282, 0.363]
real churn rate by discount band:  [0.118, 0.127, 0.126, 0.343, 0.331]
```

The boosting model's dial disagrees: flat from 0.1 to 0.2, then a sharp rise between 0.3 and 0.4. The real churn rate by discount band agrees with the curve (about 12% up to 0.35, then about 34%). A single odds ratio averages a kink into a slope: the first of the three ways coefficients mislead.

GUIDED LAB

- Open `labs/module-01/l7_1-interpreting-models/fraud_drivers_starter.py` and run it as it is: it trains the fraud model and prints five TODO lines.
- TODO 1 and 2 (together): permutation importance scored by average precision, then by ROC AUC. Which feature changes the most between the two?
- TODO 3 to 5 (in pairs): the hour dial, the same dial for known and new devices, and the real fraud rate by time of day.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `pipeline_importance_starter.py` (the full Foundations pipeline, categories and all), then `debug_interpret.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Three coefficients become odds ratios.)

```
import numpy as np
print(round(np.exp(0.69), 2), round(np.exp(-0.69), 2), np.exp(0.0))
```

P2

Predict the output, then run it. (A one-split tree that only ever looks at column 0.)

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.inspection import permutation_importance
rng = np.random.default_rng(0)
X = rng.normal(size=(200, 2))
y = (X[:, 0] > 0).astype(int)                # only column 0 matters
tree = DecisionTreeClassifier(max_depth=1, random_state=0).fit(X, y)
r = permutation_importance(tree, X, y, n_repeats=5, random_state=0)
print(r.importances_mean.round(2))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The churn-drivers briefing (lesson page, "Project: The Churn-Drivers Briefing"). In class: Steps 1 and 2. At home: Steps 3 to 5.

- ☐ Step 1: train and check. `test ROC AUC 0.865 (baseline 0.500)`
- ☐ Step 2: permutation importance with mean and std; top `days_since_last_order 0.140 0.012`
- ☐ Step 3: `group_importance` for purchase volume, returns and engagement
- ☐ Step 4: turning points from partial dependence on percentile grids
- ☐ Step 5: `churn_drivers.csv` (11 rows) and a summary that says the drivers are not causes

SELF-CHECK

Q1

In a scaled logistic regression, `days_since_last_order` has a coefficient of 0.89. Which sentence is right?

A) One more day without an order raises the churn probability by 0.89

B) The model is 89% sure that recency causes churn

C) One more standard deviation of days without an order multiplies the odds of churn by about 2.4, other features held fixed

D) One more standard deviation of days without an order raises the churn probability by 2.4 points

Q2

What does this print?

```
import numpy as np
import pandas as pd
coefs = pd.Series([0.40, -0.69], index=["support_tickets_12m", "app_user"])
print(np.exp(coefs).round(2).tolist())
```

Q3

You add a column of pure random noise to the churn data. The forest's `feature_importances_` ranks it sixth, above `return_rate` and `app_user`; permutation importance on the test set gives it 0.005. What follows?

A) The noise column must hold a little real churn signal

B) Impurity importance cannot be trusted for the ranking: it is measured on training data and rewards columns the trees can memorize with

C) `app_user` is useless and can be dropped

D) Permutation importance is broken for random forests

Q4

In Lab 7.1 A, shuffling `hour` costs the fraud model 0.074 of average precision but nothing (-0.005) of ROC AUC. What is the best reading?

A) The hour column is broken and should be removed

B) Average precision is a buggy metric for permutation importance

C) The model does not use the hour at all

D) Importance depends on the metric: the hour lifts a few night frauds to the very top of the list, which average precision rewards and the whole-ranking AUC hardly notices

Q5

What does this print?

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.inspection import partial_dependence
X = pd.DataFrame({"a": [0.0, 1.0, 2.0, 3.0], "b": [0.0, 0.0, 4.0, 4.0]})
model = LinearRegression().fit(X, X["a"] + X["b"])       # learns y = a + b ex
res = partial_dependence(model, X, ["a"], method="brute", custom_values={"a":
print(res["average"][0].round(2))
```

Q6

Which sentence belongs in the churn-drivers briefing?

A) Customers with long gaps since their last order are predicted to churn more; the risk jumps after about four months

B) Long gaps since the last order cause churn, so emailing customers at three months will cut churn

C) Recency is the cause of 14% of churn

D) The app reduces churn by half, so the app campaign will work

Write the three sentences you would say to Pinecrest's retention team about what drives the churn model. Then write the one sentence you would add so that nobody leaves the meeting thinking the model has found the causes of churn.

7.2 SHAP Values

BY THE END YOU CAN

- Explain the Shapley idea (average contribution over every order in which features join) and compute it by hand for a tiny model
- Compute SHAP values for a gradient-boosting model with `shap.TreeExplainer` and check that they add up to the prediction
- Read waterfall, beeswarm and scatter plots, including the units (log-odds) they are drawn in
- Use a model-agnostic explainer on a pipeline, and turn SHAP values into plain-language reason codes

KEY IDEAS

- **The team bonus.** A Shapley value is a feature's average contribution over every order in which the features could arrive. Interaction effects are split fairly; a feature that never changes the score gets 0.
- **The bookkeeping.** Base value + all SHAP values = the model output being explained, exactly. Check it on every new model.
- **Units.** `TreeExplainer` on a boosting classifier explains the raw score in log-odds. Compare sizes and signs; never add them to a probability; squash the end points with `expit`.
- **Plots.** Waterfall: one prediction as a staircase. Beeswarm and mean |SHAP|: many explanations as a global picture. Scatter: shapes and interactions.
- **Any model.** `PermutationExplainer(function, background)` works on pipelines and anything that returns a number; slower, and the background defines "typical".
- **Reason codes.** Positive shares become neutral phrases filled with the customer's real value: "flagged because", never "leaving because".

WORKED EXAMPLE: A CUSTOMER ON THE BREAK-EVEN LINE

The lesson explained the riskiest customer. Real decisions happen at the threshold. Take the test customer whose predicted churn is closest to 0.25, the break-even probability of the $15 / $60 campaign, and ask why the model puts them right on the line.

```
import numpy as np
import pandas as pd
import shap
from scipy.special import expit
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "days_s
            "returns_12m", "return_rate", "support_tickets_12m", "discount_share",
            "email_opens_90d"]
train, test = train_test_split(customers, test_size=0.25, stratify=customers["chur
X_train, X_test = train[features].astype(float), test[features].astype(float)
hgb = HistGradientBoostingClassifier(random_state=42).fit(X_train, train["churned"

proba = hgb.predict_proba(X_test)[:, 1]
j = int(np.abs(proba - 0.25).argmin())          # the customer closest to the 0.25
print(test["customer_id"].iloc[j], f"churn probability {proba[j]:.3f}")
```

```
C104089 churn probability 0.250
```

```
sv = shap.TreeExplainer(hgb)(X_test.iloc[[j]])
shares = pd.Series(sv.values[0], index=features)
print("pushes up:  ", shares.nlargest(3).round(2).to_dict())
print("pushes down:", shares.nsmallest(2).round(2).to_dict())
start, end = sv.base_values[0], sv.base_values[0] + shares.sum()
print(f"log-odds {start:.2f} -> {end:.2f}   probability {expit(start):.3f} -> {exp
```

```
pushes up:   {'discount_share': 2.92, 'app_user': 0.26, 'email_opens_90d': 0.11}
pushes down: {'tenure_months': -0.68, 'days_since_last_order': -0.5}
log-odds -2.95 -> -1.10   probability 0.050 -> 0.250
```

```
cols = ["days_since_last_order", "discount_share", "tenure_months", "orders_12m",
print(X_test.iloc[j][cols].to_dict())
```

```
{'days_since_last_order': 76.0, 'discount_share': 0.733, 'tenure_months': 72.0, '
```

One feature does almost all the work: a discount share of 0.73 (nearly three quarters of this customer's spend was discounted) adds 2.92 log-odds. A recent order (76 days) and a long tenure pull the other way. The end points, squashed: from the typical 5% to exactly the break-even 25%. An agent deciding whether to spend $15 on this customer now knows the call is about discounts, and that a slightly different discount history would move them across the line.

GUIDED LAB

- **Lab B first (pairs):** `labs/module-01/l7_2-shap-values/shapley_by_hand_starter.py`. Before running, predict how the 0.30 night-and-new-device bonus is split and what `gift_card` gets.
- **Lab A:** `labs/module-01/l7_2-shap-values/fraud_reasons_starter.py`. SHAP values for 15,000 payments, the additivity check, the review queue with three reasons each, and the night x new-device interaction.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `debug_shap.py`: two loud bugs and one silent one.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (The logistic function at three log-odds values.)

```
from scipy.special import expit
print(expit(0.0), round(expit(-2.951), 3), round(expit(3.55), 3))
```

P4

Predict the output, then run it. (Two features earn a bonus only together.)

```
import itertools
def score(a, b):
    return 0.1 + (0.4 if a and b else 0.0)    # a bonus only when BOTH are pre
shares = {"a": 0.0, "b": 0.0}
for order in itertools.permutations(["a", "b"]):
    present = {"a": 0, "b": 0}
    for name in order:
        before = score(**present)
        present[name] = 1
        shares[name] += (score(**present) - before) / 2
print(shares)
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Reasons for the call list (lesson page, "Project: Reasons for the Call List"). In class: Steps 1 to 3. At home: Steps 4 and 5.

- ☐ Step 1: split the whole table. Check: `4500 1500 C102112`
- ☐ Step 2: SHAP values for the 20 riskiest customers (C104051 first)
- ☐ Step 3: a neutral phrase per feature; `C104051 ['no order in 364 days', '43% of spend on discounted items', '3 support tickets this year']`
- ☐ Step 4: the call list, three reasons per customer
- ☐ Step 5: the most common top reason, and `call_list_reasons.csv` with no outcome column

SELF-CHECK

Q7

Lab 7.2 B's fraud rule never looks at `gift_card`. Explained against a typical payment, what Shapley value does `gift_card` get, and why?

A) A quarter of the total, because the four features share the gap equally

B) 0, because adding it never changes the score in any arrival order

C) A negative value, to balance the interaction bonus

D) It cannot be computed: Shapley values need every feature to matter

Q8

What does this print?

```
import numpy as np
from scipy.special import expit
base = -2.0
shares = np.array([1.5, 0.7, -0.2])
print(round(base + shares.sum(), 2), round(expit(base + shares.sum()), 3))
```

Q9

For the fraud model, `device_new` tops the mean |SHAP| ranking (0.247) while `velocity_24h` tops permutation importance. How can both be right?

A) One of the two calculations has a bug

B) SHAP ranks only the riskiest payments

C) Mean |SHAP| measures how much a feature moves predictions; permutation importance measures how much the score depends on it, and the two need not agree

D) Permutation importance is always more correct

Q10

A random forest's explanation has `values.shape` equal to `(200, 11, 2)`. Which object do you pass to `shap.plots.waterfall` for the first customer's churn class?

A) `forest_sv[0]`

B) `forest_sv[:, :, 0]`

C) `forest_sv.values.sum()`

D) `forest_sv[0, :, 1]`

Q11

You explain the logistic-regression pipeline with a PermutationExplainer and swap the background from 100 random customers to 100 Gold-tier customers. What changes?

A) The base value and the shares change; the customer's predicted probability does not

B) Nothing: Shapley values do not depend on the background

C) The predicted probability changes, because the model is refitted on Gold customers

D) The shares stay the same but no longer add up

Q12

What does this print?

```
phrases = {"discount_share": "{:.0%} of spend on discounted items",
           "days_since_last_order": "no order in {:.0f} days"}
print(phrases["discount_share"].format(0.432), "|", phrases["days_since_last_c
```

Explain the team-bonus idea to a friend who has never heard of Shapley values, using the toy churn score. Then write how you would word the three reasons for customer C104051 if an agent were going to read them aloud on a call, and what you would make sure the agent does not say.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them.

E1

In one sentence each: what does permutation importance tell you about a model, what does partial dependence tell you, and what does neither tell you?

E2

A SHAP waterfall for the boosting churn model starts at -2.95 and ends at +0.50. What churn probability does the model give, and why can you not add the bars to 16.5%?

E3

Write the reason code an agent could read aloud for `days_since_last_order` = 240 with a SHAP value of +2.1, and one thing the agent must not say.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the churn-drivers briefing (Steps 3 to 5) and the call-list reasons (Steps 4 and 5)
- ☐ Lesson 7.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 7.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ 7.2 "Before the Next Lesson": keep the notebook; guess whether the call list would look the same for younger and older customers
- ☐ If my check said READY for Sessions 1 to 4: run it again with `--download` at home before Session 5
- ☐ Optional: `diagnostic_starter.py` (if I missed two or more refresher questions) and `pipeline_importance_starter.py`
- ☐ Bring to Session 2: a READY laptop, `pinecrest-ml` with `data/` and `labs/`, my 7.2 project notebook and `call_list_reasons.csv`, this workbook and my journal

GLOSSARY

TERM	MEANING
global / local explanation	About the whole model / about one prediction
odds ratio	`np.exp(coef)`: how a one-unit (here one-standard-deviation) step multiplies the odds
impurity importance (MDI)	`feature_importances_` of a tree ensemble: measured on training data, biased to many-valued columns
permutation importance	The drop in a held-out score when one column (or a group) is shuffled
partial dependence (PDP)	The average prediction as one feature is set to each grid value for everyone
ICE curve	Partial dependence for one row; the PDP is the average of the ICE curves
interaction	One feature's effect depends on another's value (tickets matter for non-app users only)

TERM	MEANING
Shapley value	A feature's average contribution over every arrival order of the features
base value	The model's average output over the background data; where every explanation starts
additivity	Base value + all SHAP values = the output being explained
log-odds	The raw score of a logistic model or a boosting classifier; `expit` turns it into a probability
background data	The rows that define "a typical value" when a feature is left out
reason code	A neutral phrase built from a positive SHAP value and the customer's real value

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 2: Local Explanations, Fairness and K-Means

Your workbook for Session 2: a warm-up on Session 1, the key ideas of Lessons 7.3 and 8.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 7.3–8.1 | Session 2 · 3 hours | Name: _____

SESSION GOALS

- Explain one prediction with LIME, and check its fidelity and stability before you trust it.
- Audit a model by group: flag rate, recall and precision, group sizes, and a permutation test.
- Find groups without labels with K-Means, and know what its assign-and-move loop does.
- Scale before you cluster, start several times, and read every cluster in real units.

WARM-UP

Five questions on Session 1. Write first; we go through them together.

W1

A scaled logistic regression gives `discount_share` a coefficient of 0.68. Say what that means in one sentence.

W2

Shuffling `days_since_last_order` in the TEST set lowers the boosting model's ROC AUC by 0.140. What does that measure, and why the test set?

W3

What does this print?

```
import numpy as np
from scipy.special import expit
base = -2.95                    # C104051's waterfall starts here (log-odds)
total_shares = 6.50             # and its 11 SHAP values add up to this
print(round(base + total_shares, 2), round(expit(base + total_shares), 3))
```

W4

Why should a briefing never rank features with a forest's `feature_importances_`?

W5

Which phrase can an agent read aloud: "you are leaving because you stopped buying" or "no order in 240 days"? Why?

LESSONS

7.3 LIME and Fairness Checks

BY THE END YOU CAN

- Describe LIME's recipe (perturb, predict, weight by closeness, fit a simple model) and build a basic version by hand
- Explain a prediction with `lime.lime_tabular.LimeTabularExplainer` and judge it by its fidelity score and its stability across random seeds
- Say when LIME and SHAP answer different questions, and run simple what-if checks on one customer
- Run a fairness check: compare flag rate, recall and precision across age groups and test whether a gap is larger than chance

KEY IDEAS

- **A flat map near one customer.** Perturb, predict, weight by closeness, fit a weighted straight line: its slopes (or binned rules) are the explanation.
- **Two checks.** Fidelity (`exp.score` , and `exp.local_pred` close to the model) and stability (the top reasons survive another seed). Use 5,000 variants and a fixed `random_state` for anything you report.
- **Slope vs share.** LIME's local slope is sensitivity nearby; SHAP's share is the contribution to the gap from a typical customer. They can rank features differently and both be right.
- **What-ifs** change one value and ask again: they describe the model's answer, not what a customer would do.
- **Fairness through unawareness guarantees nothing.** Measure outcomes by group: flag rate (demographic parity), recall (equal opportunity), precision. Choose the one that matches the harm.
- **Small groups are noisy.** Report group sizes; a permutation test (shuffle the group labels) says whether a gap is bigger than chance. "No evidence" is not "proven fair".

WORKED EXAMPLE: A WHAT-IF, AND THREE FAIRNESS VIEWS ON TEN CUSTOMERS

Session 1's worked example found customer C104089 exactly on the 0.25 break-even, pushed there by a discount share of 0.73. A what-if asks the model: how far would that share have to fall for the model to stop flagging them?

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "days_s
            "returns_12m", "return_rate", "support_tickets_12m", "discount_share",
            "email_opens_90d"]
train, test = train_test_split(customers, test_size=0.25, stratify=customers["chur
X_train, X_test = train[features].astype(float), test[features].astype(float)
model = HistGradientBoostingClassifier(random_state=42).fit(X_train, train["churne

row = X_test[test["customer_id"] == "C104089"]                # the break-even cu
for share in [0.733, 0.6, 0.5, 0.4, 0.3, 0.2]:
    what_if = row.assign(discount_share=share)
    print(f"discount share {share:.3f} -> predicted churn {model.predict_proba(wha
```

```
discount share 0.733 -> predicted churn 0.250
discount share 0.600 -> predicted churn 0.070
discount share 0.500 -> predicted churn 0.092
discount share 0.400 -> predicted churn 0.060
discount share 0.300 -> predicted churn 0.008
discount share 0.200 -> predicted churn 0.008
```

The model's answer is a cliff, not a slope: from 0.733 to 0.6 the predicted churn falls from 0.25 to 0.07, far below the break-even. This is a statement about the model (its discount threshold sits between 0.6 and 0.73 for a customer like this), not a promise that cutting this customer's discounts would keep them.

Second, why the fairness definition matters. Ten customers in each of two groups, three flagged in each:

```
toy = pd.DataFrame({"group":   ["A"] * 10 + ["B"] * 10,
                    "churned": [1, 1, 1, 1, 0, 0, 0, 0, 0, 0] + [1, 1, 0, 0, 0, 0,
                    "flag":    [1, 1, 0, 0, 1, 0, 0, 0, 0, 0] + [1, 1, 1, 0, 0, 0,
view = toy.groupby("group").agg(flag_rate=("flag", "mean"))
view["recall"] = toy[toy["churned"] == 1].groupby("group")["flag"].mean()
view["precision"] = toy[toy["flag"] == 1].groupby("group")["churned"].mean()
print(view.round(2))
```

```
       flag_rate  recall  precision
group
A            0.3     0.5       0.67
B            0.3     1.0       0.67
```

Equal flag rates (demographic parity) and equal precision, yet group A's churners are reached half as often as group B's (recall 0.5 against 1.0). Which column matters depends on the harm: for an offer that helps churners, recall.

GUIDED LAB

- **Lab A:** `labs/module-02/l7_3-lime-fairness/lime_fraud_starter.py`. LIME on three fraud payments: fidelity, the surrogate's prediction, stability at 500 and 5,000 variants, and SHAP's second opinion. Predict first: will the surrogate land near the model's 0.990?
- **Lab B (pairs):** `labs/module-02/l7_3-lime-fairness/fairness_calibration_starter.py`. Calibration and campaign profit by age group; TODO 1 and 2 in class, 3 and 4 at home.
- Done when your numbers match the "Expected when done" lines. Finished early? `debug_lime.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P1

Predict the output, then run it. (LIME's kernel weights at three distances.)

```
import numpy as np
width = 0.75 * np.sqrt(11)                    # LIME's default kernel width t
distance = np.array([0.0, 2.0, 5.0])          # in standard deviations from t
print(np.exp(-distance ** 2 / width ** 2).round(3))
```

P2

Predict the output, then run it. (Recall for each age group: churners only.)

```
import pandas as pd
audit = pd.DataFrame({"age": ["young"] * 4 + ["older"] * 4,
                      "churned": [1, 1, 0, 0, 1, 1, 1, 0],
                      "flag":    [1, 0, 0, 1, 1, 1, 0, 0]})
print(audit[audit["churned"] == 1].groupby("age")["flag"].mean().round(2).to_c
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Auditing the retention campaign (lesson page). In class: Steps 1 and 2. At home: Steps 3 to 5.

- ☐ Step 1: flag the top 20%. Check: `flagged 300 of 1500; churners among them 52.3%`
- ☐ Step 2: LIME on three flagged customers, with fidelity and the surrogate's prediction
- ☐ Step 3: each top reason rechecked with five seeds
- ☐ Step 4: flag rates by age group, with a permutation test
- ☐ Step 5: `fairness_audit.csv` and a note that says what was not checked

SELF-CHECK

Q1

In Lab 7.3 A, LIME explains a fraud payment the model scores at 0.990. The explanation has fidelity 0.12 and a surrogate prediction of 0.077. What should you do?

A) Treat it as unreliable: the flat map does not imitate the model near this payment; report SHAP's reasons, or fewer reasons

B) Report it: fidelity only matters for regression models

C) Trust the surrogate: the payment's real risk is 0.077

D) Raise `num_features` until fidelity reaches 0.9, then report every rule

Q2

What does this print?

```
import numpy as np
width = 0.75 * np.sqrt(4)
print(np.exp(-np.array([0.0, 1.5, 3.0]) ** 2 / width ** 2).round(3))
```

Q3

A what-if check sets C104051's days since the last order to 60 and the model still predicts 0.645. What is the best lesson from this?

A) The model is broken: recency is its top driver

B) Getting C104051 to order will not keep them

C) A single top reason hides the rest: discounts and tickets still hold the risk up, and the what-if describes the model, not the customer

D) LIME was wrong to rank recency first

Q4

Pinecrest's offer is a benefit: a churner who is flagged gets a discount, one who is missed does not. Which comparison across age groups matches that harm?

A) Flag rate (demographic parity)

B) Recall among churners (equal opportunity)

C) Precision among the flagged

D) Accuracy

Q5

What does this print?

```
import pandas as pd
audit = pd.DataFrame({"group": ["A", "A", "A", "B", "B", "B", "B"],
                      "churned": [1, 1, 0, 1, 1, 1, 0],
                      "flag":    [1, 0, 1, 1, 1, 0, 0]})
churners = audit[audit["churned"] == 1]
print(churners.groupby("group")["flag"].mean().round(2).tolist())
```

Q6

Lab 7.3 B finds the model over-predicts churn for 45 to 54 year-olds by 2.5 points and under-predicts for 35 to 44 by 4.2 points; 64% of shuffles produce a gap at least as large. What do you report?

A) The model is miscalibrated against older customers and must be retrained per age group

B) The model is proven equally calibrated for every age group

C) Age should be added as a feature to fix the calibration

D) No evidence of a real calibration difference at this sample size; the groups are small, so keep monitoring

JOURNAL

You found a 23-point recall gap between two age groups and then showed it was consistent with chance. Write how you would report both facts to a manager in three sentences, without making the model sound either guilty or certified fair. Then note one decision in your own life or work where a model, or a rule of thumb, could treat groups differently without ever looking at group membership.

8.1 K-Means Clustering

BY THE END YOU CAN

- Explain what clustering is for and how it differs from the supervised models of Modules 1 to 7
- Describe Lloyd's algorithm (assign each point to the nearest center, move each center to the mean) and write it in NumPy
- Fit `KMeans` and read `labels_`, `cluster_centers_`, `inertia_` and `n_iter_`
- Explain why K-Means needs scaled features and several starts (`n_init`, k-means++), and show what goes wrong without them
- Profile clusters in the original units and recognize the cluster shapes K-Means cannot find

KEY IDEAS

- **Unsupervised.** X but no y: clustering puts similar rows together. It always returns k groups, even in data that has none, so checking is your job.
- **Lloyd's loop.** Assign each point to the nearest center, move each center to its points' mean, repeat. Each round can only lower **inertia**, the total squared distance to the own center.
- **Local minima.** A poor start can get stuck. k-means++ spreads the starting centers; `n_init=10` keeps the best of ten runs.
- **Scale first.** Distances depend on units; the scale you choose says how much each feature matters. Read centers in real units (`inverse_transform`, or a groupby).
- **Labels are names.** Cluster 2 is not "more" than cluster 1, and numbers change between runs: name clusters from their profiles.
- **Shapes.** K-Means draws straight borders halfway between centers: fine for round, similar clouds; wrong for moons, stretched shapes and very different spreads.

WORKED EXAMPLE: LLOYD'S LOOP FROM A POOR START

Six numbers, two clusters, and a deliberately poor start: both centers at the left end. Follow Lloyd's loop round by round.

```
import numpy as np

points = np.array([1.0, 2.0, 3.0, 10.0, 11.0, 30.0])
centers = np.array([1.0, 2.0])                       # a poor start: both centers
for round_ in range(1, 6):
    labels = np.abs(points[:, None] - centers[None, :]).argmin(axis=1)
    new = np.array([points[labels == j].mean() for j in range(2)])
    inertia = ((points - new[labels]) ** 2).sum()
    print(f"round {round_}: labels {labels.tolist()}  centers {new.round(2).tolist
    if np.allclose(new, centers):
        break
    centers = new
```

```
round 1: labels [0, 1, 1, 1, 1, 1]  centers [1.0, 11.2]  inertia 506.8
round 2: labels [0, 0, 0, 1, 1, 1]  centers [2.0, 17.0]  inertia 256.0
round 3: labels [0, 0, 0, 1, 1, 1]  centers [2.0, 17.0]  inertia 256.0
```

Round 1 sends the 1 to the first center and everything else to the second; the centers move to 1.0 and 11.2. Round 2 regroups {1, 2, 3} and {10, 11, 30}; round 3 changes nothing, so the loop stops at an inertia of 256.

```
from sklearn.cluster import KMeans
best = KMeans(n_clusters=2, n_init=10, random_state=42).fit(points.reshape(-1, 1))
print(best.labels_.tolist(), best.cluster_centers_.ravel().round(2).tolist(), rou
```

```
[0, 0, 0, 0, 0, 1] [5.4, 30.0] 89.2
```

Ten k-means++ starts find a far better answer: the 30 on its own, everything else together, inertia 89.2. The loop only ever goes downhill from where it starts; the first run got stuck in a local minimum. That is why `n_init=10`.

GUIDED LAB

- **Lab B first:** `labs/module-02/l8_1-kmeans/scaling_choices_starter.py`. Four groups from recency, spend and tickets: raw, standardized, and with recency counted twice.
- **Lab A:** `labs/module-02/l8_1-kmeans/payment_clusters_starter.py`. What kinds of payments does Pinecrest take? Raw first, then logged and scaled; name the groups; compare their fraud rates.
- Done when your numbers match the "Expected when done" lines. Finished early? `debug_kmeans.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P3

Predict the output, then run it. (Four points, two clusters.)

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[0], [2], [10], [12
print(sorted(km.cluster_centers_.ravel().tolist()), km.inertia_)
```

P4

Predict the output, then run it. (Which center is nearer?)

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[0], [2], [10], [12
print(km.predict([[5], [7]]) == km.predict([[1], [11]]))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Pinecrest's first customer groups (lesson page). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: four behavior columns. Check: `(6000, 4) missing: 0`
- ☐ Step 2: scale and fit five clusters. Check: `inertia 5713 after 5 rounds`
- ☐ Step 3: the same inertia for three other seeds
- ☐ Step 4: the profile in plain units, with app use and churn
- ☐ Step 5: names from the profile, not the numbers
- ☐ Step 6: `customer_groups.csv` with 6,000 rows

SELF-CHECK

Q7

What does this print?

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[1], [3], [10], [14
print(km.inertia_)
```

Q8

Lab 8.1 A clusters 50,000 payments on amount, account age, items and card velocity without scaling. The four groups differ only in account age. Why?

A) Account age is the most important fraud signal

B) K-Means sorts by the first column with many values

C) The other three columns have too many ties

D) Distances are dominated by the column with the largest spread: account age varies by about 700 days, items by about 1

Q9

What does `KMeans(n_clusters=5, n_init=10, random_state=42)` do with `n_init=10`?

A) Runs the whole algorithm from ten starts and keeps the run with the lowest inertia

B) Runs ten rounds of assign and move, then stops

C) Tries k from 1 to 10 and keeps the best k

D) Averages the centers of ten runs

Q10

The segmentation was fitted on scaled data. Three customers sign up next month. How do you assign them?

A) Fit a new scaler on the three customers, then `predict`

B) Pass the fitted scaler's `transform` of their rows to the fitted model's `predict`

C) Refit K-Means on all customers plus the three

D) `predict` on their raw, unscaled rows

Q11

What does this print?

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit([[2.0], [4.0], [6.0]])
print(scaler.inverse_transform([[1.5]]).round(3))
```

Q12

In Lab 8.1 A, the "Busy cards" group has a fraud rate of 2.13% and holds 243 of the 438 frauds; the clustering never saw `is_fraud`. Which statement is right?

A) Busy cards is a fraud detector: flag the whole group

B) The groups differ in fraud, which shows they capture real behavior, but 97.9% of busy-card payments are legitimate: a cluster describes, it does not detect

C) The clustering leaked the fraud labels

D) The groups are meaningless because none of them is mostly fraud

JOURNAL

Explain K-Means to a Pinecrest marketer in four sentences using the campfire picture, without the words "centroid" or "inertia". Then write the one piece of evidence from your project that makes you believe the five groups are more than an artifact of the algorithm, and one reason you still are not sure.

EXIT TICKET

Three questions, no notes.

E1

Name the two checks before you trust a LIME explanation, and how you run each.

E2

The churn model never sees age. Why audit by age anyway, and which metric fits "a churner misses out on an offer"?

E3

Why scale features before K-Means, and what does `n_init=10` do?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the campaign audit (Steps 3 to 5) and the customer groups (Steps 4 to 6)
- ☐ Lesson 7.3 Exercises 1 to 3; Lesson 8.1 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Lab 7.3 B TODO 3 and 4, if not done in class
- ☐ 8.1 "Before the Next Lesson": histograms of `avg_order_value` and `orders_12m`; the project with 4 and 6 clusters
- ☐ Bring to Session 3: a READY laptop, `customer_groups.csv` and my 8.1 notebook, this workbook and my journal

GLOSSARY

TERM	MEANING
LIME	Local Interpretable Model-agnostic Explanations: a simple surrogate fitted to the model's scores on weighted nearby variants
surrogate	The simple model that imitates the real one near one row
fidelity	How well the surrogate imitates the model in the neighborhood (`exp.score`, a weighted R^2)
stability	Whether an explanation survives a change of random seed
what-if / counterfactual	Change a value and ask the model again; the smallest change that flips the answer
demographic parity	Equal flag (selection) rates across groups
equal opportunity	Equal recall across groups: real positives equally likely to be flagged
permutation test	Shuffle group labels many times to see how large a gap chance alone produces
clustering	Grouping rows by similarity with no target column
centroid	The mean of a cluster's rows, in `cluster_centers_`
inertia	Total squared distance from each row to its own centroid
local minimum	A solution no single assign or move step can improve, though a better one exists

TERM	MEANING
k-means++	A start that spreads the initial centers out

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 3: Choosing k and Hierarchical Clustering

Your workbook for Session 3: a warm-up on Session 2, the key ideas of Lessons 8.2 and 8.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 8.2–8.3 | Session 3 · 3 hours | Name: _____

SESSION GOALS

- Choose k from four kinds of evidence (elbow, silhouette, stability, usefulness), and write the trade-off down.
- Compute a silhouette by hand and read it per cluster; compare two labelings with the adjusted Rand index.
- Build a tree of merges, read its linkage matrix and dendrogram, and cut it by height or by number.
- Pick a linkage and a distance that fit the question, and know when the pairwise cost rules the method out.

WARM-UP

Five questions on Session 2. Write first; we go through them together.

W1

Name LIME's two checks before you trust an explanation.

W2

Recall by age group ranges from 54% to 77% and a permutation test finds a gap this large in 47% of shuffles. Say the honest conclusion in one sentence.

W3

What does this print?

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[0], [4], [10], [14]])
print(km.inertia_)
```

W4

Why scale before K-Means?

W5

Yesterday cluster 3 was the discount segment; after a refit it is cluster 1. What happened?

LESSONS

8.2 Choosing k and Customer Segments

BY THE END YOU CAN

- Read an elbow plot and explain why inertia alone cannot choose k
- Compute and interpret silhouette scores, overall and per cluster, from the formula and with scikit-learn
- Test whether clusters are stable by refitting on resampled data and comparing labelings with the adjusted Rand index
- Build a reusable clustering pipeline (log transform, scaling, K-Means) and use `MiniBatchKMeans` when data gets large
- Profile, name and size segments, link them to an outcome such as churn, and write recommendations with honest limits

KEY IDEAS

- **No answer key.** Collect four kinds of evidence and look for agreement: elbow, silhouette, stability, usefulness.
- **Elbow.** Inertia always falls as k grows; read where the drops become small, never its minimum.
- **Silhouette.** s = (b - a) / max(a, b) per point; near 1 snug, near 0 on a border, negative probably misassigned. Look per cluster, not only the mean. It costs every pairwise distance: sample on big data.
- **Stability.** Refit on bootstrap samples and compare with the adjusted Rand index (same groups = 1, random = about 0, numbers ignored).
- **Prepare features.** Log right-skewed counts and money, then scale, all in one pipeline so new rows get the same fitted steps.
- **Segments.** Profile (an index of 100 = average), name, size, link to outcomes the clustering never saw, suggest actions. Segments describe; they do not explain or target.

WORKED EXAMPLE: THE ADJUSTED RAND INDEX BY PAIR COUNTING

The ARI drives the stability check, so it is worth seeing once at full size. Two labelings of six points: the second run uses other numbers and moved one point.

```
from sklearn.metrics import adjusted_rand_score, rand_score
import itertools

first  = [0, 0, 0, 1, 1, 1]
second = [5, 5, 7, 7, 9, 9]          # another run: other numbers, one point moved
pairs = list(itertools.combinations(range(6), 2))
agree = sum((first[a] == first[b]) == (second[a] == second[b]) for a, b in pairs)
print(f"pairs: {len(pairs)}, labelings agree on {agree}: Rand index {agree / len(p
print(f"rand_score {rand_score(first, second):.3f}   adjusted_rand_score {adjusted
```

```
pairs: 15, labelings agree on 10: Rand index 0.667
rand_score 0.667   adjusted_rand_score 0.242
```

Of the 15 pairs of points, the two labelings agree (both "together" or both "apart") on 10: a Rand index of 0.667. That sounds good, but random labelings also agree on many pairs, mostly "apart" ones. The ADJUSTED Rand index subtracts what chance would give:

```
import numpy as np
rng = np.random.default_rng(42)
random_aris = [adjusted_rand_score(first, rng.integers(0, 3, size=6)) for _ in ran
print(f"ARI of random labelings: mean {np.mean(random_aris):.3f}")
```

```
ARI of random labelings: mean -0.009
```

Random labelings score about 0, so 0.242 is modest agreement, and the 0.98 to 0.99 of the customer segments under resampling is very strong.

GUIDED LAB

- **Lab A (pairs):** `labs/module-03/l8_2-choosing-k/choose_k_tastes_starter.py`. Customers by WHAT they buy (activity shares): elbow, silhouette, stability, a profile. Then decide between the two best candidates and write why.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `segment_economics_starter.py` (usefulness in dollars), then `debug_choose_k.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P1

Predict the output, then run it. (Two tight pairs, far apart.)

```
from sklearn.metrics import silhouette_samples
print(silhouette_samples([[0], [1], [4], [5]], [0, 0, 1, 1]).round(2))
```

P2

Predict the output, then run it. (Same groups renamed, then a different grouping.)

```
from sklearn.metrics import adjusted_rand_score
print(adjusted_rand_score([0, 0, 1, 1], [1, 1, 0, 0]), adjusted_rand_score([0,
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The segment brief (lesson page). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: one pipeline, log + scale + K-Means. Check: sizes 1253, 1374, 1449, 811, 1113
- ☐ Step 2: the evidence. Check: `{4: 0.424, 5: 0.479, 6: 0.452}`
- ☐ Step 3: the index table and five names from it
- ☐ Step 4: the brief table with an action per segment
- ☐ Step 5: three newcomers assigned with `segmenter.predict`
- ☐ Step 6: `customer_segments.csv` and the printed brief

SELF-CHECK

Q1

Your loop prints inertia for k = 2 to 9. Why is "pick the k with the lowest inertia" useless?

A) Inertia is only defined for k = 5

B) Inertia can only fall as k grows, so its minimum is always the largest k you tried

C) Inertia rises after the true k, so its minimum is too small

D) Inertia is random, so its minimum changes every run

Q2

What does this print?

```
from sklearn.metrics import silhouette_samples
print(silhouette_samples([[0], [2], [10]], [0, 0, 1]).round(2))
```

Q3

Bootstrap refits at k = 3 give a worst ARI of 0.60, while k = 4 and 5 stay above 0.98. What does that say about three clusters?

A) Three is the best k because its ARI varies most

B) The ARI is broken for k = 3

C) Three clusters are too few to have a silhouette

D) Three is not a natural number of groups here: K-Means must make an arbitrary choice about what to merge, and resamples disagree

Q4

In Lab 8.2 B, a blanket retention offer loses money in four of five segments, and on the test customers the churn model's 300 offers earn $4,920 against -$7,620 for offering everyone. What is the lesson for the segment brief?

A) Segments describe who is together; choosing whom to call is the model's job

B) Segments are useless and should be dropped

C) Offer everyone in Deal Hunters and nobody else: that beats the model

D) Refit the segments with churn as a fifth column

Q5

What does this print?

```
from sklearn.metrics import adjusted_rand_score
a = [0, 0, 0, 1, 1, 1]
b = [3, 3, 3, 8, 8, 8]
print(adjusted_rand_score(a, b), round(adjusted_rand_score(a, [0, 0, 1, 1, 1,
```

Q6

For the activity-share clusters (Lab 8.2 A), the silhouette is 0.453 at k = 5 and 0.429 at k = 6, both are stable, and k = 6 adds a "general gear" group of 484 customers. What is the most sensible next step?

A) Choose 9, the k with the lowest inertia

B) Stop: the scores disagree, so the data has no clusters

C) Compare the profiles for 5 and 6 and choose the one whose groups lead to distinct actions

D) Average the two clusterings

JOURNAL

Write the paragraph you would put at the top of the segment brief explaining why five segments, citing at least three kinds of evidence from this lesson. Then name one decision marketing should not make from the

segments alone, and what they would need to make it safely.

8.3 Hierarchical Clustering

BY THE END YOU CAN

- Explain how agglomerative clustering builds a hierarchy by repeatedly merging the two closest groups
- Read a SciPy linkage matrix and a dendrogram, and cut the tree by number of clusters or by height
- Compare single, complete, average and Ward linkage and predict how each behaves on chains, blobs and outliers
- Use `AgglomerativeClustering` with the right metric and linkage, including non-Euclidean distances
- Judge when hierarchical clustering beats K-Means, and when its cost rules it out

KEY IDEAS

- **Agglomerative.** Start with every point alone; merge the two closest groups; record the height; n − 1 merges in all. No random start, no k up front.
- **Linkage matrix and dendrogram.** Each row: the two clusters merged, the distance, the new size. A horizontal cut gives flat clusters; a long empty stretch suggests a natural number.
- **Linkage.** Single (closest pair: chains), complete (farthest pair: compact, outlier-sensitive), average (all pairs: any distance), Ward (variance increase: round groups, Euclidean only).
- **Any distance.** `metric=` (or SciPy's `pdist`): correlation and cosine compare patterns, not volumes.
- **scikit-learn details.** `metric=` not `affinity=`; `n_clusters=None` with `distance_threshold`; no `predict`.
- **Cost.** Every pairwise distance: fine for thousands, impossible for millions. Sample, build the tree, assign the rest to the nearest mean.

WORKED EXAMPLE: COMPLETE AND SINGLE LINKAGE BY HAND

Four points on a line: 0, 4, 5 and 11. Work out the merges by hand for complete and for single linkage, then check.

```
import numpy as np
from scipy.cluster.hierarchy import linkage

points = np.array([[0.0], [4.0], [5.0], [11.0]])
print("complete:\n", linkage(points, method="complete").round(1))
print("single:\n", linkage(points, method="single").round(1))
```

```
complete:
 [[ 1.  2.  1.  2.]
 [ 0.  4.  5.  3.]
 [ 3.  5. 11.  4.]]
single:
 [[1. 2. 1. 2.]
 [0. 4. 4. 3.]
 [3. 5. 6. 4.]]
```

Both start by merging 4 and 5 (distance 1) into cluster 4. Then they disagree on HOW FAR the box {4, 5} is: complete linkage uses the farthest member, so 0 is 5 away and 11 is 7 away; 0 joins at 5, and 11 joins last at 11 (its distance to the far end, 0). Single linkage uses the nearest member: 0 joins at 4 and 11 at 6. Same merges in the same order here, very different heights: the cut that gives two clusters lies between 5 and 11 for complete, between 4 and 6 for single.

GUIDED LAB

- **Lab A:** `labs/module-03/l8_3-hierarchical/seasonal_families_starter.py`. The 24 products by WHEN they sell: a tree on monthly averages with correlation distance; cuts; Euclidean for contrast.
- **Lab B:** `labs/module-03/l8_3-hierarchical/sample_then_assign_starter.py`. Ward on a sample, everyone to the nearest mean, then K-Means polished from those means.
- Done when your numbers match the "Expected when done" lines. Finished early? `debug_hierarchical.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code.

P3

Predict the output, then run it. (Five points on a line, single linkage.)

```python
from scipy.cluster.hierarchy import linkage
Z = linkage([[0.0], [1.0], [2.0], [10.0], [30.0]], method="single")
print(len(Z), Z[:, 2].tolist())
```

P4

Predict the output, then run it. (Correlation distance compares shapes.)

```
from scipy.spatial.distance import pdist
print(pdist([[1, 0, 1, 0], [0, 1, 0, 1], [2, 0, 2, 0]], metric="correlation").
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A product family tree (lesson page). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: the product x customer matrix. Check: `(200, 3000)`
- ☐ Step 2: the tree with average linkage and correlation distance (199 merges)
- ☐ Step 3: the dendrogram cut at 0.97
- ☐ Step 4: five families against `primary_activity`
- ☐ Step 5: single linkage chaining, with sizes and ARI
- ☐ Step 6: family names and `product_families.csv`

SELF-CHECK

Q7

What does this print?

```
import numpy as np
from scipy.cluster.hierarchy import linkage
Z = linkage(np.array([[0.0], [1.0], [3.0], [10.0], [11.0], [30.0]]), method="s
print(len(Z), Z[-1, 3])
```

Q8

Lab 8.3 A clusters 24 products by their monthly sales. With Euclidean distance the tree puts the two best sellers on their own; with correlation distance it finds a winter family and a summer family. Why?

A) Euclidean distance is wrong for numbers between 0 and 100

B) Correlation distance ignores the months

C) Euclidean distance mostly measures volume (how many units), correlation distance compares the shape of the year

D) Average linkage only works with correlation

Q9

Which call cuts a tree at height 50 and lets the number of clusters follow?

A) `AgglomerativeClustering(n_clusters=None, distance_threshold=50, linkage="ward")`

B) `AgglomerativeClustering(n_clusters=5, distance_threshold=50, linkage="ward")`

C) `AgglomerativeClustering(affinity="euclidean", distance_threshold=50)`

D) `AgglomerativeClustering(n_clusters=50, linkage="ward")`

Q10

A dendrogram's highest merges are at 1.54, then 0.29, 0.28, 0.22. How many families does the tree suggest, and why?

A) Four, one per merge listed

B) Twenty-four, one per product

C) It depends only on the business

D) Two: the long empty stretch between 0.29 and 1.54 means any cut there gives the same two groups

Q11

What does this print?

```
import numpy as np
from scipy.spatial.distance import pdist
summer = [10, 20, 40, 20]          # units per season
winter = [40, 20, 10, 20]
big_summer = [100, 200, 400, 200]  # same shape as summer, ten times the volu
print(pdist(np.array([summer, winter, big_summer]), metric="correlation").rou
```

Q12

Why does Lab 8.3 B build the Ward tree on a sample of 500 customers and then assign everyone to the nearest cluster mean?

A) Ward cannot handle more than 500 rows

B) Agglomerative clustering needs every pairwise distance (18 million for 6,000, billions for 100,000) and has no predict, so: tree on a sample, nearest mean for the rest

C) A sample always gives better clusters than the full data

D) The nearest-mean step makes the clusters hierarchical

JOURNAL

Draw (on paper) the dendrogram of five things you own, merged by how similar they are, and mark where you would cut it for two different purposes. Then write two sentences on why correlation distance found the product families when Euclidean distance would have mostly measured popularity.

EXIT TICKET

Three questions, no notes.

E1

Name the four kinds of evidence for choosing k, and one thing each can tell you.

E2

Why did correlation distance find product families (by buyers, or by season) when Euclidean distance would not?

E3

When would you choose K-Means over agglomerative clustering?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the segment brief (Steps 4 to 6) and the product family tree (Steps 4 to 6)
- ☐ Lesson 8.2 Exercises 1 to 3; Lesson 8.3 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Lab 8.2 B (segment economics), if not done in class

- ☐ 8.3 "Before the Next Lesson": keep `product_families.csv`; the moons experiment; three one-sentence answers
- ☐ Image data: if my check said READY for Sessions 1 to 4, run it with `--download` before Session 5
- ☐ Bring to Session 4: a READY laptop, my 8.2 notebook and `customer_segments.csv`, this workbook and my journal

GLOSSARY

TERM	MEANING
elbow	The bend in the inertia curve where extra clusters stop paying off
silhouette	(b - a) / max(a, b) per point: cohesion against separation, from -1 to 1
adjusted Rand index (ARI)	Agreement between two labelings over pairs of points, corrected for chance; ignores cluster numbers
bootstrap	A sample of the same size drawn with replacement, to test stability
segment	A profiled, named cluster with a size, linked outcomes and an action
agglomerative clustering	Bottom-up merging of the two closest groups, n - 1 times
linkage matrix	SciPy's record of the merges: the two clusters, their distance, the new size
dendrogram	The tree drawing of the merges; a horizontal cut gives flat clusters
linkage	How the distance between two groups is defined: single, complete, average, Ward
chaining	Single linkage gluing groups together through a trail of close points
correlation distance	1 minus the correlation of two rows: 0 same pattern, 1 unrelated, 2 opposite
cophenetic correlation	How well the tree's merge heights preserve the original distances

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 4: Density-Based Clustering and Gaussian Mixtures

Your workbook for Session 4: a warm-up on Lessons 8.2 and 8.3, the key ideas of Lessons 8.4 and 8.5, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 8.4-8.5 | Session 4 · 3 hours | Name: _____

SESSION GOALS

- Explain density-based clustering with core, border and noise points, and choose eps from the data.
- Say why one radius fails when clusters have different densities, and use HDBSCAN instead.
- Count clusters and noise correctly, profile every cluster, and spot clusters that are artifacts of whole-number columns.
- Fit a Gaussian mixture, read its means, covariances and weights, and use probabilities instead of hard labels.
- Choose a covariance type and a number of components with BIC, and tell a borderline customer from an atypical one.

WARM-UP: CHOOSING K AND HIERARCHICAL CLUSTERING

Five questions from Lessons 8.2 and 8.3. Two minutes, no notes. Circle your answer before the class reveals it.

W1

What does this print?

```
from sklearn.metrics import adjusted_rand_score
a = [0, 0, 1, 1, 2, 2]
b = [2, 2, 0, 0, 1, 1]
print(adjusted_rand_score(a, b))
```

A) `0.0`

B) `0.333`

C) `1.0`

D) An error: the labels do not match

W2

Lesson 8.2 chose k for Pinecrest's customers. Which piece of evidence is NOT a reason to pick k?

A) Where inertia stops dropping steeply (the elbow)

B) A high mean silhouette, checked per cluster

C) Clusters that reappear when you refit on bootstrap samples

D) The k where inertia reaches its minimum

W3

What does this print?

```
from scipy.cluster.hierarchy import linkage
Z = linkage([[0.0], [1.0], [10.0], [11.0]], method="single")
print(Z[:, 2])
```

A) `[ 1. 1. 10.]`

B) `[1. 1. 9.]`

C) `[0. 1. 9.]`

D) `[ 1. 10. 11.]`

W4

Two groups of customers are joined by a thin trail of in-between customers. Which linkage merges them too early?

A) Single

B) Complete

C) Average

D) Ward

W5

Lesson 8.2 segmented customers on four columns with a log and a scaler. To segment next month's NEW customers, what do you reuse?

A) Only the K-Means centers

B) The whole fitted pipeline (log, scaler, K-Means): call predict on the new rows

C) A new scaler fitted on the new customers, then the old centers

D) Nothing: refit K-Means on the new customers

LESSONS

8.4 Density-Based Clustering

BY THE END YOU CAN

- Explain core, border and noise points, and what eps and min_samples control in DBSCAN
- Choose a starting eps from a k-distance plot and read the effect of changing it
- Read an OPTICS reachability plot and explain why one eps fails when clusters have different densities
- Cluster with HDBSCAN, count clusters and noise correctly, and profile the result
- Judge whether a density-based clustering found real structure or an artifact of the features

KEY IDEAS

- **A city at night.** A cluster is a crowded region separated by sparser space; points in sparse space are noise, label -1. The number of clusters comes out of the data.
- **DBSCAN's two settings.** `eps` is a radius; `min_samples` is how many points (itself included) make a neighborhood crowded. Core, border, noise.
- **Choosing eps.** The k-distance plot: each point's distance to its min_samples-th neighbor, sorted. Start just above the knee, then sweep a few values.
- **One radius fails** when clusters have different densities: small eps loses loose groups, large eps merges tight ones.
- **OPTICS** draws a reachability plot (clusters are valleys); **HDBSCAN** keeps the clusters stable across densities and needs mainly `min_cluster_size` (pass `copy=True`).
- **Count clusters without -1**, scale first, and remember these models have no `predict`.
- **Profile every cluster.** Whole-number columns stack points on exact values and create dense "clusters" that are artifacts.

WORKED EXAMPLE: ROLES IN A CROWD AND A TRICKLE

A crowd of 300 points and a trickle of 40 scattered around it. How do the roles change when DBSCAN asks for a bigger crowd?

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN

rng = np.random.default_rng(7)
crowd, _ = make_blobs(n_samples=300, centers=[[0, 0]], cluster_std=0.5, random_sta
trickle = rng.uniform(-4, 4, size=(40, 2))          # scattered points around the
X = np.vstack([crowd, trickle])

for min_samples in [5, 15]:
    db = DBSCAN(eps=0.4, min_samples=min_samples).fit(X)
    core = np.zeros(len(X), dtype=bool)
    core[db.core_sample_indices_] = True
    noise = db.labels_ == -1
    print(f"min_samples={min_samples:<3} clusters={len(set(db.labels_) - {-1})}  "
          f"core={core.sum()}  border={(~core & ~noise).sum()}  noise={noise.sum()
```

```
min_samples=5   clusters=1  core=294  border=5  noise=41
min_samples=15  clusters=1  core=276  border=20  noise=44
```

With `min_samples=5`, 294 points are core and only 5 are border points: almost everyone in the crowd is crowded enough. Asking for 15 neighbors makes the fringe of the crowd thinner than the rule: 18 fewer core points, four times as many border points, and a few more noise points. The cluster is the same; the roles moved. A larger `min_samples` is a stricter definition of "crowded", which is why eps and min_samples must be chosen together.

GUIDED LAB

- Open `labs/module-04/l8_4-density-clustering/payment_density_starter.py` and run it once as it is.
- TODO 1 and 2 (together): k-distances and an eps sweep. Is there one eps you would trust?
- TODO 3 to 5 (pairs): HDBSCAN, a profile table in original units, and the fraud rate in the noise against the clusters. The fraud label is never a feature.
- TODO 6: does the finding survive other values of `min_cluster_size`?
- Done when your numbers match the "Expected when done" lines. Finished early? `rings_starter.py`, then `debug_density.py`.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Which points have two neighbors within 0.25?)

```
import numpy as np
from sklearn.cluster import DBSCAN
pts = np.array([[1.0], [1.2], [1.4], [3.0], [3.1], [3.2], [9.0]])
print(DBSCAN(eps=0.25, min_samples=3).fit_predict(pts))
```

P2

Predict the output, then run it. (Count labels; then the share of -1.)

```
import numpy as np
labels = np.array([0, 0, -1, 1, 1, 2, -1, 2])
print(len(set(labels)), (labels == -1).mean())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Core Segments and Misfits (lesson page). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: logs and scaling. Check: `(6000, 4)`, means 0, standard deviations 1
- ☐ Step 2: k-distances `[0.28 0.41 0.6 ]`; no eps gives a usable answer
- ☐ Step 3: HDBSCAN with `min_cluster_size=100, min_samples=25`: 7 clusters, 32.3% noise
- ☐ Step 4: profile table with churn; spot the two pairs that differ only by a whole-number order count
- ☐ Step 5: merged segment names; misfits churn about 0.203
- ☐ Step 6: `customer_density_segments.csv`, 6,000 rows

SELF-CHECK

Q1

What does this print?

```
import numpy as np
from sklearn.cluster import DBSCAN
pts = np.array([[0.0], [1.0], [2.0], [10.0], [11.0], [30.0]])
print(DBSCAN(eps=1.5, min_samples=3).fit_predict(pts))
```

A) `[0 0 0 1 1 2]`

B) `[ 0 0 0 1 1 -1]`

C) `[ 0 0 0 -1 -1 -1]`

D) `[-1 -1 -1 -1 -1 -1]`

Q2

You compute each point's distance to its 10th nearest neighbor (itself included), sort the distances and plot them. What is the plot for?

A) Choosing the number of clusters

B) Choosing a starting eps for DBSCAN with min_samples=10: just above the knee

C) Choosing `min_samples`: the height of the knee

D) Checking whether the silhouette is high enough

Q3

What does this print?

```
import numpy as np
labels = np.array([0, 0, 1, -1, 1, -1, 2])
print(len(set(labels)), len(set(labels) - {-1}))
```

Q4

You switch from DBSCAN to HDBSCAN on a new data set. Which setting do you choose first, and from what?

A) `eps`, from the knee of the k-distance plot

B) `n_clusters`, from the elbow of the inertia curve

C) `min_cluster_size`, from the smallest group you would act on, then check that nearby values agree

D) `xi`, from the steepest wall in the reachability plot

Q5

HDBSCAN on card payments returns clusters that each hold payments with exactly 1, 2, 3 or 4 items. What is the best reading?

A) Four genuinely different kinds of shopper, sorted by basket size

B) The items column needs more weight than the others

C) The features were not scaled, so items dominated

D) items only takes whole values, so payments stack on exact values and look dense: profile further and treat the split as an artifact unless the business cares about it

Q6

What does this print?

```
from sklearn.cluster import HDBSCAN
print(hasattr(HDBSCAN(), "fit_predict"), hasattr(HDBSCAN(), "predict"))
```

A) `True True`

B) `True False`

C) `False True`

D) `False False`

Explain core, border and noise points with the city-at-night picture. Then write two sentences for Pinecrest's marketing lead about the misfit customers: how many there are, how they behave, and why "they fit no segment" is a finding rather than a failure. Finally, note one decision in this lesson that was a judgment call rather than a calculation.

8.5 Gaussian Mixture Models

BY THE END YOU CAN

- Explain a Gaussian mixture as a weighted sum of components with means, covariances and weights
- Fit a mixture with EM and read predict_proba as soft cluster membership
- Choose the covariance type and the number of components with BIC and AIC
- Compare a mixture with K-Means on elongated or overlapping clusters
- Use a fitted mixture to score how typical a point is and to generate new samples

KEY IDEAS

- **Paint from base colors.** A mixture is a weighted sum of Gaussian components, each with a mean (`means_`), a covariance (`covariances_`) and a weight (`weights_`).
- **EM** alternates soft assignments (E-step) and refitting (M-step). It finds a good answer, not always the best: use `n_init` and `random_state`.
- **Soft membership.** `predict_proba` rows add up to 1; keep the top probability as a confidence instead of stopping at `predict`.
- **Ellipses, not circles.** Full or tied covariances follow stretched groups K-Means cuts across. Compare covariance types with BIC (lower is better).
- **How many components?** Read where the big BIC drops stop; on real data BIC often keeps creeping down.
- **Numbers are arbitrary.** Name components by their profile; match runs with ARI or a crosstab.
- **A density model.** `score_samples` scores how typical a point is (the bridge to anomaly detection); `sample` generates synthetic data, only as realistic as the Gaussian assumption.

WORKED EXAMPLE: BORDERLINE IS NOT THE SAME AS ATYPICAL

Two columns of fractions (discount share and return rate), four components. Which customers does the mixture find least typical, and how sure is it about them?

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture

customers = pd.read_csv("data/customers_churn.csv")
cols = ["discount_share", "return_rate"]
Z = StandardScaler().fit_transform(customers[cols])
gm = GaussianMixture(n_components=4, n_init=3, random_state=42).fit(Z)

customers["log_density"] = gm.score_samples(Z)
customers["confidence"] = gm.predict_proba(Z).max(axis=1)
print("weights:", np.sort(gm.weights_).round(3))
print(f"confidence below 0.6: {(customers['confidence'] < 0.6).sum()} customers")
```

```
weights: [0.14  0.187 0.276 0.397]
confidence below 0.6: 396 customers
```

```
print(customers.nsmallest(4, "log_density")[cols + ["log_density", "confidence"]].
```

```
      discount_share  return_rate  log_density  confidence
880            0.576        0.332      -11.365       0.995
2966           0.593        0.261       -9.868       0.839
4834           0.544        0.289       -9.751       0.920
4818           0.490        0.418       -9.551       1.000
```

The least typical customers combine a high discount share with a high return rate, a mix no component describes well, so their log-density is the lowest. Yet look at the confidence column: the mixture is 0.84 to 1.00 sure which component each belongs to. Confidence answers "which component is nearest?"; log-density answers "is this customer like any of them?". Borderline and atypical are different questions.

GUIDED LAB

- **Lab A:** `labs/module-04/l8_5-gaussian-mixtures/discount_mixture_starter.py`. One real column, discount share: how many bell curves does BIC see, where does the model switch its vote, and how does soft membership line up with churn?
- **Lab B (pairs):** `labs/module-04/l8_5-gaussian-mixtures/lifecycle_mixture_starter.py`. Lifecycle stages from tenure, recency and spend; two seeds; a brand-new customer.
- Never assume a component number: find the one you want from its mean or profile.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Three values near 0, two near 8.)

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.2], [0.4], [8.0], [8.2]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(np.sort(gm.weights_).round(1))
```

P4

Predict the output, then run it. (What does each row of predict_proba add up to?)

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.2], [0.4], [8.0], [8.2]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(gm.predict_proba([[4.1], [100.0]]).sum(axis=1).round(2))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Soft Customer Segments (lesson page). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: scale the four raw features (not the logs: read the warning card). Check: `(6000, 4)`
- ☐ Step 2: BIC and AIC for 1 to 8 components; the big drops stop at 5
- ☐ Step 3: five components, profiled and named by profile
- ☐ Step 4: 305 customers below 0.8 confidence, with their runner-up segment
- ☐ Step 5: what a sixth component adds (and whether you would keep it)

- ☐ Step 6: two new customers scored; `customer_soft_segments.csv`, 6,000 rows

SELF-CHECK

Q7

What does this print?

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.1], [5.0], [5.1], [5.2]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(gm.predict_proba([[100.0]]).max().round(2))
```

A) `0.0`

B) `0.5`

C) `nan`

D) `1.0`

Q8

BIC for 1 to 6 components is 9,800, 8,100, 7,950, 7,930, 7,920 and 7,915. What is the most sensible choice?

A) About 2 or 3: the big drops stop there; then check whether the third component means something

B) 6, because BIC is lowest there

C) 1, because BIC only rewards simple models

D) None: BIC needs a minimum to be usable

Q9

Three groups were made by stretching three identical round blobs with the same matrix. Which covariance_type should BIC prefer?

A) `"spherical"`

B) `"diag"`

C) `"tied"`

D) `"full"`

Q10

What does this print, and what does it tell you?

```
from sklearn.metrics import adjusted_rand_score
run_a = [0, 0, 1, 1, 2, 2]
run_b = [1, 1, 2, 2, 0, 0]
print(adjusted_rand_score(run_a, run_b))
```

Q11

A customer sits far from every component, yet their largest predict_proba value is 0.99. Which output flags them as unusual?

A) `score_samples`: a very low log-density

B) `predict`: it returns -1 for outliers

C) `weights_`: the component's share is small

D) `means_`: the customer equals no mean

Q12

What happens when this runs?

```
from sklearn.mixture import GaussianMixture
GaussianMixture(n_components=4, random_state=42).fit([[0.0], [1.0], [2.0]])
```

JOURNAL

Explain the paint-palette picture in your own words, then describe the difference between a borderline customer (low confidence) and an atypical customer (low density) with one example of each from your project. Which of the two would you want a marketing team to know about, and why?

EXIT TICKET

Three questions, no notes.

E1

HDBSCAN returns the labels -1, 0, 1, 2 and 30% of the points are -1. How many clusters did it find, and what do you tell the marketing team about the 30%?

E2

Name one thing a Gaussian mixture can do that HDBSCAN cannot, and one thing HDBSCAN can do that a mixture cannot.

E3

A customer far from every segment gets predict_proba 0.99 for one component. Is the model sure this is a typical member? What would you check?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish Core Segments and Misfits (Steps 3 to 6) and Soft Customer Segments (Steps 3 to 6)
- ☐ Lesson 8.4 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 8.5 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ 8.5 "Before the Next Lesson": look at `transactions.csv`; unusual as good news and as bad news
- ☐ Optional: `lifecycle_mixture_starter.py`, `rings_starter.py`, `debug_density.py`
- ☐ Bring to Session 5: a ready laptop, `pinecrest-ml` with `data/` and `labs/`, both project notebooks and their CSV files, this workbook and my journal

GLOSSARY

TERM	MEANING
eps	DBSCAN's neighborhood radius
min_samples	Points (itself included) that make a neighborhood crowded

TERM	MEANING
core / border / noise point	Crowded itself / next to a core point / neither (label -1)
k-distance plot	Sorted distances to the min_samples-th neighbor; its knee suggests eps
reachability plot	OPTICS's picture: points in visiting order, clusters as valleys
HDBSCAN	Keeps the clusters that are stable across density levels; main setting min_cluster_size
Gaussian mixture	A weighted sum of bell-shaped components
component	One Gaussian: a mean, a covariance (an ellipse), a weight
EM	Expectation-maximization: alternate soft assignments and refits until the log-likelihood stops improving
covariance_type	spherical, diag, tied or full: how flexible each component's shape is
BIC / AIC	Fit minus a penalty per parameter; lower is better; AIC penalizes less
log-density	`score_samples`: how likely a point is under the whole model; low = atypical
label switching	The same clustering with the components numbered differently

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 5: Anomaly Detection and PCA

Your workbook for Session 5: a warm-up on Lessons 8.4 and 8.5, the key ideas of Lessons 8.6 and 9.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 8.6-9.1 | Session 5 · 3 hours | Name: _____

SESSION GOALS

- Build an anomaly score from normal data and use it as a review queue, not a verdict.
- Read the detectors' conventions correctly: -1 for outliers, score_samples higher for normal, contamination only a threshold.
- Judge a detector with AUC, average precision and catches within a budget, on a time split, against a rule.
- Run PCA on scaled data, choose how many components to keep, and name them from their loadings.
- Test whether PCA helps a model, and use reconstruction for compression and anomaly scores.

WARM-UP: DENSITY METHODS AND MIXTURES

Five questions from Lessons 8.4 and 8.5. Two minutes, no notes.

W1

What does this print?

```
import numpy as np
from sklearn.cluster import DBSCAN
pts = np.array([[0.0], [0.1], [0.2], [4.0], [4.1], [4.2], [9.0]])
labels = DBSCAN(eps=0.15, min_samples=2).fit_predict(pts)
print(labels, len(set(labels) - {-1}))
```

A) `[0 0 0 1 1 1 2] 3`

B) `[ 0 0 0 1 1 1 -1] 2`

C) `[ 0 0 0 1 1 1 -1] 3`

D) `[-1 -1 -1 -1 -1 -1 -1] 0`

W2

Why can you not score next month's customers with a fitted HDBSCAN?

A) It forgets its clusters after fit

B) It has fit_predict but no predict: it describes the data it saw

C) It needs the churn label

D) It only works on two features

W3

What does this print?

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.3], [0.5], [6.0], [6.4]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(gm.predict_proba([[50.0]]).round(2).sum())
```

A) `0.0`

B) `0.5`

C) `1.0`

D) A very small number

W4

BIC for 1 to 6 components: 50,600; 47,200; 46,700; 46,300; 46,100; 46,050. Which reading is right?

A) 6, the minimum

B) The big drop is from 1 to 2; after that gains shrink: choose 2 to 4 and defend it with profiles

C) BIC cannot be used without a minimum

D) 1: simpler is always better

W5

A customer has probabilities 0.97 for one component but the lowest log-density in the data. Borderline, atypical, or both?

A) Borderline

B) Atypical

C) Both

D) Neither

8.6 Anomaly Detection

BY THE END YOU CAN

- Tell point, contextual and collective anomalies apart and choose between supervised and unsupervised detection
- Apply simple statistical rules (z-score, interquartile range) as a baseline
- Score anomalies with Isolation Forest and Local Outlier Factor, and read the -1/1 and score conventions correctly
- Choose a threshold from a review budget instead of tuning contamination
- Evaluate a detector with ROC AUC, average precision and catches within a budget on a time-based split

KEY IDEAS

- **The night guard.** Learn what normal looks like, score how far each case is from it, review as many top scores as you can afford.
- **Three kinds:** point, contextual, collective. Good features (per-category amounts, velocity) turn the last two into the first.
- **Rules** (z-score, IQR fence) are transparent baselines that see one column at a time.
- **Isolation Forest:** random cuts isolate odd points quickly. **LOF:** compares a point's density with its neighbors'; needs scaling; `novelty=True` for new data.
- **Conventions:** `predict` gives -1 (outlier) and 1; `score_samples` is higher for NORMAL points: negate it.
- **contamination** only sets the threshold; choose the threshold from a review budget and costs.
- **Judge** with ROC AUC, average precision and catches in the budget, on a time split, against a rule. Unusual is not the same as bad.

WORKED EXAMPLE: TWO RULES ON A SKEWED COLUMN

200 ordinary order amounts plus three huge ones. Which rule notices what?

```
import numpy as np
rng = np.random.default_rng(42)
amounts = np.append(rng.lognormal(4.8, 0.5, size=200).round(2), [2400.0, 2600.0, 3

z = (amounts - amounts.mean()) / amounts.std(ddof=1)
q1, q3 = np.percentile(amounts, [25, 75])
fence = q3 + 1.5 * (q3 - q1)
print(f"mean {amounts.mean():.1f}, std {amounts.std(ddof=1):.1f} (without the thre
print(f"z-score above 3: {(z > 3).sum()} flagged; smallest flagged {amounts[z > 3]
print(f"IQR fence ${fence:.0f}: {(amounts > fence).sum()} flagged; smallest flagge
```

```
mean 170.3, std 319.3 (without the three: 65.2)
z-score above 3: 3 flagged; smallest flagged 2400
IQR fence $278: 9 flagged; smallest flagged 288
```

The three huge orders inflate the standard deviation from 65 to 319, so the z-score rule sees only those three: everything else looks ordinary next to them. The IQR fence uses quartiles, which the extremes cannot drag, so it flags the three plus six genuinely high ordinary orders from the long tail. Neither is "right": the z-score is easily blinded by the outliers it is looking for, and the fence flags a skewed tail. That is why rules are a baseline.

GUIDED LAB

- **Lab A:** `labs/module-05/l8_6-anomaly-detection/monthly_queue_starter.py`. Fit once on January to March; score April, May, June with a 2% queue; watch a fixed threshold drift.
- **Lab B (pairs):** `labs/module-05/l8_6-anomaly-detection/unusual_customers_starter.py`. The 60 most unusual customers by two detectors: do they agree, and do they churn?
- Check the sign once: higher must mean more unusual.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P1

Predict the output, then run it. (Which way do the scores point?)

```
from sklearn.metrics import roc_auc_score
print(roc_auc_score([0, 0, 0, 1], [0.9, 0.7, 0.8, 0.1]))
```

P2

Predict the output, then run it. (Which value is an outlier?)

```
import numpy as np
pred = np.array([1, 1, -1, 1, 1, 1, -1, 1])
print((pred == 1).sum(), (pred == -1).sum())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Fraud Review Queue (lesson page). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: time split; June 8,262 payments, 64 frauds, budget 165
- ☐ Step 2: `review_report`; amount rule 8 caught, velocity rule 26
- ☐ Step 3: Isolation Forest without labels: 32 of 64
- ☐ Step 4: LOF (16) and a Gaussian mixture (24)
- ☐ Step 5: supervised boosting (41) and the overlap of the two queues
- ☐ Step 6: review budgets from costs; false alarms for new vs established accounts

SELF-CHECK

Q1

What does this print?

```
import numpy as np
from sklearn.ensemble import IsolationForest
rng = np.random.default_rng(42)
X = np.vstack([rng.normal(0, 1, size=(49, 2)), [[8.0, 8.0]]])
pred = IsolationForest(contamination=0.02, random_state=42).fit(X).predict(X)
print(np.unique(pred, return_counts=True))
```

A) `(array([0, 1]), array([ 1, 49]))`

B) `(array([-1, 1]), array([ 1, 49]))`

C) `(array([-1, 1]), array([49, 1]))`

D) `(array([1]), array([50]))`

Q2

Your detector's ROC AUC on June is 0.12, computed from score_samples. What is the most likely explanation?

A) The detector ranks frauds worse than random: drop it

B) contamination is too low

C) Too few trees in the forest

D) score_samples is a normality score (higher = more normal); negate it and the AUC is 0.88

Q3

What does this print?

```
import numpy as np
from sklearn.ensemble import IsolationForest
X = np.random.default_rng(3).normal(size=(200, 2))
for c in [0.01, 0.05]:
    model = IsolationForest(contamination=c, random_state=42).fit(X)
    print(c, (model.predict(X) == -1).sum())
```

Q4

An Isolation Forest with contamination=0.02 fitted on January to March flags 2.16% of March but 1.72% of June. Why, and what does the lesson recommend?

A) The forest was refitted on June; fix the random_state

B) June's payments are distributed differently from the fit months, so a fixed threshold drifts: rank each month and review the top N the team can handle

C) contamination is a random setting; average several runs

D) Rounding: 2% of a smaller month is fewer payments

Q5

In today's lab, the Isolation Forest's 60 most unusual customers churn at 0.067, well below the 0.165 average. What is the right conclusion?

A) The detector is broken: anomalies should churn more

B) Unusual customers are loyal; target them with fewer offers

C) An anomaly score finds the unusual, not the bad: these are heavy buyers; whether unusual means a problem is a separate question

D) Use the LOF's top 60 instead, because they churn at 0.333

Q6

You fit Local Outlier Factor on May's payments and want to score June's. Which setup works?

A) `LocalOutlierFactor()`, then `predict(june)`

B) `LocalOutlierFactor()`, then `fit_predict(june)`

C) `LocalOutlierFactor(novelty=True)` fitted on May, then `-score_samples(june)`

D) `LocalOutlierFactor(novelty=True)` fitted on May, then `score_samples(may)`

JOURNAL

Explain the night guard in your own words, then write the three numbers from your project you would show the payments manager, and why those three. Finally: most payments in your best queue were honest. How would you explain to a customer why their order was held?

9.1 Principal Component Analysis

BY THE END YOU CAN

- Explain what a principal component is and why scaling matters
- Choose the number of components from explained variance
- Interpret loadings to name what each component captures
- Test whether PCA before a model helps, inside a pipeline
- Use reconstruction for compression, denoising and anomaly scores

KEY IDEAS

- **The flashlight.** PCA turns the cloud until its shadow is widest; each later component is the widest direction left, at a right angle.
- **Unsupervised, mixed, linear.** It never sees the target; each component mixes all columns; directions are straight.
- **Scale first** when units differ; do not standardize columns that share one meaningful unit.
- **How many?** Scree elbow, a variance target (`n_components=0.90`), or cross-validation for a model.
- **Loadings** (`components_`) name a component; signs are arbitrary.
- **Most variance is not most useful:** on churn, two components lost much of the signal; test PCA inside a pipeline.
- **Reconstruction** (`inverse_transform`) compresses, denoises and gives an anomaly score.

WORKED EXAMPLE: THREE COLUMNS THAT ALL MEAN "HOW MUCH"

Three scaled churn columns that all measure "how much": orders, spend, returns. What does PCA make of them?

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

customers = pd.read_csv("data/customers_churn.csv")
cols = ["orders_12m", "spend_12m", "returns_12m"]
Z = StandardScaler().fit_transform(customers[cols])
pca = PCA().fit(Z)
print("share of variance:", pca.explained_variance_ratio_.round(3))
print(pd.DataFrame(pca.components_.T, index=cols, columns=["PC1", "PC2", "PC3"]).r
```

```
share of variance: [0.624 0.239 0.137]
               PC1   PC2   PC3
orders_12m    0.60 -0.48 -0.64
spend_12m     0.63 -0.22  0.75
returns_12m   0.50  0.85 -0.17
```

PC1 weights all three positively and about equally: "how much this customer buys", 62% of the variance. PC2 sets returns against orders: customers who return a lot for what they buy. PC3 (14%) sets spend against orders: big tickets versus many small orders. Three correlated columns, one dominant shared direction plus two smaller contrasts.

GUIDED LAB

- **Lab A:** `labs/module-05/l9_1-pca/payment_pca_starter.py`. Unscaled vs scaled PCA on payments; name PC1 and PC2; where is the fraud?
- **Lab B (pairs):** `labs/module-05/l9_1-pca/daily_sales_pca_starter.py`. Days × four stores: the chain's day, and what PC2 follows.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P3

Predict the output, then run it. (Rows are components, columns are features.)

```python
import numpy as np
from sklearn.decomposition import PCA
rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5))
print(PCA(n_components=2).fit(X).components_.shape)
```

P4

Predict the output, then run it. (All three columns are multiples of x.)

```
import numpy as np
from sklearn.decomposition import PCA
x = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
X = np.column_stack([x, 3 * x, -x])
print(PCA().fit(X).explained_variance_ratio_.round(2))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Seasonal Fingerprints (lesson page). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: `(96, 24)` monthly table
- ☐ Step 2: log1p, remove each row's mean (do not standardize: why?)
- ☐ Step 3: PC1 explains 0.799
- ☐ Step 4: the PC1 wave over 24 months
- ☐ Step 5: seasonal score per product and category
- ☐ Step 6: one history rebuilt from PC1; `seasonal_scores.csv`

SELF-CHECK

Q7

What does this print?

```
import numpy as np
from sklearn.decomposition import PCA
rng = np.random.default_rng(42)
a, b = rng.normal(size=200), rng.normal(size=200)
X = np.column_stack([a, b, a + b])
print(PCA().fit(X).explained_variance_ratio_.round(3))
```

A) `[0.333 0.333 0.333]`

B) `[1. 0. 0.]`

C) `[0.5 0.5 0. ]`

D) `[0.732 0.268 0. ]`

Q8

Unscaled PCA on the nine payment features gives a PC1 that explains 95% of the variance and is almost exactly account_age_days. Why?

A) Account age is the most important fraud signal

B) Account age is measured in days, up to about 7,000, so its variance dwarfs the 0/1 flags and small counts

C) PCA always puts the first column first

D) The other columns are categorical

Q9

What does this print?

```
import numpy as np
from sklearn.decomposition import PCA
X = np.random.default_rng(1).normal(size=(50, 6))
p = PCA(n_components=3).fit(X)
print(p.transform(X).shape, p.components_.shape)
```

Q10

In which situation is PCA before a model most likely to help?

A) Ten business columns that each mean something, with the strongest signal in a low-variance column

B) Four categorical columns, one-hot encoded

C) A target that depends on one column PCA spreads over several components

D) 784 highly correlated pixel columns feeding a slow k-nearest-neighbors model

Q11

On June's payments, PCA reconstruction error reached AUC 0.819 but only 19 frauds in the 165-payment queue; Isolation Forest caught 32. What explains the gap?

A) Much of this fraud is extreme on a few columns rather than breaking the correlations most payments share

B) PCA leaked the labels

C) Four components were too many

D) Reconstruction error cannot be used as a score

Q12

On the ten scaled churn features, how many components does `PCA(n_components=0.90)` keep, and is that a useful reduction?

JOURNAL

PCA compressed 24 months of sales into one number per product almost perfectly, but it could not compress ten churn columns without losing the churn signal. Write down what was different about the two tables. Then name one dataset where you would expect PCA to compress well, and one where you would not.

EXIT TICKET

Three questions, no notes.

E1

An Isolation Forest's predict returns -1 and 1, and score_samples gives an AUC of 0.2. Write the two lines you would use instead to build a review queue of 165.

E2

Your manager asks you to raise contamination from 0.01 to 0.05 "so the detector catches more fraud". What do you explain?

E3

You run PCA on ten customer columns without scaling and PC1 explains 92%. What happened, and when would skipping the scaler be right?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish A Fraud Review Queue (Steps 3 to 6) and Seasonal Fingerprints (Steps 4 to 6)
- ☐ Lesson 8.6 Exercises 1 to 3; Lesson 9.1 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Download Fashion-MNIST for Session 6 (the command is in the bring list) and check that `import umap` works
- ☐ Optional: the second lab of each block, `debug_anomaly.py`
- ☐ Bring to Session 6: a ready laptop, Fashion-MNIST in `data/`, my Module 7 to 9 notebooks (Checkpoint 1 is open notebook), this workbook and my journal

GLOSSARY

TERM	MEANING
anomaly score	How far a case is from normal; higher = more unusual
point / contextual / collective anomaly	Unusual alone / only in context / only as a group
Isolation Forest	Random cuts; short average path = anomaly

TERM	MEANING
Local Outlier Factor	A point's density compared with its neighbors'; `novelty=True` scores new data
contamination	Expected share of outliers; sets only the threshold (`offset_`)
average precision	Area under the precision-recall curve; rewards frauds at the very top
review budget	How many top scores the team can check; the real threshold
principal component	A direction of maximum remaining variance; a mix of all columns
explained variance ratio	A component's share of the total variance
loadings	The weights in `components_` ; read the large ones, ignore the sign
scree plot	Variance share per component; look for an elbow
reconstruction error	The gap between a row and its rebuild from the kept components

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 6: LDA, t-SNE and UMAP

Your workbook for Session 6: a warm-up on Lessons 8.6 and 9.1, the key ideas of Lessons 9.2 and 9.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, Checkpoint 1 and your exit ticket.

Lessons 9.2–9.3 | Session 6 · 3 hours | Name: _____

SESSION GOALS

- Explain how LDA uses the labels to find separating directions, and how that differs from PCA.
- Use LDA as a reducer and as a classifier, and keep it inside the pipeline.
- Make t-SNE and UMAP maps, and read them by the five rules.
- Check a map with numbers, and place new rows with UMAP.
- Show what you can do with Modules 7 to 9 in Checkpoint 1.

WARM-UP: ANOMALY DETECTION AND PCA

Five questions from Lessons 8.6 and 9.1. Two minutes, no notes.

W1

What does this print?

```
import numpy as np
pred = np.array([1, -1, 1, 1, -1, 1])
print((pred == -1).sum(), (pred == 1).mean().round(2))
```

A) `4 0.33`

B) `2 0.67`

C) `2 0.33`

D) `4 0.67`

W2

You raise an Isolation Forest's contamination from 0.01 to 0.05. What changes?

A) The scores

B) The ranking

C) Only the threshold: more rows get -1, in the same order

D) The trees

W3

What does this print?

```
import numpy as np
from sklearn.decomposition import PCA
x = np.arange(6.0)
X = np.column_stack([x, 2 * x + 1])
print(PCA().fit(X).explained_variance_ratio_.round(2))
```

A) `[0.5 0.5]`

B) `[0.67 0.33]`

C) `[1. 0.]`

D) `[0.8 0.2]`

W4

Logistic regression on the first two principal components of the churn columns scored AUC 0.687, against 0.829 on all ten. Why?

A) PCA leaked

B) PCA picks directions of spread, not of churn; the recency signal is spread over several components

C) Two features are too few for any model

D) The features were not scaled

W5

When should you NOT standardize before PCA?

A) Never

B) When columns have different units

C) When all columns share one meaningful unit, like monthly sales or pixels

D) When there are many rows

LESSONS

9.2 Linear Discriminant Analysis

BY THE END YOU CAN

- Explain how LDA differs from PCA: it uses the labels to find directions that separate classes
- Use LDA to reduce dimensions for a plot or as a step before another model
- Use LDA as a classifier and compare it with logistic regression
- Know the limit on components (classes minus one) and when shrinkage helps
- Keep LDA inside the pipeline so cross-validation stays honest

KEY IDEAS

- **The team photo.** PCA keeps the widest spread; LDA keeps the class means far apart relative to the spread inside each class.
- **At most classes minus one axes:** one for churn, two for three tiers.
- `scalings_` are the axis weights; `explained_variance_ratio_` is the share of class separation.
- **As a classifier** LDA draws straight boundaries and often matches logistic regression; QDA allows curves but needs far more data.
- **Priors** move the threshold, not the ranking. **Shrinkage** (`lsqr` or `eigen`) for many features and few rows.
- **Inside the pipeline.** A supervised reducer fitted before cross-validation can separate pure noise.

WORKED EXAMPLE: ONE NUMBER AS A CHURN RISK SCORE

One number per customer as a churn risk score: the LDA axis against the first principal component, each cut into ten deciles on the test customers.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "days_s
            "returns_12m", "return_rate", "support_tickets_12m", "discount_share",
X_train, X_test, y_train, y_test = train_test_split(customers[features], customers
                                                    test_size=0.25, stratify=custo
ld1 = make_pipeline(StandardScaler(), LinearDiscriminantAnalysis()).fit(X_train, y
pc1 = make_pipeline(StandardScaler(), PCA(n_components=1)).fit(X_train).transform(

for name, score in [("LD1", ld1), ("PC1", -pc1)]:          # PC1 negated: low acti
    decile = pd.qcut(pd.Series(score).rank(method="first"), 10, labels=range(1, 11
    rates = pd.Series(y_test.to_numpy()).groupby(decile, observed=True).mean()
    print(f"{name}: churn in the lowest decile {rates.iloc[0]:.3f}, highest decile
```

```
LD1: churn in the lowest decile 0.013, highest decile 0.580
PC1: churn in the lowest decile 0.080, highest decile 0.393
```

The riskiest tenth by LD1 churned at 58%, the safest at 1.3%: a 45-fold spread from ONE number. PC1 (negated so that low activity means risk) spreads churn far less, from 8% to 39%. Both are a single straight direction through the same ten columns; only one was chosen with the labels.

GUIDED LAB

- **Lab A:** `labs/module-06/l9_2-lda/payment_category_lda_starter.py`. Ten categories from nine payment columns: how many axes, what LD1 is, how well it classifies.
- **Lab B (pairs):** `labs/module-06/l9_2-lda/noise_leak_starter.py`. The leak on real customers with 290 noise columns, shuffled and true labels.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P1

Predict the output, then run it. (Three classes, five features.)

```
from sklearn.datasets import make_blobs
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X, y = make_blobs(n_samples=90, centers=3, n_features=5, random_state=0)
print(LinearDiscriminantAnalysis().fit(X, y).transform(X).shape)
```

P2

Predict the output, then run it. (Three of four labels are 0.)

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = [[0.0], [0.5], [1.0], [5.0]]
y = [0, 0, 0, 1]
print(LinearDiscriminantAnalysis().fit(X, y).priors_)
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Loyalty-Tier Map (lesson page). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: stratify on loyalty_tier (Basic 2129, Silver 2015, Gold 356)
- ☐ Step 2: baseline 0.473, LDA 0.909, logistic regression 0.921
- ☐ Step 3: LD1 carries 0.948: spend and tenure
- ☐ Step 4: PCA and LDA maps side by side
- ☐ Step 5: errors only between neighboring tiers
- ☐ Step 6: `tier_map.csv`

SELF-CHECK

Q1

What does this print?

```
from sklearn.datasets import make_blobs
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X, y = make_blobs(n_samples=200, centers=4, n_features=2, random_state=1)
print(LinearDiscriminantAnalysis().fit(X, y).transform(X).shape)
```

A) `(200, 3)`

B) `(200, 2)`

C) `(200, 4)`

D) `(200, 1)`

Q2

What does LDA choose a direction for, compared with PCA?

A) Class means far apart relative to the spread inside each class; PCA chooses the largest total spread and never sees the labels

B) The largest total spread, like PCA, but faster

C) Curved directions where PCA uses straight ones

D) The direction of the most important single feature

Q3

What does this print?

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = [[1.0], [2.0], [3.0], [9.0], [10.0]]
y = ["a", "a", "a", "b", "b"]
print(LinearDiscriminantAnalysis(priors=[0.5, 0.5]).fit(X, y).priors_)
```

Q4

LDA on ten payment categories gives nine axes, but the first carries 0.995 of the separation and leans on amount (+1.42) and items (-0.59). What is the best reading?

A) Nine useful directions: plot them all

B) The categories are perfectly separable

C) The payment details separate categories along essentially one direction, price per item, and only weakly (accuracy 0.290 against 0.219)

D) Amount and items were not scaled

Q5

On 400 customers with 290 added noise columns and SHUFFLED churn labels, LDA fitted on all rows and then cross-validated scores AUC 0.998. Inside a pipeline it scores 0.481. Why?

A) The real columns still carry churn signal

B) kNN with 15 neighbors overfits

C) Shuffling did not work

D) LDA used every row's label, including the test folds', to build its axis; with 300 columns it can separate any labels

Q6

You have 80 rows, 1,000 features, and want an LDA plot AND classification. Which setting?

A) `LinearDiscriminantAnalysis()`

B) `LinearDiscriminantAnalysis(solver="eigen", shrinkage="auto")`

C) `LinearDiscriminantAnalysis(solver="lsqr", shrinkage="auto")`

D) `LinearDiscriminantAnalysis(shrinkage="auto")`

JOURNAL

Describe the team photo in your own words, then write the two numbers that impressed you most today: a leaky score on random labels and the chance level it fell back to inside the pipeline. Write one sentence for a colleague who wants to "just reduce the features first, then cross-validate".

9.3 t-SNE and UMAP

BY THE END YOU CAN

- Explain what t-SNE and UMAP preserve and what they distort
- Tune perplexity (t-SNE) and n_neighbors and min_dist (UMAP), and see how the picture changes
- Read an embedding without over-interpreting cluster sizes or distances
- Check a map with numbers (neighbors kept, trustworthiness) instead of by eye
- Choose between PCA, t-SNE and UMAP for a task, and place new rows with UMAP

KEY IDEAS

- **The seating chart.** t-SNE and UMAP keep neighbors close and bend everything else.
- **Settings:** perplexity (t-SNE), n_neighbors and min_dist (UMAP). Try more than one.
- **Five rules:** neighbors meaningful; sizes not; distances between clusters not; axes meaningless; validate clusters in the original space.
- **Low perplexity invents clumps** from pure noise.
- **Check with numbers:** neighbors kept, trustworthiness. Maps flatter separation.
- **PCA to about 50 first** on wide data; scale tabular data.
- **UMAP has transform** and can place new rows; t-SNE cannot and never goes in a model pipeline.

WORKED EXAMPLE: TWO PERPLEXITIES, ONE SET OF CUSTOMERS

Two perplexities on 600 customers' four behavior columns. Which neighborhoods does each map keep?

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE, trustworthiness

customers = pd.read_csv("data/customers_churn.csv").sample(600, random_state=7)
Z = StandardScaler().fit_transform(customers[["orders_12m", "avg_order_value", "di
for perp in [10, 50]:
    emb = TSNE(perplexity=perp, random_state=42).fit_transform(Z)
    print(f"perplexity {perp}: trustworthiness k=10 {trustworthiness(Z, emb, n_nei
          f"   k=50 {trustworthiness(Z, emb, n_neighbors=50):.3f}   x-span {np.ptp
```

```
perplexity 10: trustworthiness k=10 0.991   k=50 0.977   x-span 120
perplexity 50: trustworthiness k=10 0.991   k=50 0.986   x-span 51
```

Both maps keep the 10 nearest neighbors equally well (0.991). The larger perplexity keeps more of the 50-neighbor structure (0.986 against 0.977), and its map is less than half as wide: the axis numbers have no units. Choose perplexity for the scale of structure you want to see.

GUIDED LAB

- **Lab A:** `labs/module-06/l9_3-tsne-umap/fashion_map_starter.py`. 1,500 training images: two maps, neighbor checks, hardest garments, 500 new images placed.
- **Lab B (pairs):** `labs/module-06/l9_3-tsne-umap/fraud_map_starter.py`. Is fraud its own island? Check the claim against the original space.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P3

Predict the output, then run it. (Which one can place new rows?)

```
from sklearn.manifold import TSNE
import umap
print(hasattr(TSNE(), "transform"), hasattr(umap.UMAP(), "transform"))
```

P4

Predict the output, then run it. (25 rows, perplexity 5.)

```
import numpy as np
from sklearn.manifold import TSNE
X = np.random.default_rng(0).normal(size=(25, 4))
print(TSNE(perplexity=5, random_state=42).fit_transform(X).shape)
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Customer Map (lesson page). In class: Steps 1 and 2. At home: Steps 3 to 6.

- ☐ Step 1: four behavior columns, two logged, scaled
- ☐ Step 2: K-Means in 4-D, silhouette 0.479, five named segments
- ☐ Step 3: t-SNE and UMAP maps of 2,000 customers
- ☐ Step 4: neighbors kept; silhouette on the maps vs in 4-D
- ☐ Step 5: place the other 4,000 with UMAP's transform
- ☐ Step 6: `customer_map.csv` (no churn column)

SELF-CHECK

Q7

On a t-SNE map, segment A is a big island and segment B a small island far away. What can you conclude?

A) A is more varied than B

B) B is very different from every other segment

C) Little about size or distance: check spread and separation in the original data

D) B is noise

Q8

In today's fraud-map lab, 0.630 of a fraud's map neighbors are frauds, and 0.614 of its neighbors in the original space are. What does that tell you about the colleague's "fraud is an island" map?

A) The map reproduces a neighborhood structure that exists in the data; now confirm any group in the original space (a clustering found 239 frauds together, 199 elsewhere)

B) The map created the fraud island

C) The map is wrong because the numbers differ

D) Use a lower perplexity to make the island clearer

Q9

What happens when this runs?

```
import numpy as np
from sklearn.manifold import TSNE
X = np.random.default_rng(0).normal(size=(12, 3))
TSNE(random_state=42).fit_transform(X)
```

Q10

Each day's new customers must be placed on an existing map, and the same embedding should feed a model inside a pipeline. Which method?

A) t-SNE with a fixed random_state

B) t-SNE rerun every day on all customers

C) PCA to 2 components, because only PCA has transform

D) UMAP: fit once, then transform new rows; it can sit in a pipeline

Q11

Why reduce 784 pixels to about 50 PCA components before t-SNE?

A) t-SNE refuses more than 50 columns

B) Speed, and the last few hundred components of image data are mostly noise

C) PCA adds the labels

D) It makes the map's axes interpretable

Q12

A t-SNE map of 200 rows at perplexity 5 shows six clumps. What do you do before telling anyone there are six groups?

A) Nothing: clumps are clusters

B) Change the seed until the clumps are clearer

C) Raise min_dist

D) Try several perplexities and confirm with a clustering and its profiles in the original space

JOURNAL

Explain the seating chart in your own words, then write the five reading rules from memory. Which one would you have broken before this lesson? Finally, write the one number from your project you would show next to the customer map so that nobody over-reads it.

CHECKPOINT 1: MODULES 7 TO 9 (30 MINUTES)

Open notebook: your lesson notebooks and the lesson pages are allowed; another person is not. Open `labs/module-06/checkpoint-1/checkpoint_starter.py`, run it once, then answer in order. Write your key numbers here as you go.

PART	QUESTION	YOUR ANSWER
A (Module 7)	Which three columns does the churn model rely on most (permutation importance on the test rows)?	
B (Module 8)	Five soft segments: which one would you act on first, with its churn rate? How many customers are borderline (below 0.8)?	
C (Module 9)	Components for 90% of the variance; largest LD1 weight; largest PC1 loading; AUC of PC1 alone and of LD1 alone	

Part D: three sentences for the retention lead, one number in each.

EXIT TICKET

Three questions, no notes.

E1

LDA on churn gives one axis, PCA gives ten components. Why the difference?

E2

Name two things you must NOT read off a t-SNE map.

E3

You need tomorrow's customers on today's map. Which method, and how?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish The Loyalty-Tier Map (Steps 3 to 6) and The Customer Map (Steps 3 to 6)

- ☐ Lesson 9.2 Exercises 1 to 3; Lesson 9.3 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Check that `import torch` works
- ☐ Bring to Session 7: a laptop with PyTorch, `pinecrest-ml` with `data/` (keep Fashion-MNIST) and `labs/`, this workbook and my journal

GLOSSARY

TERM	MEANING
LDA	Linear discriminant analysis: directions that separate labeled classes
Fisher's criterion	Between-class spread divided by within-class spread along a direction
scalings_	The weights of LDA's axes
priors	Assumed class shares; they move LDA's decision threshold
shrinkage	Pulls a noisy covariance estimate toward a stabler shape
QDA	Quadratic discriminant analysis: one covariance per class, curved boundaries
t-SNE	A nonlinear map that keeps each point's neighbors close
perplexity	Roughly how many neighbors each point attends to in t-SNE
UMAP	A neighbor-graph map; n_neighbors and min_dist; has transform
trustworthiness	0 to 1: penalizes map neighbors that were far apart in the original data
neighbors kept	Share of each point's k nearest original neighbors that are also map neighbors

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 7: Neural Networks and PyTorch

Your workbook for Session 7: a warm-up on Lessons 9.2 and 9.3, the key ideas of Lessons 10.1 and 10.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 10.1–10.2 | Session 7 · 3 hours | Name: _____

SESSION GOALS

- Build a neural network by hand: a forward pass with shapes that line up, a loss, gradient descent and backpropagation, checked numerically.
- Let PyTorch's autograd compute the same gradients, and know the two lines every autograd loop needs.
- Write the standard PyTorch workflow: float32 tensors, an `nn.Module`, a loss and an optimizer, mini-batches, a validation set with early stopping, a saved `state_dict`.
- Judge a network honestly against logistic regression and gradient boosting, and say when a network is (and is not) worth it.

WARM-UP

FIVE QUESTIONS ON LESSONS 9.2 AND 9.3

Answer from memory, then check with the class.

W1

What does this print? (Lesson 9.2)

```
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X, y = make_classification(n_samples=300, n_features=6, n_informative=4, n_classes=
print(LinearDiscriminantAnalysis().fit(X, y).transform(X).shape)
```

W2

PCA and LDA both flatten data onto a few directions. What is the one big difference in how they choose them? (9.2)

W3

A colleague fits LDA on all 200 rows of 500 random features, then cross-validates a classifier on the LDA output and gets 0.99. What happened? (9.2)

W4

On a t-SNE map of customers, one island is twice the size of the others and far away from them. What may you conclude from the size and the gap? (9.3)

W5

Next month 200 new customers arrive and you want to place them on this month's 2-D map. t-SNE or UMAP, and why? (9.3)

LESSONS

10.1 Neural Networks from Scratch

BY THE END YOU CAN

- Describe a neuron as a weighted sum plus a bias passed through an activation, and compute one by hand
- Write the forward pass of a one-hidden-layer network with matrix shapes that line up
- Explain cross-entropy loss, gradient descent and the learning rate, and implement backpropagation for a small network
- Check hand-written gradients numerically, and reproduce them with PyTorch's autograd
- Judge a network honestly against a baseline, logistic regression and gradient boosting on tabular data

KEY IDEAS

- **A neuron** is activation(x · w + b). With a sigmoid it is logistic regression.
- **A layer is a matrix multiplication:** (n, inputs) @ (inputs, units) → (n, units). Inner dimensions match and disappear. Write the shapes next to the code.
- **Activations bend the boundary.** Without them, any number of layers collapses into one straight line. ReLU (slope 1 where active) is the hidden-layer default; the sigmoid sits at the output.
- **Binary cross-entropy** measures how wrong the probabilities are; **0.693** is the coin-flip value. A loss stuck there has learned nothing.
- **Gradient descent:** parameter -= learning rate × gradient. Too small crawls; too large bounces or explodes. Always look at the loss curve.
- **Backpropagation** gets every gradient in one backward sweep (the output error is p - y). A numeric nudge checks it; each gradient has its parameter's shape.
- **Autograd:** `requires_grad=True`, `loss.backward()`, read `.grad`. Update under `torch.no_grad()` and reset `.grad` every step: gradients add up.
- **Honesty:** on the churn table a network beats logistic regression (AUC 0.856 vs 0.823) but not gradient boosting (0.858).

WORKED EXAMPLE: XOR WITH TWO HIDDEN UNITS

A neural network can do what no straight line can. XOR is the classic case: the output is 1 when exactly one input is 1. Here are two ReLU units with hand-set weights and an output unit, no training at all.

```
import numpy as np

# Two hidden ReLU units with hand-set weights, then an output unit: XOR, which no
W1 = np.array([[1.0, 1.0],
               [1.0, 1.0]])           # (2 inputs, 2 hidden units)
b1 = np.array([0.0, -1.0])
W2 = np.array([[1.0], [-2.0]])      # (2 hidden units, 1 output)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)

hidden = np.maximum(0, X @ W1 + b1)
print(hidden)
print((hidden @ W2).ravel())
```

```
[[0. 0.]
 [1. 0.]
 [1. 0.]
 [2. 1.]]
[0. 1. 1. 0.]
```

The first hidden unit computes x1 + x2; the second computes x1 + x2 - 1, cut at zero, so it is only on for (1, 1). The output takes the first and subtracts twice the second: 0, 1, 1, 0. That is XOR. Each hidden unit drew a line; the output combined them.

```
from sklearn.linear_model import LogisticRegression
y = np.array([0, 1, 1, 0])
straight = LogisticRegression().fit(X, y)
print("logistic regression predicts:", straight.predict(X), " accuracy", straight.
```

```
logistic regression predicts: [0 0 0 0]  accuracy 0.5
```

Logistic regression, with one straight line, can do no better than chance on these four points (here it predicts 0 everywhere: accuracy 0.5). Training a network means finding weights like the ones above automatically, by gradient descent.

GUIDED LAB

- **Lab A:** `labs/module-07/l10_1-neural-networks/circles_network_starter.py`. One class inside a ring of the other. Write `forward` and `bce_loss`, check a gradient by nudging, write the training loop, then find the smallest number of hidden units that can enclose a ring. Predict that number before TODO 5.
- **Lab B (pairs):** `churn_neuron_starter.py` in the same folder. Train ONE neuron by hand on churn and compare it with `LogisticRegression`. One types, one predicts; swap at TODO 3.

- Done when your numbers match the "Expected when done" lines the starter prints.
- Finished early? `debug_backprop.py`: one bug that stops the script, two that do not.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (A (5, 3) matrix times a (3, 2) matrix.)

```
import numpy as np
X = np.ones((5, 3))
W = np.full((3, 2), 2.0)
print((X @ W).shape, (X @ W)[0])
```

P2

Predict the output, then run it. (The slope of $(2w)^2$ is $8w$.)

```
import torch
w = torch.tensor(1.5, requires_grad=True)
loss = (2 * w) ** 2
loss.backward()
print(w.grad)
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A churn network from scratch (lesson page, "Project: A Churn Network from Scratch"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: split and scale. Check: `(4500, 10) (1500, 10) churn rate 0.165`
- ☐ Step 2: yardsticks. Check: baseline AUC 0.500, logistic regression 0.823

- ☐ Step 3: train 10 → 16 → 1 (193 parameters); loss 0.2994 at epoch 1,000
- ☐ Step 4: network test AUC (about 0.856)
- ☐ Step 5: gradient boosting with defaults (about 0.858)
- ☐ Step 6: `churn_net.npz` reloads to identical predictions; summary printed

SELF-CHECK

Q1

A hidden layer takes 10 inputs and has 16 units. How many parameters (weights and biases) does it have?

A) 160

B) 176

C) 26

D) 170

Q2

What does this print?

```
import numpy as np
x = np.array([1.0, 2.0])
w = np.array([0.5, -1.0])
z = x @ w + 1.0
print(z, max(0.0, z), round(1 / (1 + np.exp(-z)), 3))
```

Q3

After 2,000 epochs your network's training loss is still `0.6931` and its test accuracy is 0.500. Which bug is the most likely?

A) The learning rate is far too large

B) The test set is too small

C) Every weight was initialized to zero

D) The network has too many hidden units

Q4

What does this print?

```
w = 0.0
for step in range(2):
    w = w - 0.25 * 2 * (w - 1)        # gradient descent on (w - 1) ** 2
print(w)
```

Q5

Why is ReLU the default activation for hidden layers rather than the sigmoid?

A) Its slope is 1 wherever it is active, so gradients pass back through many layers undiminished

B) It outputs probabilities between 0 and 1

C) It keeps the network linear, which trains faster

D) It has no weights of its own, unlike the sigmoid

Q6

What happens when this runs?

```
import torch
w = torch.tensor([1.0, 2.0], requires_grad=True)
out = (w * 3).sum()
out.numpy()
```

A) It prints `9.0`

B) It prints `[3. 6.]`

C) `TypeError`: a sum cannot be converted to NumPy

D) `RuntimeError`: Can't call numpy() on Tensor that requires grad

JOURNAL

Explain the training loop (forward, loss, backward, update) to someone who has never seen code, using the panel of junior analysts. Then write one sentence on why your churn network beat logistic regression, and one on why you would still ship gradient boosting.

10.2 PyTorch Fundamentals

BY THE END YOU CAN

- Create and reshape tensors, convert between NumPy and PyTorch, and get the float32 dtype right
- Define a network with nn.Linear, nn.Sequential and your own nn.Module subclass
- Choose the loss function that matches the task, and train with an optimizer, a Dataset and a DataLoader
- Write the standard training loop with model.train(), model.eval(), torch.no_grad(), a validation set and early stopping
- Save and reload a trained model with state_dict, and judge it honestly against gradient boosting

KEY IDEAS

- **Tensors** have a shape, a dtype and a device. Models use float32; NumPy, pandas and scikit-learn give float64: convert with `dtype=torch.float32`.
- `nn.Module`: layers in `__init__`, wiring in `forward`; call `model(x)`. `nn.Linear(10, 16)` stores a (16, 10) weight.
- **Logits out, sigmoid in the loss:** `BCEWithLogitsLoss` (targets float, shape (n, 1)); `CrossEntropyLoss` for k classes (integer targets, shape (n,)); `MSELoss` for numbers.
- **The loop:** `optimizer.zero_grad()` → `loss = loss_fn(model(xb), yb)` → `loss.backward()` → `optimizer.step()`, over mini-batches from a shuffled `DataLoader`.
- **Modes are not gradients:** `model.train()` / `model.eval()` switch dropout and batch norm; `torch.no_grad()` switches off tracking. Prediction code uses both.
- **Early stopping** keeps a `copy.deepcopy` of the best `state_dict` by validation loss and rolls back. The validation score is then optimistic; the test set is scored once.
- **Save** `state_dict` plus the settings and the fitted preprocessing; load with `weights_only=True`.
- **Honesty again:** network 0.846, gradient boosting 0.850 on churn: a tie, and boosting needed none of the choices.

WORKED EXAMPLE: THE SMALLEST TRAINING LOOP

The smallest possible PyTorch training loop: one `nn.Linear(1, 1)` rediscovering the rule y = 3x + 2 from 50 points. Every line of the professional loop is here.

```python
import torch
from torch import nn

torch.manual_seed(42)
x = torch.linspace(0, 1, 50).unsqueeze(1)            # (50, 1): one feature, as a
y = 3 * x + 2                                         # the rule to rediscover
model = nn.Linear(1, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
loss_fn = nn.MSELoss()
for step in range(1, 301):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
    if step in (1, 10, 100, 300):
        print(f"step {step:3d}  loss {loss.item():.2e}")
print(f"weight {model.weight.item():.3f}  bias {model.bias.item():.3f}")
```

```
step   1  loss 5.67e+00
step  10  loss 3.83e-02
step 100  loss 1.11e-07
step 300  loss 2.25e-13
weight 3.000  bias 2.000
```

The loss collapses toward zero, and the layer's one weight and one bias become 3.000 and 2.000. `zero_grad` → forward and loss → `backward` → `step`: the same four lines train the churn model, the fraud model, and next week's image and sequence models. Only the model, the data and the loss change.

GUIDED LAB

- **Lab A:** `labs/module-07/l10_2-pytorch/fraud_mlp_starter.py`. The whole PyTorch workflow on card fraud: float32 tensors, a `FraudNet` module, mini-batches, early stopping, one look at the test set, and an honest comparison with logistic regression and gradient boosting.
- Done when your numbers match the "Expected when done" lines (training numbers may differ in the last digit on your laptop).
- Finished early? `debug_training_loop.py`: two loud bugs and three silent ones.
- At home: `batch_size_starter.py`: why ten epochs with one giant batch learn almost nothing.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Integers and floats, PyTorch's defaults.)

```
import torch
print(torch.tensor([1, 2, 3]).dtype, torch.tensor([1.0, 2.0]).dtype)
```

P4

Predict the output, then run it. (Weights plus biases, two layers.)

```
from torch import nn
net = nn.Sequential(nn.Linear(3, 5), nn.ReLU(), nn.Linear(5, 2))
print(sum(p.numel() for p in net.parameters()))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A PyTorch churn model (lesson page, "Project: A PyTorch Churn Model"). In class: Steps 1 to 4. At home: Steps 5 and 6.

- ☐ Step 1: three-way split without the leak. Check: `3600 900 1500`
- ☐ Step 2: preprocessing fitted on the fitting rows. Check: 40 columns
- ☐ Step 3: `ChurnMLP` with dropout. Check: 1,345 parameters
- ☐ Step 4: early stopping (best epoch about 17)
- ☐ Step 5: one test, four models (network about 0.846, boosting about 0.850)
- ☐ Step 6: `churn_mlp.pt` (settings + weights) and `churn_preprocess.joblib` rebuild a working model

SELF-CHECK

Q7

What does this print?

```
from torch import nn
layer = nn.Linear(4, 3)
print(layer.weight.shape, sum(p.numel() for p in layer.parameters()))
```

Q8

What happens when this runs?

```
import torch
from torch import nn
logits = torch.zeros(8, 1)
targets = torch.ones(8)
nn.BCEWithLogitsLoss()(logits, targets)
```

A) `ValueError`: Target size (torch.Size([8])) must be the same as input size (torch.Size([8, 1]))

B) It returns a loss of about 0.6931

C) `RuntimeError`: expected integer (Long) targets

D) It returns a loss of 0.0

Q9

Which statement about `model.eval()` and `torch.no_grad()` is true?

A) They are two names for the same switch

B) `torch.no_grad()` switches dropout off; `model.eval()` stops gradient tracking

C) `model.eval()` changes how layers such as dropout behave; `torch.no_grad()` stops gradient tracking; prediction code uses both

D) `model.eval()` is only needed when the model runs on a GPU

Q10

What does this print?

```
import torch
from torch.utils.data import TensorDataset, DataLoader
data = TensorDataset(torch.zeros(1000, 5), torch.zeros(1000, 1))
loader = DataLoader(data, batch_size=64, shuffle=True)
sizes = [len(xb) for xb, yb in loader]
print(len(loader), sizes[-1])
```

Q11

In the lesson's early-stopping function, why does the best state go through `copy.deepcopy(model.state_dict())`?

A) `torch.save` cannot save a state_dict that was not copied

B) `state_dict()` returns references to the live tensors, which later epochs would keep changing

C) `deepcopy` switches the model to evaluation mode

D) It converts the weights to float64 so nothing is lost

Q12

You predict `loyalty_tier` (Basic, Silver, Gold) with `nn.CrossEntropyLoss`. What must the targets be?

A) float32 values 0.0 to 2.0, shape (n, 1)

B) the strings Basic, Silver and Gold

C) float32 values 0.0 or 1.0, shape (n, 1)

D) integer class indices 0, 1, 2 (`torch.long`), shape (n,)

JOURNAL

For each line of the training loop (zero_grad, forward, loss, backward, step), write which part of your Lesson 10.1 code it replaces. Then write the recommendation you would give Pinecrest about the churn model, with the two test AUCs that support it.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Write the four steps of the training loop and the PyTorch line for each.

E2

Your churn model returns logits. How do you get a churn probability for a report, and why is there no sigmoid inside the model?

E3

On churn the network's test AUC is 0.846 and gradient boosting's 0.850. Which do you recommend, and why?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the 10.1 project (Steps 4 to 6) and the 10.2 project (Steps 5 and 6)
- ☐ Lesson 10.1 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 10.2 Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Finish `churn_neuron_starter.py`; do `batch_size_starter.py`
- ☐ Save `train_one_epoch`, `evaluate` and `fit` in `training.py`
- ☐ Fashion-MNIST in my `data/` folder (about 30 MB, downloaded at home)
- ☐ Bring to Session 8: a charged, plugged-in laptop, `pinecrest-ml` with `data/` and `labs/`, `training.py`, this workbook and my journal

GLOSSARY

TERM	MEANING
neuron (unit)	activation(x · w + b): a weighted sum plus a bias, through an activation
activation	The nonlinear function after the weighted sum: ReLU, sigmoid, tanh
hidden layer	A layer between input and output; its width is the number of units
forward pass	Computing the outputs from the inputs
loss	One number for how wrong the model is; binary cross-entropy for yes/no targets
gradient	The slopes of the loss with respect to every parameter
learning rate	The step size of gradient descent
backpropagation	Computing every gradient in one backward sweep with the chain rule
autograd	PyTorch's automatic gradients: `requires_grad`, `backward()`, `.grad`
epoch / batch	One pass over the training data / the rows used for one update step
tensor	PyTorch's array: shape, dtype, device; can track gradients

TERM	MEANING
logit	A raw score before the sigmoid or softmax
optimizer	The object that updates the parameters: `SGD`, `Adam`
early stopping	Stop when the validation loss stops improving; restore the best weights
state_dict	A dictionary of a model's named weight tensors: what you save and load
dropout	Randomly zeroing hidden units while training (switched off by `eval()`)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 8: CNNs and Sequence Models

Your workbook for Session 8: a warm-up on Lessons 10.1 and 10.2, the key ideas of Lessons 10.3 and 10.4, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 10.3–10.4 | Session 8 · 3 hours | Name: _____

SESSION GOALS

- Explain a convolution and pooling, predict shapes and parameter counts, and trace a dummy batch through a CNN.
- Train a CNN and a dense baseline with the same loop and seed, and read per-class results, not just the average.
- Turn a time series into windows split by time, beat the naive forecasts, and report errors in real units.
- Explain the hidden state, the LSTM's gates and attention as a lookup, and judge a sequence model against simpler ones.

WARM-UP

FIVE QUESTIONS ON LESSONS 10.1 AND 10.2

Answer from memory, then check with the class.

W1

What does this print? (Lesson 10.2)

```
import numpy as np
import torch
print(torch.tensor(np.zeros(3)).dtype, torch.tensor(np.zeros(3), dtype=torch.float3
```

W2

Put the four lines of the batch loop in order: loss.backward(), optimizer.step(), optimizer.zero_grad(), loss = loss_fn(model(xb), yb). (10.2)

W3

Your binary model ends with nn.Linear(32, 1) and trains with BCEWithLogitsLoss. Where does the sigmoid go? (10.2)

W4

A model with dropout gives a different score for the same customer on every call, inside with torch.no_grad(). Why? (10.2)

W5

On the churn table the PyTorch network scored test AUC 0.846 and gradient boosting 0.850. What was the verdict, and what kind of data should change it? (10.1, 10.2)

LESSONS

10.3 Convolutional Neural Networks

BY THE END YOU CAN

- Explain what a convolution filter and a pooling layer compute, and why sharing one small filter across the whole image suits pictures
- Predict the output shape and the parameter count of nn.Conv2d and nn.MaxPool2d layers
- Build a CNN as an nn.Module and train it on Fashion-MNIST with a validation set and a fixed seed
- Compare it with a dense-network baseline using test accuracy, per-class accuracy and a confusion matrix
- Save trained weights with state_dict and reload them safely with weights_only=True

KEY IDEAS

- **A convolution** slides a small filter over the image; each filter makes one feature map (one output channel). Deep-learning libraries do not flip the kernel; nobody minds.
- **Local connectivity and weight sharing:** each output looks at a small window, and the same filter is used everywhere, so a pattern learned once is found anywhere, for very few weights.
- **Shapes:** batches are (batch, channels, height, width); per side out = (n + 2p - k) / s + 1, rounded down; `padding=1` with a 3 × 3 kernel keeps the size.
- **Parameters:** a conv layer has out × (in × k × k) + out; max pooling has none and halves height and width.
- **Trace a dummy batch** through the layers to size the first `nn.Linear`.
- **Logits for** `CrossEntropyLoss`; `model.train()` / `model.eval()` switch dropout and batch norm; evaluate under `torch.no_grad()`.
- **A fair comparison:** same images, same order, same loop; only the architecture changes. Dense 0.840, CNN 0.864, improved CNN 0.892 on the test set.
- **Report the weakest classes:** shirts, T-shirts, pullovers and coats look alike at 28 × 28.

WORKED EXAMPLE: A COLOR CNN, LAYER BY LAYER

Lesson 10.5 will work with 64 × 64 color images. Trace one through a small two-block CNN and count where the parameters live.

```python
import torch
from torch import nn

net = nn.Sequential(
    nn.Conv2d(3, 8, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(8, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(), nn.Linear(16 * 16 * 16, 10),
)
x = torch.zeros(1, 3, 64, 64)                        # one 64 x 64 color image
for layer in net:
    x = layer(x)
    n = sum(p.numel() for p in layer.parameters())
    print(f"{layer.__class__.__name__:10s} {str(tuple(x.shape)):18s} {n:6,d} param
```

```
Conv2d     (1, 8, 64, 64)        224 parameters
ReLU       (1, 8, 64, 64)          0 parameters
MaxPool2d  (1, 8, 32, 32)          0 parameters
Conv2d     (1, 16, 32, 32)     1,168 parameters
ReLU       (1, 16, 32, 32)         0 parameters
MaxPool2d  (1, 16, 16, 16)         0 parameters
Flatten    (1, 4096)               0 parameters
Linear     (1, 10)            40,970 parameters
```

Three channels in, so each first-layer filter has $3 \times 3 \times 3$ weights: $8 \times 27 + 8 = 224$. Each pooling halves the side: $64 \to 32 \to 16$. The convolutions together use under 1,400 parameters; the single dense layer at the end uses 40,970, because it has one weight per position of every feature map. That is where to save parameters, and why big CNNs end with global pooling.

GUIDED LAB

- **Lab A:** `labs/module-08/l10_3-cnn/edge_features_starter.py`. Build features from two hand-made edge filters and race logistic regression on them against the lesson's CNN at 1,000, 5,000 and 20,000 images. Predict the winner at each size first.
- **Lab B (pairs):** `shift_test_starter.py` in the same folder. Load the lesson's two trained models and move every test image a few pixels. Does the CNN really not care where the sneaker is?
- Done when your numbers match the "Expected when done" lines.
- Finished early? `debug_cnn.py`: two shape errors and two silent bugs.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Pooling halves height and width.)

```
import torch
from torch import nn
print(nn.MaxPool2d(2)(torch.zeros(5, 8, 28, 28)).shape)
```

P2

Predict the output, then run it. (Four 5 × 5 filters on one channel.)

```
from torch import nn
print(sum(p.numel() for p in nn.Conv2d(1, 4, kernel_size=5).parameters()))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A better clothing classifier (lesson page, "Project: A Better Clothing Classifier"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: three sets. Check: `20000 5000 10000`
- ☐ Step 2: `FashionCNN` traced. Check: 421,834 parameters; features (2, 64, 7, 7); logits (2, 10)
- ☐ Step 3: four epochs, best kept (validation about 0.903)
- ☐ Step 4: test once (about 0.892) and the four hardest classes
- ☐ Step 5: `fashion_cnn.pt` reloads with `weights_only=True`, identical predictions
- ☐ Step 6: twelve predictions plotted; summary printed

SELF-CHECK

Q1

What does this print?

```
import torch
from torch import nn
conv = nn.Conv2d(3, 16, kernel_size=3, stride=2, padding=1)
print(tuple(conv(torch.zeros(8, 3, 32, 32)).shape))
```

Q2

How many parameters does `nn.Conv2d(3, 8, kernel_size=3)` have?

A) 224

B) 216

C) 72

D) 80

Q3

Why does the lesson use `padding=1` with every 3 × 3 kernel?

A) It halves the height and width

B) It adds a border of learnable weights

C) It keeps the height and width unchanged: (n + 2 - 3) / 1 + 1 = n

D) Grayscale images require it

Q4

What does this print?

```
import torch
import torch.nn.functional as F
x = torch.arange(16.0).reshape(1, 1, 4, 4)
print(F.max_pool2d(x, 2).squeeze())
```

Q5

In the shift lab, the lesson's CNN loses about 12 points of test accuracy when every image moves 2 pixels to the right. Convolutions share weights across positions, so why does it lose anything?

A) The model has no max pooling

B) The shifted images were scaled differently

C) Each convolution uses different weights at different positions

D) The `Flatten` + `Linear` head reads WHERE each feature map lit up, so positions still matter

Q6

This model fails. What should the first argument of its nn.Linear be?

```
import torch
from torch import nn
model = nn.Sequential(
    nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.MaxPool2d(2), nn.Flatten(), nn.Linear(64 * 14 * 14, 10),
)
model(torch.zeros(64, 1, 28, 28))
```

A) 12544 (64 × 14 × 14)

B) 3136 (64 × 7 × 7)

C) 1568 (32 × 7 × 7)

D) 64 (one number per filter)

JOURNAL

Explain the stencil-and-flashlight picture in your own words, then write the shape of a batch at every stage of your FashionCNN, from (64, 1, 28, 28) to (64, 10), without looking. Which classes did your model confuse most, and would a person confuse them too?

10.4 Sequence Models

BY THE END YOU CAN

- Measure naive and seasonal-naive forecasts and a gradient-boosting model on lag features, and use them as the yardsticks any sequence model must beat
- Turn a time series into sliding windows with a time-based train/test split, and explain why a random split leaks
- Explain how a recurrent cell carries a hidden state through a sequence, and why LSTMs and GRUs add gates
- Build, train and evaluate an LSTM forecaster in PyTorch, reporting the error in dollars
- Describe attention as a soft lookup, compute it by hand, and say when a transformer is (and is not) worth it

KEY IDEAS

- **Order matters:** split by time (train 2023 and 2024, test 2025), never at random.
- **Naive forecasts are the yardsticks:** yesterday ($1,350 MAE) and the same day last week ($1,072).
- **Windows:** a window of recent days predicts the next day; neighbors overlap, which is why random splits leak. Shape (batch, time steps, features) with `batch_first=True`.
- **A recurrent cell** updates a hidden state with the same weights at every step; plain RNNs forget because gradients vanish. LSTMs and GRUs add gates.
- **Scale, then undo:** log1p and standardize with the training years; report the error in dollars.
- **The LSTM** (28 days + weekday) beat seasonal naive: $702; with tomorrow's promotion and closed flags, $515.
- **Attention is a soft lookup:** queries score keys, a softmax makes weights, the output averages the values. Transformers stack learned attention.
- **Honesty:** default boosting on lag features with the same information scored $616: the information fed in mattered more than the architecture.

WORKED EXAMPLE: A NOTEBOOK THAT FORGETS

A recurrent cell with one input weight, one memory weight and a one-number hidden state. One busy day, then four quiet ones: how long does the notebook remember?

```
import numpy as np

w_x, w_h, b = 1.0, 0.5, 0.0                          # one input weight, one memory
h = 0.0
for t, x in enumerate([2.0, 0.0, 0.0, 0.0, 0.0]):    # one big day, then four quiet
    h = np.tanh(w_x * x + w_h * h + b)
    print(f"step {t}: input {x:.1f}  hidden state {h:.3f}")
```

```
step 0: input 2.0  hidden state 0.964
step 1: input 0.0  hidden state 0.448
step 2: input 0.0  hidden state 0.220
step 3: input 0.0  hidden state 0.110
step 4: input 0.0  hidden state 0.055
```

With a memory weight of 0.5 the impression halves every day. Change the memory weight:

```
for w_h in [0.5, 0.9, 1.5]:
    h = 0.0
    for x in [2.0, 0.0, 0.0, 0.0, 0.0]:
        h = np.tanh(1.0 * x + w_h * h)
    print(f"memory weight {w_h}: hidden state after four quiet days {h:.3f}")
```

```
memory weight 0.5: hidden state after four quiet days 0.055
memory weight 0.9: hidden state after four quiet days 0.395
memory weight 1.5: hidden state after four quiet days 0.861
```

A small memory weight forgets fast; a larger one keeps the busy day in mind for longer. Training chooses such weights. Over 28 steps, the gradient that teaches them is multiplied 28 times by factors like these, which is why plain RNNs struggle with the distant past and LSTMs add a cell state updated by adding and scaling.

GUIDED LAB

- **Lab A:** `labs/module-08/l10_4-sequences/transactions_gru_starter.py`. Forecast tomorrow's number of transactions with a GRU: naive yardsticks first, windows, then with and without the calendar.
- **Lab B:** `attention_lookup_starter.py`. Change what attention's keys describe and how far back it looks, and score ordinary and special days separately.
- Done when your numbers match the "Expected when done" lines (GRU numbers may differ in the last digit).
- Finished early? `debug_sequence.py`: one loud bug, three silent ones.

PREDICT, THEN RUN

Write your prediction on the line first, then run the code. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Windows of 3 from 6 values.)

```
import numpy as np
series = np.arange(6)
X = np.stack([series[i:i + 3] for i in range(len(series) - 3)])
print(X.shape, series[3:])
```

P4

Predict the output, then run it. (Four gates' worth of weights and two bias vectors.)

```
from torch import nn
print(sum(p.numel() for p in nn.LSTM(input_size=2, hidden_size=4).parameters()
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A calendar-aware forecaster (lesson page, "Project: A Calendar-Aware Forecaster"). In class: Steps 1 to 3. At home: Steps 4 to 7.

- ☐ Step 1: calendar (1096, 9): 54 promotion days, 6 closed days
- ☐ Step 2: windows (4272, 28, 10); 1,460 test windows
- ☐ Step 3: calendar-aware LSTM (MAE about $515)
- ☐ Step 4: errors per store and on special days
- ☐ Step 5: gradient boosting on lags with the same information (about $616) and with the event type (about $569)

- ☐ Step 6: the holiday-season chart for Boise
- ☐ Step 7: `sales_lstm.pt` reloads; the six-line summary

SELF-CHECK

Q7

What does this print?

```
import torch
from torch import nn
out, h = nn.GRU(5, 8, batch_first=True)(torch.zeros(4, 10, 5))
print(tuple(out.shape), tuple(h.shape))
```

Q8

A store has 100 days of sales. How many training examples do windows of 7 days (each predicting the next day) give?

A) 100

B) 94

C) 93

D) 7

Q9

For Pinecrest's daily store sales, which forecast is the seasonal naive one?

A) Tomorrow = the same weekday last week

B) Tomorrow = today

C) Tomorrow = the average of the training years

D) Tomorrow = the same date last year

Q10

What does this print?

```
import numpy as np
keys = np.array([[1, 0], [0, 1], [1, 0]])
values = np.array([10.0, 50.0, 30.0])
query = np.array([1, 0])
w = np.exp(5 * (keys @ query))
w = w / w.sum()
print(w.round(3), round(w @ values, 1))
```

Q11

Your LSTM predicts standardized log sales. What must happen before you compute the MAE you report?

A) Round the predictions to whole numbers

B) Nothing: targets and predictions are on the same scale, so the MAE is fair

C) Multiply the predictions by sd

D) Undo the standardization, then the log (expm1), and compare in dollars

Q12

You create `nn.LSTM(8, 32)` and forget `batch_first=True`, while your windows are (batch, 28, 8). What happens?

A) A RuntimeError about the input size

B) It runs and trains on nonsense: PyTorch reads (batch, time) as (time, batch)

C) PyTorch notices and transposes the input

D) The output becomes (28, 64, 32), so the linear head fails

Explain the store manager's notebook and the binder in your own words: what does the hidden state hold, and what does attention do differently? Then write your project's MAE numbers and one sentence a store manager would understand about what the best model gets right and where it still struggles.

EXIT TICKET

Three questions, no notes.

E1

A batch (64, 1, 28, 28) goes through `nn.Conv2d(1, 16, kernel_size=3, padding=1)` and then `nn.MaxPool2d(2)`. What is the shape after each?

E2

Why must a time series be split by time and not at random?

E3

With the same information, the calendar-aware LSTM forecast daily sales with MAE $515 and default gradient boosting on lag features $616 ($569 given the event type). What do you tell the store managers?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the 10.3 project (Steps 4 to 6) and the 10.4 project (Steps 4 to 7)
- ☐ Lesson 10.3 Exercises 1 to 3; Lesson 10.4 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Finish `shift_test_starter.py` and `attention_lookup_starter.py`
- ☐ Retrain the calendar LSTM with `torch.manual_seed(7)`: how big is the wobble?
- ☐ AT HOME: EuroSAT into `data/eurosat/` and the ResNet-18 and MobileNetV3-small weights cached (the command is on the bring-list slide)
- ☐ Bring to Session 9: a plugged-in laptop, `pinecrest-ml` with `data/` and `labs/`, `sales_lstm.pt` and `fashion_cnn.pt` with my project notebooks, `training.py`, this workbook and my journal

GLOSSARY

TERM	MEANING
convolution	Sliding a small filter over an image; each position gives one number of a feature map
filter (kernel)	The small grid of learned weights, usually 3 × 3, spanning all input channels
feature map / channel	The output of one filter; a conv layer outputs one channel per filter
padding / stride	Zeros around the border (keeps the size) / the step between positions (2 halves it)
max pooling	Keeps the largest value of each small block; no weights; halves height and width
weight sharing	One filter reused at every position: patterns found anywhere, few weights
batch normalization	Rescales activations batch by batch; stores statistics used in eval mode
seasonal naive	Forecast = the value one season ago (same weekday last week)
window	A run of recent time steps used as one input; the next step is its target
hidden state	The vector a recurrent cell carries from step to step: its notebook
LSTM / GRU	Recurrent cells with gates that decide what to keep, write and reveal
vanishing gradient	Gradients shrinking over many steps, so the distant past is hard to learn
attention	Weights from matching a query with keys (softmax), used to average values

TERM	MEANING
transformer	A model built from stacked learned attention layers plus positional information

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 9: Transfer Learning and Text Classification

Your workbook for Session 9: a warm-up on CNNs and sequences, the key ideas of Lessons 10.5 and 11.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 10.5–11.1 | Session 9 · 3 hours | Name: _____

SESSION GOALS

- Reuse a pretrained network: cut off its head, freeze its backbone, train a new head on its features, and fine-tune its last block safely.
- Measure what transfer learning buys against a from-scratch CNN, and how many labels are worth paying for.
- Turn reviews into a sparse TF-IDF matrix inside a pipeline, train a sentiment classifier and read its words.
- Spot the stop-word trap and keep negation with bigrams; choose a flagging threshold without touching the test set.

WARM-UP: CNNS AND SEQUENCES

Five questions on Lessons 10.3 and 10.4. Answer alone first (2 minutes).

W1

What shape does this print?

```
import torch
from torch import nn
print(nn.Conv2d(1, 8, kernel_size=3)(torch.zeros(4, 1, 28, 28)).shape)
```

W2

How many parameters? (This is ResNet-18's first layer; Lesson 10.3's Before the Next Lesson asked the same.)

```
from torch import nn
print(sum(p.numel() for p in nn.Conv2d(3, 64, kernel_size=7, bias=False).parameters
```

W3

What does this print?

```
import torch
from torch import nn
lstm = nn.LSTM(input_size=2, hidden_size=8, batch_first=True)
out, (h, c) = lstm(torch.zeros(5, 14, 2))
print(tuple(out.shape), tuple(h.shape))
```

W4

Why must `model.eval()` come before measuring test accuracy of a model with dropout and batch norm?

W5

A colleague splits overlapping sales windows with `train_test_split(..., random_state=42)` and beats the seasonal naive forecast by a mile. Your first suspicion?

LESSONS

10.5 Transfer Learning

BY THE END YOU CAN

- Explain why features learned on ImageNet transfer to a new image task, and when they will not
- Load a pretrained torchvision model with the `weights=` API and prepare images the way it expects
- Use a frozen backbone as a feature extractor and train a small head on its features
- Fine-tune the last block with a lower learning rate, and compare the result with a CNN trained from scratch
- Measure how many labeled images a model needs before you pay for labeling more

KEY IDEAS

- **The experienced photo editor.** A pretrained backbone already sees edges, textures and colors; you teach only the last step, the head.
- **Backbone and head.** ResNet-18's head is `fc` (512 → 1,000 ImageNet classes). `nn.Identity()` in its place gives features; `nn.Linear(512, 10)` gives your classes.
- **Prepare images the model's way.** `weights.transforms()` holds the recipe; normalize with ImageNet's `mean` and `std` (not your data's), 3 channels, a sensible size (128 px on a CPU).
- **Frozen features are a table.** In `eval()` mode under `torch.inference_mode()`, compute them once; then any classifier (a linear head, logistic regression, kNN) learns from them in seconds.
- **The numbers:** from scratch 0.719 with 2,000 tiles; frozen + linear head 0.913; 100 labels already beat the from-scratch CNN; fine-tuned layer4 0.928 on test.
- **Freeze first, then attach the head.** New layers are trainable by default. Check `count_trainable(model)` before training.
- **Fine-tune in stages.** Head first, then unfreeze the last block with a ten times smaller learning rate (parameter groups); keep it only if validation improves.
- **Silent mistakes:** no normalization (0.840 instead of 0.894), features in training mode (batch-dependent features, drifting statistics).

WORKED EXAMPLE: FIND THE HEAD, MATCH THE CHANNELS

Where is the head of a network you have never used? Ask it. MobileNetV3-small, built for phones, keeps its head in `classifier`, not `fc`. Building the architecture with `weights=None` needs no download.

```
import torch
from torch import nn
from torchvision import models

mobile = models.mobilenet_v3_small(weights=None)      # architecture only: no down
print(mobile.classifier)
print(f"parameters: {sum(p.numel() for p in mobile.parameters()):,}")
```

```
Sequential(
  (0): Linear(in_features=576, out_features=1024, bias=True)
  (1): Hardswish()
  (2): Dropout(p=0.2, inplace=True)
  (3): Linear(in_features=1024, out_features=1000, bias=True)
)
parameters: 2,542,856
```

Its head is four layers ending in 1,000 ImageNet classes. Replace the whole `classifier` and the network outputs its features:

```
mobile.classifier = nn.Identity()
mobile.eval()
with torch.no_grad():
    print(tuple(mobile(torch.zeros(2, 3, 128, 128)).shape))
```

```
(2, 576)
```

576 features per image (ResNet-18 gives 512). And a grayscale image? Repeat its one channel three times so it fits the RGB stem:

```
gray = torch.rand(4, 1, 128, 128)                     # four grayscale images
rgb_like = gray.repeat(1, 3, 1, 1)                     # copy the one channel thre
with torch.no_grad():
    print(tuple(rgb_like.shape), "->", tuple(mobile(rgb_like).shape))
```

```
(4, 3, 128, 128) -> (4, 576)
```

Three habits in one example: find the head by printing the model, replace it with `nn.Identity()` for features, and match the channels the backbone expects. Setting `mobile.fc = nn.Identity()` instead would raise no error and change nothing: always check the output shape.

◆ GUIDED LAB

- **Lab A:** `labs/module-09/l10_5-transfer-learning/feature_table_starter.py`. Frozen ResNet-18 features of 3,000 new tiles, loaded from `_fixtures/` in a second: a baseline, a cross-validated head, k-nearest neighbors, the weakest classes and a "send to a human" queue.
- **Lab B (pairs):** `freeze_audit_starter.py` in the same folder. Audit a fine-tuning setup on one batch: what is trainable, what the optimizer received, what changes after one step.
- Done when your numbers match the "Expected when done" lines. Finished early? `debug_transfer.py`: four bugs, two of them silent.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P1

Predict the output, then run it. (A head for MobileNetV3-small's 576 features and 10 classes.)

```
from torch import nn
head = nn.Linear(576, 10)
print(sum(p.numel() for p in head.parameters()))
```

P2

Predict the output, then run it. (Normalize subtracts the mean, then divides by the std.)

```
import torch
from torchvision import transforms
norm = transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.25, 0.25, 0.25])
x = torch.full((3, 2, 2), 0.75)
print(norm(x)[:, 0, 0])
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The land-cover classifier (lesson page, "Project: A Land-Cover Classifier"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: `2000 500 1000` from one seeded permutation
- ☐ Step 2: 5,130 trainable parameters; validation 0.164 before training
- ☐ Step 3: stage 1, two epochs: validation about 0.824 then 0.876 (yours may vary slightly)
- ☐ Step 4: stage 2 with layer4 unfrozen at 1e-4: keep it only if validation improves (about 0.924)
- ☐ Step 5: test once (about 0.928); weakest class and the confusion matrix
- ☐ Step 6: `landcover_resnet18.pt` reloads with `weights_only=True`

SELF-CHECK

Q1

With a frozen backbone, the lesson computes the 512 features of every tile ONCE and then trains the head on that table. Why is that allowed?

A) Frozen weights never change, so each tile's features are the same in every epoch

B) Computing features once is required by PyTorch for pretrained models

C) The head needs the features as NumPy arrays, not tensors

D) Features computed once are more accurate than features computed every epoch

Q2

What does this print?

```
import torch
from torch import nn
from torchvision import models
m = models.mobilenet_v3_small(weights=None)
for p in m.parameters():
    p.requires_grad = False
m.classifier = nn.Linear(576, 3)
print(sum(p.numel() for p in m.parameters() if p.requires_grad))
```

Q3

The label curve: frozen features reach 0.755 with 100 labeled tiles and 0.902 with 2,000; the from-scratch CNN reached 0.719 with 2,000. Pinecrest offers to pay for 5,000 more labels for the frozen-feature model. Your advice?

A) Yes: accuracy always keeps growing with more labels

B) Yes: the from-scratch CNN would then beat transfer learning

C) Probably not: the curve has flattened (1,000 to 2,000 added under a point); fine-tuning is the better next step

D) No: transfer learning cannot use more than 2,000 labels

Q4

A learner extracts features with a freshly created ResNet-18 (never called `.eval()`) under `torch.no_grad()`. What goes wrong?

A) Nothing: no_grad already switches the model to evaluation mode

B) Batch normalization uses each batch's statistics, so a tile's features depend on its batch, and the stored ImageNet statistics are overwritten

C) PyTorch raises an error because the model is in training mode

D) The features are computed with dropout, so they are all zero

Q5

What happens when this runs?

```
import torch
from torchvision import models
net = models.resnet18(weights=None).eval()
net(torch.zeros(1, 1, 64, 64))
```

A) It returns 1,000 ImageNet scores for the image

B) It returns 512 features because the model is in eval() mode

C) `ValueError`: the image must be resized to 224 pixels first

D) `RuntimeError`: the first convolution expects 3 channels and the image has 1

Q6

What does this print?

```
from torchvision import models
print(models.resnet18(weights=None).fc)
```

JOURNAL

Explain the experienced photo editor using the words backbone, head, frozen and fine-tune. Write the three accuracies side by side and the number of labels at which frozen features passed the from-scratch CNN. Would you pay for 5,000 more labels? What evidence would you show?

11.1 Text Classification

BY THE END YOU CAN

- Clean and tokenize text with Python's `re` module, and explain what scikit-learn's vectorizers do to a sentence
- Build bag-of-words and TF-IDF feature matrices, and read the sparse matrix and vocabulary they produce
- Train a leak-free sentiment classifier with a `Pipeline` of `TfidfVectorizer` and `LogisticRegression`, and compare it with a baseline
- Explain the stop-word trap, and use bigrams so that "not waterproof" and "not bad" become features of their own
- Describe what word embeddings add over TF-IDF, and when the simpler model is the right choice

KEY IDEAS

- **The tally sheet.** A bag of words counts which words appear, ignoring order. Text becomes a fixed-length row of numbers; the rest is ordinary scikit-learn.
- **Vectorizers tokenize for you.** Lowercase, runs of 2+ word characters: one-letter words disappear. `build_analyzer()` shows exactly what happens to a sentence.
- **Sparse matrices.** Mostly zeros, so only non-zero values are stored (5.3% here). Never `.toarray()` a real one.
- **TF-IDF** weighs a count by rarity: `idf = ln((1 + n) / (1 + df)) + 1`, then each row is scaled to length 1. A word in every document gets exactly 1.
- **Pipeline = no leak.** Vocabulary and IDF are learned in `fit`, from training text only, also inside every CV fold.
- **The numbers:** baseline 0.745; TF-IDF + logistic regression 0.877; stop words removed 0.864.
- **Read the words, probe with sentences.** "bad" pulls positive (Not bad at all). The English stop-word list removes not, no, never: never use it unread for sentiment.
- **Bigrams** give "not waterproof" and "not bad" their own columns. Choose a threshold from cross-validated probabilities, then grade once on test.

WORKED EXAMPLE: BIGRAMS AND IDF BY HAND

Four short notes about a sleeping bag. With `ngram_range=(1, 2)` and `min_df=2`, which terms survive?

```
from sklearn.feature_extraction.text import TfidfVectorizer
notes = ["warm bag", "not warm", "not warm at all", "light and warm"]
vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2).fit(notes)
print(vec.get_feature_names_out())
```

```
['not' 'not warm' 'warm']
```

Only terms in at least two notes are kept: `not`, `not warm` and `warm`. Now the weights:

```
import pandas as pd
print(pd.Series(vec.idf_, index=vec.get_feature_names_out()).round(3).to_dict())
print(pd.DataFrame(vec.transform(["not warm", "warm bag"]).toarray(),
                   columns=vec.get_feature_names_out()).round(2))
```

```
{'not': 1.511, 'not warm': 1.511, 'warm': 1.0}
    not  not warm  warm
0  0.64      0.64  0.42
1  0.00      0.00  1.00
```

`warm` is in all four notes, so its IDF is exactly 1; `not` and `not warm` are in two: ln(5/3) + 1 = 1.511. In the row for "not warm" the pair `not warm` carries as much weight as `not`, so a model can give the phrase its own coefficient. "warm bag" keeps only `warm` (bag is in one note, below `min_df`).

GUIDED LAB

- **Lab A:** `labs/module-09/l11_1-text-classification/recommend_flag_starter.py`. Predict the "would recommend" checkbox from the text of all 4,000 reviews: three vectorizer settings, one test, the words, accuracy by star rating, probes.
- **Lab B (pairs):** `category_router_starter.py`: route a message to one of ten product teams.
- Finished early? `debug_text_pipeline.py`: four bugs.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P3

Predict the output, then run it. (One document, unigrams and bigrams.)

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer(ngram_range=(1, 2)).fit(["not warm at all"]).get_feature
```

P4

Predict the output, then run it. (n = 4 documents; count the documents each word is in.)

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
v = TfidfVectorizer().fit(["tent leaked", "tent warm", "warm bag", "warm boots
print(pd.Series(v.idf_, index=v.get_feature_names_out()).round(3).to_dict())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Review triage (lesson page, "Project: Review Triage"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: `2321 train, 774 test, 905 three-star reviews held aside`
- ☐ Step 2: bigram pipeline, 0.877 at the 0.5 cut-off
- ☐ Step 3: cross-validated recall 0.570 / 0.756 / 0.851 / 0.926 for thresholds 0.5 / 0.4 / 0.3 / 0.2; choose 0.3
- ☐ Step 4: test recall 0.858, precision 0.698
- ☐ Step 5: read the five worst mistakes with their titles
- ☐ Step 6: `review_follow_up.csv`, 698 reviews

SELF-CHECK

Q7

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer().build_analyzer()("I am a 2-person tent fan!"))
```

Q8

Why does the lesson put `TfidfVectorizer` inside the pipeline instead of vectorizing all 3,095 reviews first?

A) Pipelines make TF-IDF faster on sparse matrices

B) A vectorizer outside a pipeline cannot produce bigrams

C) The vocabulary and IDF values are learned from data, so they must come from the training text only (also inside every CV fold)

D) LogisticRegression refuses a sparse matrix that was not produced inside a pipeline

Q9

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer(stop_words="english").build_analyzer()("Would not buy it
```

A) `['buy']`

B) `['would', 'not', 'buy', 'again']`

C) `['not', 'buy']`

D) `['would', 'buy', 'again']`

Q10

The training matrix is 2,321 × 333 with 5.3% of cells filled. Why does scikit-learn return it as a sparse matrix?

A) Sparse matrices make logistic regression more accurate

B) So that it can be turned into a pandas DataFrame

C) Because TF-IDF values are always between 0 and 1

D) Almost every cell is zero, so storing only the non-zero values saves memory, which matters with real vocabularies

Q11

What does this print? (scikit-learn's default IDF is ln((1 + n) / (1 + df)) + 1.)

```
from sklearn.feature_extraction.text import TfidfVectorizer
v = TfidfVectorizer().fit(["warm tent", "warm bag", "light tent", "light bag"]
print(round(v.idf_[v.vocabulary_["warm"]], 3))
```

Q12

In the review-triage project, lowering the flagging threshold for P(negative) from 0.5 to 0.3 does what?

A) Raises both recall and precision for negative reviews

B) Catches more unhappy customers (recall up) at the cost of more false alarms (precision down)

C) Changes the model's coefficients

D) Leaks the test set into training

JOURNAL

Explain the tally sheet in your own words. Write one sentence a bag of words gets wrong and the change that fixes it. Name one kind of text in your work that a model like this could sort, and what a false alarm and a miss would cost.

EXIT TICKET

E1

Name the backbone and the head of a pretrained ResNet-18, and say which one you replace and which one you freeze for feature extraction.

E2

Give one sentence a unigram bag of words gets wrong and the change that fixes it.

E3

Name one thing that must be learned from the training data only, in each lesson.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the land-cover classifier (Steps 4 to 6) and save `landcover_resnet18.pt`
- ☐ Finish review triage (Steps 4 to 6) and save `review_follow_up.csv`
- ☐ Lesson 10.5 Exercises 1 to 3; Lesson 11.1 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Lab A of each lesson to its Expected line
- ☐ 11.1 "Before the Next Lesson": five sentences of my own through the bigram model; `.head()` and `.shape` of `products_ml.csv` and `interactions.csv`
- ☐ Bring to Session 10: a READY laptop (MLflow and skops), `data/` and `labs/`, `landcover_resnet18.pt`, my 11.1 notebook and `review_follow_up.csv`, this workbook

GLOSSARY

TERM	MEANING
transfer learning	Reusing a model trained on a large dataset and training only what is specific to your task
backbone	The pretrained convolutional layers that turn an image into features
head	The final layer(s) that turn features into class scores (`fc` in ResNet, `classifier` in MobileNet)
feature extraction	Using a frozen backbone's outputs as inputs for a new, small model
fine-tuning	Letting some pretrained layers keep learning, with a small learning rate
`requires_grad`	Per-tensor switch: False freezes the tensor (no gradient, no update)
parameter groups	Optimizer groups with their own learning rate (layer4 1e-4, head 1e-3)
label curve	Accuracy against the number of labeled training examples: is labeling more worth it?
token / tokenizing	A unit of text that is counted (usually a word) / cutting text into tokens
bag of words	A document represented by word counts, ignoring order
sparse matrix	Stores only non-zero cells and their positions

TERM	MEANING
TF-IDF	Term frequency times inverse document frequency: counts weighted by rarity, rows scaled to length 1
stop words	Very common words some tools remove; the English list includes not, no, never
n-gram / bigram	A run of n consecutive tokens / a pair of words
embedding	A dense vector for a word or text, learned so that similar meanings are close

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 10: Recommendations and Reproducibility

Your workbook for Session 10: a warm-up on transfer learning and text, the key ideas of Lessons 11.2 and 12.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 11.2–12.1 | Session 10 · 3 hours | Name: _____

SESSION GOALS

- Build recommenders from a sparse customer-by-product matrix: best sellers, item-based filtering, matrix factorization, content.
- Evaluate them honestly: leave the last purchase out, hit rate at 10 against random and best sellers, settings chosen on a validation item.
- Make a result reproducible: seeds, a data fingerprint, recorded versions, settings in one place.
- Save models safely with skops and track experiments with MLflow (SQLite store).

WARM-UP: TRANSFER LEARNING AND TEXT

Five questions on Lessons 10.5 and 11.1. Answer alone first.

W1

What does this print?

```
from torch import nn
from torchvision import models
m = models.resnet18(weights=None)
for p in m.parameters():
    p.requires_grad = False
m.fc = nn.Linear(512, 5)
print(sum(p.numel() for p in m.parameters() if p.requires_grad))
```

W2

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer().build_analyzer()("Not bad, NOT bad at all!"))
```

W3

Why must the backbone be in `eval()` mode while you extract features?

W4

Someone fits a `TfidfVectorizer` on all reviews, then splits. What is wrong, and how do you fix it?

W5

Frozen features: 0.885 with 500 labels, 0.902 with 2,000. What is the better next step than labeling more?

LESSONS

11.2 Recommendation Systems

BY THE END YOU CAN

- Build a sparse customer-by-product matrix from purchase records with SciPy
- Evaluate a recommender with a leave-last-out split and hit rate at 10, against random and best-seller baselines
- Implement item-based collaborative filtering with cosine similarity, and matrix factorization with `TruncatedSVD`
- Use product attributes for content-based recommendations, and explain how they rescue a brand-new product
- Judge a recommender on more than accuracy: catalog coverage and popularity bias

KEY IDEAS

- **The clerk who remembers.** Collaborative filtering learns from many customers together which products end up with the same people.
- **Rank the empty cells.** A customer-by-product grid, 5.4% filled, stored as a sparse matrix; an empty cell is not a dislike.
- **Leave the last one out.** Hide each customer's latest purchase; HR@10 = share of customers whose hidden product is in their top 10 (owned products masked).
- **Baselines:** random 0.047, best sellers 0.209. Beat the best sellers before you celebrate; losing to random means a bug.
- **Item-kNN:** `R @ cosine_similarity(R.T)`, diagonal zeroed: 0.308. Explainable: "because you bought X".
- **Matrix factorization:** `TruncatedSVD` finds a few tastes; k = 5 scores 0.311, k = 40 memorizes (0.190). Choose k on a validation item.
- **Content-based** (attributes) is weaker (0.192) but solves the cold start for new products.
- **Beyond hit rate:** coverage and popularity bias; recommendations shape the data they learn from; purchase lists are personal data.

WORKED EXAMPLE: A RECOMMENDER YOU CAN CHECK BY HAND

The whole item-based recommender on four customers and four products, small enough to check by hand. Rows: Ana, Ben, Cy, Dee; columns: tent, stove, rope, harness.

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
# rows: Ana, Ben, Cy, Dee   columns: tent, stove, rope, harness
R = np.array([[1, 1, 0, 0],
              [1, 1, 0, 1],
              [0, 0, 1, 1],
              [1, 0, 0, 0]])
sim = cosine_similarity(R.T)
np.fill_diagonal(sim, 0)
print(sim.round(2))
```

```
[[0.   0.82 0.   0.41]
 [0.82 0.   0.   0.5 ]
 [0.   0.   0.   0.71]
 [0.41 0.5  0.71 0.  ]]
```

Tent and stove share two buyers (Ana, Ben): cos = 2 / √(3 × 2) = 0.82. Rope and tent share none: 0. Now score, mask what each customer owns, and take the top pick:

```
scores = R @ sim
scores = scores.astype(float)
scores[R > 0] = -np.inf          # never recommend what they own
print(scores.round(2))
print("top pick per customer:", np.argsort(-scores, axis=1)[:, 0])
```

```
[[-inf -inf 0.   0.91]
 [-inf -inf 0.71 -inf]
 [0.41 0.5  -inf -inf]
 [-inf 0.82 0.   0.41]]
top pick per customer: [3 2 1 1]
```

Ana (tent, stove) gets the harness (0.41 + 0.50 = 0.91); Ben gets the rope; Cy (rope, harness) gets the stove (0.50 beats the tent's 0.41); Dee (tent) gets the stove. Every top pick is "because you bought X": the explanation comes free with item-based filtering.

GUIDED LAB

- **Lab A:** `labs/module-10/l11_2-recommendations/segment_baselines_starter.py`. Best sellers of the customer's store and of their favorite activity, against global best sellers and item-kNN.
- **Lab B (pairs):** `user_based_starter.py`: customers like you, the number of neighbors chosen on a validation item, and a "because you just bought" list.
- Finished early? `debug_recommender.py`: four bugs.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P1

Predict the output, then run it. (Only the first pick of each list counts here.)

```
import numpy as np
top = np.array([[3, 1], [0, 2], [4, 5]])
test = np.array([3, 2, 5])
print((top[:, :1] == test[:, None]).any(axis=1).mean().round(3))
```

P2

Predict the output, then run it. (Owned items are masked before sorting.)

```
import numpy as np
scores = np.array([0.9, 0.2, 0.5, 0.7])
owned = np.array([True, False, False, True])
scores[owned] = -np.inf
print(np.argsort(-scores)[:2])
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Picked for You (lesson page, "Project: Picked for You"). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: `fit 26,178 validation 3,000 test 3,000`
- ☐ Step 2: helpers as functions; `(3000, 200) 26178`
- ☐ Step 3: validation HR@10: best sellers 0.209, item-kNN 0.286, SVD k=5 0.285; chosen item-kNN
- ☐ Step 4: test once: 0.308 vs 0.209 (+47%)
- ☐ Step 5: refit on everything; `picked_for_you.csv`, 15,000 rows
- ☐ Step 6: 0 owned, 112 different products, 63% of slots from the top 20

SELF-CHECK

Q1

A new recommender scores HR@10 = 0.004 on the leave-last-out test, where a random list scores 0.047. What is the most likely explanation?

A) A strong model on a hard dataset

B) A bug, such as sorting the scores ascending or forgetting to mask owned products

C) Too few factors in the SVD

D) The test item should have been chosen at random

Q2

What does this print?

```
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.metrics.pairwise import cosine_similarity
R = csr_matrix(np.array([[1, 0, 1], [1, 1, 0], [0, 1, 1], [1, 0, 0]]))
print(cosine_similarity(R.T)[0].round(2))
```

Q3

Why does `top_k` set the scores of owned products to minus infinity before sorting?

A) Because cosine similarity is undefined for owned products

B) To make the hit rate look better than it is

C) Because SciPy cannot sort sparse matrices

D) Owned products are similar to what the customer bought, so they score high and crowd the list; customers would see items they already have

Q4

The best-seller list shows 31 different products; item-kNN shows 161 and hits more often. What does that number measure?

A) Catalog coverage: how many different products appear on anyone's list

B) How many customers got a hit

C) How many products have at least one buyer

D) How many factors the model learned

Q5

What does this print?

```
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.decomposition import TruncatedSVD
svd = TruncatedSVD(n_components=3, random_state=42)
factors = svd.fit_transform(csr_matrix(np.eye(6, 8)))
print(factors.shape, svd.components_.shape)
```

Q6

You plot HR@10 against the number of SVD factors on the test item and pick the peak, k = 5. What is wrong with reporting 0.311?

A) Nothing: the test item was never used for training

B) SVD needs more than 5 factors to be valid

C) k was tuned on the test item, so 0.311 is optimistic; choose k on a validation item and score the test item once

D) Hit rate cannot be used to compare SVD models

Think of one recommendation an app made to you that felt right and one that felt wrong. Which kind of recommender produced each? Write the three numbers from your project you would show the marketing team, and one caution about feedback loops or privacy.

12.1 Reproducibility and Experiment Tracking

BY THE END YOU CAN

- Name the five ingredients of a reproducible result (code, data, environment, randomness, settings) and capture each one
- Fingerprint a data file with SHA-256 and record library versions next to a model
- Save and load a fitted pipeline with `joblib`, and explain why loading a pickle from an untrusted source can run arbitrary code
- Save and load models with `skops.io`, reviewing the untrusted types before you trust them
- Track parameters, metrics and models with MLflow, query the runs, and load a logged model back

KEY IDEAS

- **The lab notebook.** Record the five ingredients (code, data, environment, randomness, settings) plus the results.
- **Seeds** make randomness repeatable on the same software; third-decimal differences between seeds are noise (0.8612 vs 0.8560).
- **A SHA-256 fingerprint** proves which exact bytes a model saw; a re-saved file with the same meaning can have a different fingerprint.
- **Save the whole pipeline.** The classifier alone cannot read raw columns.
- **Loading a pickle runs code.** `joblib` files are pickles: never load one you did not make or fully trust.
- **skops** rebuilds only known types; review `get_untrusted_types` and pass an explicit, hand-written `trusted=` list. The list depends on the model.
- **A model card** (JSON beside the model) holds data fingerprint, settings, versions, score, trusted types and intended use.
- **MLflow** with `sqlite:///mlflow.db`: one run per setting, `log_params`, `log_metrics`, tags; choose winners with `search_runs`. The old `./mlruns` store is refused in MLflow 3.17.

WORKED EXAMPLE: A FINGERPRINT FOR YOUR SETTINGS

Settings are an ingredient too. Give each configuration a short ID that is the same whenever the settings are the same, whatever order you typed them in:

```
import hashlib, json
config_a = {"model": "forest", "n_estimators": 200, "seed": 42, "test_size": 0.25}
config_b = {"seed": 42, "test_size": 0.25, "model": "forest", "n_estimators": 200}

def config_id(cfg):
    text = json.dumps(cfg, sort_keys=True)          # one canonical spelling of th
    return hashlib.sha256(text.encode()).hexdigest()[:12]

print(config_id(config_a), config_id(config_b), config_id(config_a) == config_id(c
```

```
7c42f30604d9 7c42f30604d9 True
```

Change one setting and the ID changes; and see why `sort_keys=True` matters:

```
config_c = dict(config_a, n_estimators=201)
print(config_id(config_c))
print(json.dumps(config_a) == json.dumps(config_b), json.dumps(config_a, sort_keys
```

```
32c7a072bf5b
False True
```

A dictionary's text depends on the order of its keys; `sort_keys=True` gives one canonical spelling, so the hash identifies the settings, not the typing. Log the ID as an MLflow tag and two runs with the same ID must give the same result (given the same data fingerprint and environment).

GUIDED LAB

- **Lab A:** `labs/module-10/l12_1-reproducibility/track_demand_starter.py`. Three demand models tracked in MLflow, the winner chosen by querying the store, one test score, a seed check.
- **Lab B (pairs):** `safe_handover_starter.py`: hand a fraud model to another team with skops, a model card and a verify function.
- Everything the labs save goes to an `output/` folder next to the lab. To browse your runs, open a terminal in the lab's folder and run `mlflow ui --backend-store-uri sqlite:///output/mlflow.db`. Finished early? `debug_reproducibility.py`.

PREDICT, THEN RUN

Write your prediction first, then run the code.

P3

Predict the output, then run it. (A hash compares bytes, not meaning.)

```
import hashlib
same_hash = hashlib.sha256(b"6").hexdigest() == hashlib.sha256(b"6.0").hexdige
print(same_hash, float("6") == float("6.0"))
```

P4

Predict the output, then run it. (Two generators with the same seed, one with another.)

```
import numpy as np
a = np.random.default_rng(42).permutation(5)
b = np.random.default_rng(42).permutation(5)
c = np.random.default_rng(7).permutation(5)
print((a == b).all(), (a == c).all())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A tracked, reproducible churn model (lesson page). In class: Steps 1 to 3. At home: Steps 4 to 6.

- ☐ Step 1: `data fingerprint: ac541c7e8691d33f`
- ☐ Step 2: `(4500, 16) ['logreg', 'forest', 'hgb', 'hgb-slow']`
- ☐ Step 3: CV AUC 0.8324 / 0.8678 / 0.8615 / 0.8588, one MLflow run each
- ☐ Step 4: forest chosen by querying the store; test AUC 0.8612
- ☐ Step 5: `churn_final.skops` (trusted `numpy.dtype`, `sklearn.tree._tree.Tree`) and `churn_final.json`
- ☐ Step 6: `Reproduced : True`

SELF-CHECK

Q7

A colleague reruns your churn notebook and gets test AUC 0.8560 instead of 0.8612. The data fingerprint and every library version match. Which ingredient most likely differs?

A) The data file

B) The scikit-learn version

C) The random seed of the forest

D) AUC is computed differently on each run

Q8

What does this print?

```
import hashlib
h = hashlib.sha256(b"customers_churn.csv contents").hexdigest()
print(len(h), h == hashlib.sha256(b"customers_churn.csv contents").hexdigest()
```

Q9

What happens on the last line?

```
import skops.io as sio
from sklearn.ensemble import HistGradientBoostingClassifier
sio.dump(HistGradientBoostingClassifier().fit([[0], [1], [2], [3]], [0, 1, 0,
model = sio.load("q9.skops")
```

A) `UntrustedTypesFoundException`: it names the HistGradientBoosting tree type, which you must review and pass in `trusted=`

B) The model loads, because skops files are always safe

C) `pickle.UnpicklingError`: skops cannot read its own files

D) The model loads, but prints a warning about the scikit-learn version

Q10

Why should a model you share with another team be saved with skops rather than joblib?

A) skops files are smaller

B) joblib cannot save pipelines

C) skops makes the model more accurate

D) Loading a joblib file (a pickle) can run arbitrary code; skops only rebuilds known types or types you trust explicitly

Q11

What does this print?

```
from sklearn.ensemble import RandomForestClassifier
X, y = [[0], [1], [2], [3], [4], [5]], [0, 0, 1, 1, 0, 1]
a = RandomForestClassifier(n_estimators=5, random_state=3).fit(X, y).predict_p
b = RandomForestClassifier(n_estimators=5, random_state=3).fit(X, y).predict_p
print((a == b).all(), RandomForestClassifier().get_params()["random_state"])
```

Q12

Which call lists the runs of an experiment with the best cross-validated AUC first?

A) `mlflow.search_runs(experiment_names=["churn"], order_by=["params.cv_auc DESC"])`

B) `mlflow.search_runs(experiment_names=["churn"], order_by=["metrics.cv_auc DESC"])`

C) `mlflow.search_runs(experiment_names=["churn"], order_by=["metrics.cv_auc ASC"])`

D) `mlflow.search_runs(experiment_names=["churn"], filter_string="max(cv_auc)")`

Pick one earlier project of yours. For each of the five ingredients, could you reproduce it today, and what is missing? Then explain why loading a stranger's pickle is like running a stranger's program.

EXIT TICKET

E1

Item-kNN scores HR@10 0.308 and the best sellers 0.209. What does that comparison mean for Pinecrest's email?

E2

Name the five ingredients of a reproducible result and one tool or habit for each.

E3

A teammate sends `model.joblib`. What do you do before loading it?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish Picked for You (Steps 4 to 6) and save `picked_for_you.csv`
- ☐ Finish the tracked churn model (Steps 4 to 6): `churn_final.skops`, `churn_final.json`, `Reproduced : True`
- ☐ Lesson 11.2 Exercises 1 to 3; Lesson 12.1 Exercises 1 to 3 (Exercise 4 of each is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ `mlflow ui --backend-store-uri sqlite:///mlflow.db`: explore my runs; add `mlflow.db`, `mlruns/`, `*.skops` to `.gitignore`
- ☐ Bring to Session 11: a READY laptop (FastAPI, uvicorn, httpx), `churn_final.skops` and `churn_final.json`, `data/` and `labs/`, this workbook

GLOSSARY

TERM	MEANING
implicit / explicit feedback	What people did (purchases) / what they said (ratings)
collaborative filtering	Recommending from the behavior of many customers together
item-kNN	Scoring products by their similarity to what the customer owns
leave-last-out	Hold out each customer's latest interaction as the test item
hit rate at k (HR@k)	Share of customers whose held-out item is in their top-k list
matrix factorization	Approximating the grid by customer factors × product factors (`TruncatedSVD`)
cold start	No history yet for a new product or customer
catalog coverage	How many different products appear on anyone's list
reproducible	Rebuildable with the same result from recorded code, data, environment, randomness and settings
SHA-256 fingerprint	A 64-character hash of a file's exact bytes
pickle	Python's object format; loading one can run arbitrary code (`joblib` uses it)
skops	A safer format that rebuilds only known or explicitly trusted types
model card	A document beside a model: data, settings, versions, score, intended use

TERM	MEANING
MLflow run / experiment	One training attempt with its params, metrics and artifacts / a named group of runs
tracking store	Where MLflow keeps runs; here a SQLite file, `sqlite:///mlflow.db`

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 11: Serving, Monitoring and the Churn Case Study

Your workbook for Session 11: a warm-up, the key ideas of Lessons 12.2 and 13.1, worked examples, the guided labs, predict-then-run practice, self-checks, your capstone plan and your exit ticket.

Lessons 12.2–13.1 | Session 11 · 3 hours | Name: _____

SESSION GOALS

- Serve a model behind an order window: a pydantic schema, a FastAPI app, a test that the API matches the notebook.
- Monitor without labels: PSI for inputs, missing rates, the mean score and offer rate against a reference; tell a data bug from a real change.
- Run a whole project from a manager's brief: costs, leak check, fair comparison, tuning, a threshold from dollars, one look at the test set, a memo.
- Choose your capstone problem and write its plan.

WARM-UP

Five questions on Lessons 11.2 and 12.1. Write first, then check with the class.

W1

Lesson 11.2 hid each customer's LATEST purchase for testing, not a random one. Why?

W2

Best sellers scored a hit rate at 10 of 0.209 and item-kNN 0.308. Say what 0.308 means in one sentence.

W3

Why did SVD with 40 factors recommend worse than with 5?

W4

A colleague emails you `churn_model.joblib`. What is the risk of loading it, and what did Lesson 12.1 use instead?

W5

Your model card stores a SHA-256 of `customers_churn.csv`. Today the hash differs. What do you know, and what don't you?

12.2 Serving and Monitoring Models

BY THE END YOU CAN

- Package a fitted Pipeline with the metadata a server needs (features, dtypes, threshold, version)
- Describe valid input with a pydantic model and read its validation errors
- Serve predictions with FastAPI, test every endpoint with TestClient, and start the server with uvicorn
- Measure input drift with the Population Stability Index and prediction drift with simple batch statistics
- Tell a data-quality bug from a real change in customers, and say which one needs labels to confirm

KEY IDEAS

- **The kitchen and the order window.** FastAPI is the window, pydantic checks the order, the Pipeline is the kitchen. Bad orders get a 422 before the model runs.
- **Ship the facts the server cannot guess:** the feature list, the training dtypes, the threshold, a version, and reference statistics from out-of-fold predictions.
- **One function rebuilds the training table:** `to_frame`, with the same columns, order and dtypes as training. Prove it with a match test against the notebook, on rows with gaps.
- **NaN is not JSON:** missing values travel as `None` (null).
- **TestClient** tests endpoints in the same process; `uvicorn serve:app --reload` runs the real server, with interactive docs at `/docs`.
- **Monitoring without labels:** PSI per input (below 0.1 stable, 0.1 to 0.25 watch, above 0.25 shift), missing rates and rejections, the mean score and offer rate against the reference.
- **Drift is a question:** a data bug needs a fix (never retrain on it), a real change needs a decision, and only labels confirm the model got worse.

WORKED EXAMPLE

A different request, the same window: store managers ask the demand model for next week's forecast. What does pydantic convert, and what does it refuse?

```
from datetime import date
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field, ValidationError
class StockRequest(BaseModel):
    """A store manager asks the demand model for next week's forecast."""
    model_config = ConfigDict(extra="forbid")
    store: Literal["Asheville", "Bend", "Boise", "Flagstaff"]
    product_id: str = Field(pattern=r"^[A-Z]{3}-\d{4}$")
    week_start: date
    promo: bool
r = StockRequest(store="Bend", product_id="SNW-2001", week_start="2025-12-01", pro
print(r.week_start.strftime("%A"), r.promo, type(r.week_start).__name__)
```

```
Monday True date
```

Lax mode turned the text date into a real `date` (a Monday) and the word "yes" into `True`. Now three bad requests:

```
for bad in [{"store": "Bend", "product_id": "snw-2001", "week_start": "2025-12-01"
            {"store": "Bend", "product_id": "SNW-2001", "week_start": "2025-12-01"
            {"store": "Bend", "product_id": "SNW-2001", "week_start": "2025-13-01"
    try:
        StockRequest(**bad)
    except ValidationError as err:
        print([(e["loc"][0], e["type"]) for e in err.errors()])
```

```
[('product_id', 'string_pattern_mismatch')]
[('promo', 'bool_parsing')]
[('week_start', 'date_from_datetime_parsing'), ('units', 'extra_forbidden')]
```

A lowercase product code fails the pattern, "maybe" is not a boolean, and a request with month 13 AND an unknown field reports both problems. Finally, strict mode: no conversions at all.

```
class StrictStockRequest(StockRequest):
    model_config = ConfigDict(extra="forbid", strict=True)
try:
    StrictStockRequest(store="Bend", product_id="SNW-2001", week_start="2025-12-01
except ValidationError as err:
    print([(e["loc"][0], e["type"]) for e in err.errors()])
```

```
[('week_start', 'date_type'), ('promo', 'bool_type')]
```

Strict mode refuses text where a date or a boolean belongs. Lax is friendlier to callers that send everything as text; strict catches a caller that sends the wrong kind of value. Either way, write the choice down in the schema.

GUIDED LAB

- Open `labs/module-11/l12_2-serving/fraud_api_starter.py`. A fraud model is trained for you; TODO 1 to 6 package it, write a `Payment` schema, build the app and test it.
- Before TODO 4, write down how many problems pydantic will report for the bad payment.
- Done when your numbers match the "Expected when done" lines.
- Finished early? `fraud_monitor_starter.py` (real months, then a broken feed) or `debug_serving.py` (four bugs).

PREDICT, THEN RUN

Write your prediction first, then run it.

P1

Predict the output, then run it. (lax mode, an optional field)

```
from pydantic import BaseModel, Field
class Order(BaseModel):
    units: int = Field(ge=1, le=10)
    note: str | None = None
print(Order(units="7").units + 1, Order(units=7).note)
```

P2

Predict the output, then run it. (two PSI values)

```
import numpy as np
ref = np.array([0.25, 0.25, 0.25, 0.25])
for cur in [np.array([0.25, 0.25, 0.25, 0.25]), np.array([0.10, 0.20, 0.30, 0.
    print(round(float(np.sum((cur - ref) * np.log(cur / ref))), 3))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A weekly drift report (lesson page, Project). In class: Steps 1 to 3. At home: 4 to 6.

- ☐ Step 1: `churn-forest-2025-06-30 (4500, 17) (1500, 16)`
- ☐ Step 2: `monitor.py` with `psi` and `drift_report`
- ☐ Step 3: three weeks (tenure 44.0 / 25.0 / 42.7; email missing 8% / 6% / 100%)
- ☐ Step 4: every week scored through `/predict_batch`, status 200
- ☐ Step 5: the summary next to the reference values
- ☐ Step 6: `drift_report.csv` and a different action for a data bug than for a real change

SELF-CHECK

Q1

A caller posts a customer with `"home_store": "Denver"` to the lesson's `/predict`. What happens?

A) The one-hot encoder ignores the unknown store and the caller gets status 200 with a probability

B) pydantic rejects it before the model runs: status 422, and the body names `home_store` and the allowed stores

C) uvicorn crashes and the caller gets status 500

D) `to_frame` raises a `KeyError` and the caller gets status 500

Q2

What does this print?

```
from pydantic import BaseModel, Field
class Visit(BaseModel):
    days: int = Field(ge=0, le=365)
    share: float = Field(ge=0, le=1)
v = Visit(days="30", share=1)
print(v.days * 2, v.share)
```

Q3

Why does `to_frame` in `serve.py` end with `.astype(DTYPES)`?

A) It makes predictions faster

B) pydantic cannot produce a DataFrame without it

C) skops refuses to load the model unless the dtypes are stored

D) A one-row table with a `None` gets the `object` dtype, the imputer does not see it as missing, and the model scores a slightly different customer with no error

Q4

This week `email_opens_90d` has a PSI of 8.4, its missing rate is 100% (9% in the reference), and the mean score barely moved. What is the best reading?

A) A broken data feed: fix it and re-score the week; do not retrain on it

B) Customers changed: retrain on this week's batch

C) Nothing to do: the scores did not move, so the model is fine

D) PSI cannot be trusted above 1, so ignore it

Q5

What does this print? (A two-bin PSI.)

```
import numpy as np
ref = np.array([0.5, 0.5])
cur = np.array([0.8, 0.2])
print(round(float(np.sum((cur - ref) * np.log(cur / ref))), 3))
```

A) `0.0`

B) `-0.416`

C) `0.416`

D) `0.3`

Q6

With 30,000 rows, a Kolmogorov-Smirnov test on discount share gives p < 0.0001, while the PSI is 0.021. Should the monitor raise an alert?

A) Yes: p below 0.05 means the inputs drifted

B) No: with enough rows any difference is detectable; PSI says the shift is tiny, and an alert should be about size

C) Yes: PSI is unreliable on large batches

D) No: KS tests only work on categories

JOURNAL

List every place in this lesson where the model could have returned a wrong number without any error. For each, write the check that catches it. Which would you set up first on a real project, and why?

13.1 Case Study: Predicting Churn

BY THE END YOU CAN

- Translate a business goal into a target, a metric and a dollar value for each kind of right and wrong prediction
- Detect a target leak with cross-validation and a crosstab, and remove it before modeling
- Compare and tune models inside a Pipeline with cross-validation, keeping the test set locked until one final check
- Choose a decision threshold from business costs with TunedThresholdClassifierCV and report the result in dollars
- Explain the model honestly and write a short recommendation for a non-technical stakeholder

KEY IDEAS

- **The expedition plan:** decision and costs first, then hazards (leaks), then comparison and tuning, the algorithm late.
- **Dollars define the rule:** TP +$45, FP -$15. An offer pays when p > cost / value = 0.25, if the probabilities are calibrated.
- **Two baselines that matter to the manager:** offer to nobody ($0) and to everyone (a loss). A model must beat $0.
- **Leak check:** a sudden jump in score and a column filled only for one class are alarms; ask when each value becomes known.
- **Compare and tune with CV on the training data,** same folds and metric; a gap smaller than the spread is noise. In a Pipeline: `model__learning_rate`.
- **Confirm the threshold on data** with `TunedThresholdClassifierCV` and a profit scorer; then open the test set once.
- **Write for the reader:** the decision, the dollars against the alternative, how sure you are, what the model does NOT explain, and an experiment to learn more.

WORKED EXAMPLE

Eight customers, their predicted churn risk and (later) what happened. Each offer costs $15; reaching a would-be churner is worth $60. What is each offer worth before we know the outcome?

```
import numpy as np
import pandas as pd
toy = pd.DataFrame({"customer": list("ABCDEFGH"),
                    "p_churn": [0.82, 0.61, 0.44, 0.31, 0.27, 0.22, 0.12, 0.05],
                    "churned": [1, 1, 0, 1, 0, 0, 0, 0]})
OFFER_COST, CHURNER_VALUE = 15, 60
toy["expected_value"] = (toy["p_churn"] * CHURNER_VALUE - OFFER_COST).round(2)
print(toy.to_string(index=False))
```

```
customer  p_churn  churned  expected_value
       A     0.82        1            34.2
       B     0.61        1            21.6
       C     0.44        0            11.4
       D     0.31        1             3.6
       E     0.27        0             1.2
       F     0.22        0            -1.8
       G     0.12        0            -7.8
       H     0.05        0           -12.0
```

Expected value = p × 60 - 15: positive exactly when p is above 0.25. Now three thresholds, scored with the real outcomes:

```
def campaign_profit(y_true, flagged):
    y_true, flagged = np.asarray(y_true), np.asarray(flagged)
    return CHURNER_VALUE * ((flagged == 1) & (y_true == 1)).sum() - OFFER_COST * f
for t in [0.5, 0.25, 0.1]:
    flagged = (toy["p_churn"] >= t).astype(int)
    print(f"threshold {t:.2f}: {flagged.sum()} offers, profit ${campaign_profit(to
```

```
threshold 0.50: 2 offers, profit $90
threshold 0.25: 5 offers, profit $105
threshold 0.10: 7 offers, profit $75
```

The 0.25 line sends five offers and earns the most here: customer D (p = 0.31) churned and paid for two wasted offers. 0.5 leaves D out; 0.1 adds two customers whose expected value was negative. On eight customers luck plays a part; the case study confirms the same rule on 4,500.

GUIDED LAB

- **Lab A:** `labs/module-11/l13_1-case-study/leak_audit_starter.py`: write `audit()` that screens every column of a table for leaks, then run it on churn and on fraud.
- **Lab B (pairs):** `labs/module-11/l13_1-case-study/cost_scenarios_starter.py`: four offer costs, the profit of each block of 100 offers, and one look at the test set.

- Done when your numbers match the "Expected when done" lines.

PREDICT, THEN RUN

Write your prediction first, then run it.

P3

Predict the output, then run it. ($15 an offer, $60 a churner)

```
import numpy as np
y = np.array([1, 1, 0, 0, 0])
offer = np.array([1, 0, 1, 1, 0])
print(60 * ((offer == 1) & (y == 1)).sum() - 15 * offer.sum())
```

P4

Predict the output, then run it. (break-even = cost / value)

```
for cost, value in [(15, 60), (20, 50), (30, 60)]:
    print(f"{cost / value:.2f}", end="  ")
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The retention campaign plan (lesson page, Project). In class: Steps 1 and 2. At home: 3 to 6.

- ☐ Step 1: `4500 1500 offers allowed: 200`
- ☐ Step 2: CV AUC with and without age (0.879 / 0.880)
- ☐ Step 3: the 200 riskiest, never below break-even
- ☐ Step 4: offer rate, churn rate and precision by age group and store
- ☐ Step 5: the plan against 200 random offers and the unlimited plan, in dollars
- ☐ Step 6: `campaign_list.csv` (IDs and priority only) and the memo update

SELF-CHECK

Q7

Next quarter the retention offer costs $25 and is still worth $60 when it reaches a would-be churner. If the probabilities are calibrated, above which churn probability does an offer pay?

A) `0.25`

B) `0.5`

C) `0.417`

D) `2.4`

Q8

What does this print? ($15 an offer, $60 a churner reached.)

```
import numpy as np
y = np.array([1, 0, 1, 0, 0, 0])
flagged = np.array([1, 1, 1, 1, 0, 0])
print(60 * ((flagged == 1) & (y == 1)).sum() - 15 * flagged.sum())
```

Q9

Cross-validated AUC: random forest 0.868 ± 0.020, gradient boosting (defaults) 0.861 ± 0.013. What may you conclude?

A) The data does not yet say which tree family is better: the gap is inside one fold-to-fold spread

B) The forest is better and should be deployed

C) Boosting is better because its spread is smaller

D) Open the test set to break the tie

Q10

Which order matches the case study?

A) Tune, split, leak check, threshold, test, explain

B) Split, threshold on the test set, compare, tune, explain

C) Compare on the test set, tune with CV, choose the threshold on the test set, explain

D) Split, leak check, compare with CV, tune with CV, threshold with CV, test once, explain and write up

Q11

The manager asks whether the coupon actually keeps customers, or only goes to people who would have stayed anyway. What does the memo suggest?

A) Look at permutation importance: the top feature is the cause

B) Hold back the offer from a random 10% of the list and compare who stays

C) Compare churn of customers who used discounts with those who did not

D) Raise the threshold until precision is 100%

Q12

The budget allows 200 offers; 360 test customers are above the break-even probability. Which rule does the case-study project use?

A) The 200 riskiest customers, and never anyone below the break-even probability

B) 200 customers drawn at random from the 360

C) All 360: the budget is a guideline

D) The 200 customers closest to the break-even probability

◆ JOURNAL

Draw the case study as ten decisions, from "the target is churned" to "the threshold is 0.25". Next to each, the lesson where you learned it. Which decision would change the result most if you got it wrong?

CAPSTONE CHOICE

THE FIVE PROBLEMS (LESSON 13.2)

PROBLEM	DATA, SPLIT, METRIC	TEMPLATE BASELINE
A · Demand forecasting	`weekly_demand.csv`; train 2024, test 2025; MAE in units	13.419 (per-product mean)
B · Fraud review queue	`transactions.csv`; train Jan to May, test June; average precision + capacity	0.008 (no model)
C · Review triage	`reviews.csv` (3 stars dropped); stratified 75/25; accuracy + negative recall	0.745 (always positive)
D · Recommendations	`interactions.csv` + `products_ml.csv`; leave-last-out; hit rate at 10	0.209 (popular unowned)
E · Customer segments	`customers_churn.csv` behavior columns; no target; silhouette + stability + profiles	0.079 (loyalty tiers)

MY PLAN (ONE PARAGRAPH: DECISION, USER, TARGET, PREDICTION MOMENT, SPLIT, METRIC AND ITS BUSINESS MEANING, BASELINE, AND WHAT RESULT WOULD MAKE ME SAY "DO NOT DEPLOY")

My problem: A · B · C · D · E · own (circle one)

EXIT TICKET

E1

Name two places where a churn API can return a wrong number with status 200, and the check that catches each.

E2

The offer cost rises to $20 (value $60). The new break-even probability? Would you trust it without a check?

E3

Your capstone: the problem letter, the decision it supports and the baseline number you must beat.

NOTES

HOMEWORK CHECKLIST

- ☐ Drift report Steps 4 to 6; retention plan Steps 3 to 6
- ☐ Exercises 1 to 3 of both lessons (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Before the Next Lesson items of 12.2 and 13.1
- ☐ Read all of Lesson 13.2; create `capstone.ipynb` with my plan in the first cell (Milestone 1)
- ☐ Bring to Session 12: laptop, `pinecrest-ml` with `data/` and `labs/`, my capstone folder, my 13.1 notebook, this workbook

GLOSSARY

TERM	MEANING
serving	Letting other software call a model, usually as a web API
endpoint	One URL and method of an API, such as `POST /predict`
pydantic model	A Python class that describes and validates input
422	FastAPI's status for a request that fails validation
TestClient	Sends requests to a FastAPI app in the same process, for tests
training-serving skew	Any difference between how data was prepared for training and for serving
PSI	Population Stability Index: how far a batch's distribution is from the reference
covariate shift	The inputs change; the relationship may not
concept drift	The relationship between inputs and the target changes
break-even probability	cost / value: above it an offer pays on average
TunedThresholdClassifierCV	Chooses a decision threshold by cross-validated score
model card	A short document that travels with a model: use, performance, limits

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 12: Capstone Kickoff and Studio I

Your workbook for Session 12: the plan clinic questions, a worked example on splits, the capstone labs, your feature-timing table, the milestone checklist, the rubric, a self-check on project skills and the exit ticket.

Lesson 13.2 | Session 12 · 3 hours | Name: _____

SESSION GOALS

- Turn your choice into a plan someone else could check, with a "do not deploy" bar.
- Load and check your data, and fill the feature-timing table.
- Measure two honest baselines and build ONE evaluation function on the training data.
- Put the simplest sensible pipeline through it.

WARM-UP

W1

Why does `serve.py` rebuild the request frame with the stored training dtypes?

W2

Of input drift, data quality, prediction drift and performance, which needs labels?

W3

Where does the churn threshold 0.25 come from, and what does it assume?

W4

In the case study, when is the test set used?

W5

Your capstone in one sentence: the decision, and the baseline number you must beat.

LESSON

13.2 Capstone Project

BY THE END YOU CAN

- Frame a business request as a machine-learning task with a decision, a target, a leakage-safe split, a metric and a baseline, written down before any modeling
- Build, compare and improve models in leakage-proof pipelines, touching the test set once
- Analyze errors by subgroup and explain the model honestly, without claiming causes
- Review your model against a responsible-ML checklist and document it in a model card
- Deliver a project that reruns from a fresh start and score it yourself against a rubric

KEY IDEAS

- **The trail guide's report:** the notebook is the scouting trip; the model card is the report a manager reads in ten minutes.
- **The plan comes first:** decision, user, target, prediction moment, split, metric with its business meaning, baseline, and the result that would mean "do not deploy".
- **The template is the rules of the competition:** its split, metric and baseline are fixed unless you write down why another fits the decision better.
- **Every feature passes the timing test:** would I know this value at the moment of prediction?
- **One harness for every candidate:** cross-validation on the training data only, shaped like the real use (time-ordered where the data is).
- **A first pipeline that loses to the baseline usually has a bug.** Look before you tune.
- **The test set waits.** It is scored once, in Session 13, after every choice is made.

WORKED EXAMPLE: SHUFFLED OR TIME-ORDERED?

Why does the split have to match the data? The lesson's worked example forecasts daily store sales. Here is the same kind of model scored three ways.

```
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import KFold, TimeSeriesSplit, cross_val_score
sales = pd.read_csv("data/daily_store_sales.csv", parse_dates=["date"]).sort_value
per_store = sales.groupby("store")["sales"]
sales["lag_1"], sales["lag_7"] = per_store.shift(1), per_store.shift(7)
sales["dow"], sales["month"] = sales["date"].dt.dayofweek, sales["date"].dt.month
sales["store_code"] = sales["store"].astype("category").cat.codes
table = sales.dropna(subset=["lag_7"]).reset_index(drop=True)
cols = ["store_code", "dow", "month", "promo", "lag_1", "lag_7"]
train, test = table[table["date"] < "2025-01-01"], table[table["date"] >= "2025-01
print(len(train), "training store-days,", len(test), "test store-days (2025)")
```

```
2896 training store-days, 1460 test store-days (2025)
```

```
model = HistGradientBoostingRegressor(random_state=42)
for name, cv in [("shuffled 5-fold", KFold(5, shuffle=True, random_state=42)),
                 ("time-ordered 5-fold", TimeSeriesSplit(n_splits=5))]:
    mae = -cross_val_score(model, train[cols], train["sales"], cv=cv, scoring="neg
    print(f"{name:20s} CV MAE ${mae.mean():,.0f}")
model.fit(train[cols], train["sales"])
print(f"{'2025 test, once':20s}    MAE ${mean_absolute_error(test['sales'], model.
```

```
shuffled 5-fold      CV MAE $551
time-ordered 5-fold  CV MAE $659
2025 test, once         MAE $600
```

Shuffled folds let the model train on days AFTER the days it is tested on, and neighboring days are alike, so the score looks better than the real future: $551 against $659 for time-ordered folds. The 2025 test ($600) lands between, nearer the honest harness. Choose with the harness that matches how the model will be used, and report its number as an estimate, not a promise.

GUIDED LABS

- **Baselines:** `labs/module-12/l13_2-capstone/baselines_starter.py`, with `PROBLEM` set to your letter. One more honest baseline, and your "do not deploy" sentence.
- **Harness:** `labs/module-12/l13_2-capstone/harness_starter.py`: one function that scores any candidate on the training data the way your split works; the baseline first, then your first pipeline.
- Done when the baseline and your first pipeline both have a number from the same function.

PREDICT, THEN RUN

P1

Predict the output, then run it. (a lag)

```
import pandas as pd
s = pd.Series([10, 0, 30, 0])
print(s.shift(1).tolist())
```

P2

Predict the output, then run it. (leave the last one out)

```
import pandas as pd
df = pd.DataFrame({"customer": ["a", "a", "a", "b", "b"], "item": ["x", "y", "
print(df.groupby("customer").tail(1)["item"].tolist(), len(df.groupby("custome
```

MILESTONE CHECKLIST (TODAY AND AT HOME)

- ☐ M1 Plan cell rewritten after the clinic
- ☐ M2 Checks with one line each; what one row is; rows in train and test
- ☐ M2 Feature-timing table: every column, known at prediction time?
- ☐ M3 Two baselines on the same split and metric
- ☐ M3 `evaluate()` scores the baseline
- ☐ M4 First pipeline through `evaluate()`
- ☐ M5 (home) Attempts log; one final candidate; test set NOT opened

MY FEATURE-TIMING TABLE

MY NUMBERS

Baseline (template): _____ · Second baseline: _____ · Baseline through my harness: _____ · First pipeline: _____

SELF-CHECK (PROJECT SKILLS)

Q1

Which first line of a capstone plan names a DECISION, not just a target?

A) Each morning the two-person review team decides which of the day's payments to hold; the model ranks them

B) Predict is_fraud from the transaction columns

C) Build the most accurate fraud model possible

D) Use gradient boosting on transactions.csv

Q2

The demand order is placed three weeks ahead. Which column must NOT be a feature as it stands?

A) `promo` (planned weeks ahead)

B) `avg_high_temp_f`, the actual temperature of the forecast week

C) `week_of_year`

D) `list_price`

Q3

What does this print?

```
import pandas as pd
df = pd.DataFrame({"c": ["a", "a", "b", "b", "b"], "d": [3, 1, 2, 5, 4]}).sort
print(df.groupby("c").tail(1)["d"].tolist())
```

Q4

Why does the fraud harness validate on May instead of a shuffled 5-fold split of January to May?

A) Shuffled folds are slower

B) May has more fraud

C) The model will score future months, and the fraud rate drifts; validating on a later month mimics that

D) scikit-learn cannot shuffle time stamps

Q5

On recommendations, how should you choose the number of SVD factors k?

A) The k with the best hit rate on the test purchases

B) Always k = 5, as in Lesson 11.2

C) The largest k the laptop can fit

D) The best k on a validation purchase carved from the training rows (each customer's second-to-last)

Q6

K-Means on the four UNscaled behavior columns gives silhouette 0.545, on scaled columns 0.470. Which segments should you keep?

A) The scaled ones: unscaled distances are dominated by average order value, so the higher number comes from a worse clustering

B) The unscaled ones: a higher silhouette is always better

C) Neither: silhouette below 0.6 means no segments

D) Whichever has more equal cluster sizes

Q7

What does this print? (A per-product-mean baseline.)

```
import numpy as np
from sklearn.metrics import mean_absolute_error
train_units = {"TNT-1": [10, 20], "SNW-1": [0, 4]}
test_units = [("TNT-1", 18), ("SNW-1", 1)]
means = {k: np.mean(v) for k, v in train_units.items()}
print(mean_absolute_error([u for _, u in test_units], [means[p] for p, _ in te
```

Q8

Your first pipeline does not beat the baseline in your harness. What does the lesson say to do first?

A) Try a neural network

B) Tune harder

C) Look for a bug (a split, a feature, a leak in reverse) before looking for a better model

D) Report "do not deploy" at once

Q9

Review triage drops 3-star reviews from training. What belongs in the plan?

A) Nothing: 3-star reviews are neutral

B) Add them to the positive class

C) Delete 3-star reviews in production too

D) Say what the live system does with them and measure it (how many get flagged as negative)

Q10

When should the "do not deploy" bar be written?

A) After the test score, to explain it

B) Before any modeling, in the plan

C) Only if the model does badly

D) In the model card's last section

Q11

In June, new accounts are 8% of legitimate payments but about 29% of the payments the fraud model wrongly holds. What does the responsible-ML checklist ask?

A) Name who bears the errors, keep a human review with a fast release path, and report results by account age

B) Remove account_age_days so the model cannot see it

C) Nothing: new accounts really are riskier

D) Lower the threshold until the share matches

Q12

Demand harness inside 2024 (rolling folds): ridge MAE 13.13 against the baseline's 13.92, yet on 2025 ridge scores 11.29 against 13.42. Why is the in-2024 harness so pessimistic?

A) Ridge overfits 2024

B) The test set is easier by chance

C) Each validation block contains months the folds have never seen, so a month effect learned from one year cannot be checked forward inside that year

D) MAE is not a valid metric for rolling folds

◆ JOURNAL

Which decision in your plan was hardest to make, and what would change if you got it wrong?

THE RUBRIC (LESSON 13.2)

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Framing	"Predict X" with no decision or user.	Target and metric chosen, but the decision, the costs of errors or the prediction moment are vague.	Decision, user, target, prediction moment, split, metric and baseline written down before modeling, and followed.	Also states in advance what result would mean "do not deploy", and translates the metric into the business's units.
Data and leakage	Columns used as they come. No checks.	Shapes and missing values checked, but some feature's timing is never questioned.	Every feature's availability at prediction time is checked; transformations are fitted inside pipelines; the split matches the data's structure (time, groups).	Also demonstrates a leak caught (score before and after) and documents data quirks such as stockout zeros or mislabeled stars.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Evaluation	One score on training data, or accuracy on rare events.	Test score against a baseline, but the test set was used for choices, or the metric does not fit the decision.	Model selection with cross-validation on the training data, test used once, metric fits the decision, baseline beside it.	Also an interval (bootstrap) for the test metric and an operating point (threshold, top-k) chosen from costs or capacity.
Modeling	One model, default settings, no comparison.	Several models tried in a loop, without a reason for the order or the final pick.	A simple model first, then a stronger one; feature work and modest tuning that each earn their place in CV.	Also shows which changes did not help and stops when gains are smaller than the noise.
Error analysis and explanation	None.	A feature importance chart, unexplained.	Errors broken down by a meaningful subgroup; worst cases read; drivers explained as properties of the model.	Also turns an error finding into a fix (a feature, a rule, a caveat) and measures its effect.
Responsible ML	Not considered.	A generic paragraph about fairness.	Every checklist item answered for this problem: who bears errors, subgroup results, sensitive attributes, privacy, human review, monitoring.	Also changes the system because of a finding (a threshold, a review step, a removed feature) and says why.
Communication	The notebook is the report.	A model card exists, but numbers lack baselines, or caveats are generic.	One to two pages that lead with a recommendation, with quantified, baseline-relative evidence, specific limitations and finding-first figures.	Also readable by a non-specialist, who could act on it correctly.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Reproducibility	Fails or gives different numbers when rerun.	Runs, but needs cells in a special order, absolute paths or unseeded randomness.	Restart and Run All works from the project folder; seeds set; model saved with skops; versions pinned; card numbers match the output.	Also a README, functions with clear names, and results logged (MLflow or results.json) for each experiment.

EXIT TICKET

E1

Write your harness in one sentence: which rows train, which rows validate, which metric.

E2

Your two baselines and their numbers.

E3

Name one column you will never use, and why.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish Milestone 4
- ☐ Milestone 5: features, a stronger model, modest tuning, all in the log
- ☐ One final candidate chosen on the training data; the test set still unopened
- ☐ Lesson Exercises 1 and 2; the Quick Quiz
- ☐ Bring my notebook through Milestone 5 to Session 13

GLOSSARY

TERM	MEANING
prediction moment	The time a prediction is made; only values known then may be features
harness	One function that scores any candidate the same way on the training data
rolling-origin CV	Train on the past, validate on the next block, move forward
leave-last-out	Hide each customer's latest purchase as the test item
operating point	The threshold or list size used in practice (top 1%, top 10)
"do not deploy" bar	The result, written in advance, that would mean the model is not worth using

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 13: Capstone Studio II and Showcase

Your workbook for Session 13: intervals and subgroups, the responsible-ML checklist, your showcase sentences, the rubric, a self-check on project skills, the submission checklist and the exit ticket.

Lesson 13.2 | Session 13 · 3 hours | Name: _____

SESSION GOALS

- Score the test set once, with an interval, next to the baseline.
- Find where the model fails, by subgroup.
- Explain the model honestly and answer every responsible-ML checklist item.
- Draw the one figure, write the card's first sections from your results, and present in three minutes.

WARM-UP

W1

Why does the fraud harness validate on May, not on shuffled folds?

W2

What is the one loop the lesson does not allow?

W3

Name two parts of a model card that a weak card usually lacks.

W4

What does "explanations are not causes" mean for your card?

W5

Status: your final candidate, its CV score and the baseline's, in one line.

LESSON

13.2 Capstone Project

BY THE END YOU CAN

- Frame a business request as a machine-learning task with a decision, a target, a leakage-safe split, a metric and a baseline, written down before any modeling
- Build, compare and improve models in leakage-proof pipelines, touching the test set once
- Analyze errors by subgroup and explain the model honestly, without claiming causes
- Review your model against a responsible-ML checklist and document it in a model card
- Deliver a project that reruns from a fresh start and score it yourself against a rubric

KEY IDEAS

- **Test once:** refit the final pipeline on all training data, score the test set, write the number next to the baseline. The model is now finished.
- **An interval with every headline number:** a paired bootstrap for model vs baseline; resample whole weeks or customers when rows are not independent.
- **An operating point from capacity or costs:** the team's 1%, top 10, a threshold from dollars.
- **Averages hide failures:** break errors down by a meaningful group, with the baseline's errors beside them, and read the worst cases.
- **Explanations describe the model,** never the causes; say who bears the errors and keep a human where an error harms someone.
- **The card leads with a recommendation,** and its numbers come from the results, never from memory. "Do not deploy" with evidence is a full answer.

WORKED EXAMPLE: AN INTERVAL FOR A HIT-RATE GAIN

How sure is "better than the baseline"? On the recommendations problem, item-kNN beats the best-seller list on the test purchases. A paired bootstrap resamples the SAME customers for both models.

```
import numpy as np
import pandas as pd
from scipy.sparse import csr_matrix
from sklearn.metrics.pairwise import cosine_similarity
df = pd.read_csv("data/interactions.csv").sort_values(["customer_id", "last_purcha
test = df.groupby("customer_id").tail(1)
train = df.drop(test.index)
customers, products = np.sort(df["customer_id"].unique()), np.sort(df["product_id"
row = pd.Series(np.arange(len(customers)), index=customers)
col = pd.Series(np.arange(len(products)), index=products)
R = csr_matrix((np.ones(len(train)), (row[train["customer_id"]], col[train["produc
               shape=(len(customers), len(products)))
owned = R.toarray() > 0
hidden = col[test.set_index("customer_id").loc[customers, "product_id"]].to_numpy(
def hits(scores):
    s = np.array(scores, dtype=float)
    s[owned] = -np.inf
    return (np.argsort(-s, axis=1)[:, :10] == hidden[:, None]).any(axis=1).astype(
sim = cosine_similarity(R.T)
np.fill_diagonal(sim, 0)
pop_hit = hits(np.tile(np.asarray(R.sum(axis=0)).ravel(), (R.shape[0], 1)))
knn_hit = hits(np.asarray(R @ sim))
print(f"popularity {pop_hit.mean():.3f}   item-kNN {knn_hit.mean():.3f}   differen
```

```
popularity 0.209   item-kNN 0.308   difference 0.099
```

```
rng = np.random.default_rng(42)
diffs = []
for _ in range(1000):
    rows = rng.integers(0, len(knn_hit), len(knn_hit))          # the SAME custome
    diffs.append(knn_hit[rows].mean() - pop_hit[rows].mean())
low, high = np.percentile(diffs, [2.5, 97.5])
print(f"item-kNN minus popularity: {knn_hit.mean() - pop_hit.mean():.3f} (95% CI {
n_hist = np.asarray(R.sum(axis=1)).ravel()
group = pd.cut(n_hist, [0, 4, 8, 15, 100], labels=["2-4", "5-8", "9-15", "16+"])
print(pd.DataFrame({"customers": pd.Series(group).value_counts().sort_index(),
                    "popularity": pd.Series(pop_hit).groupby(group, observed=True)
                    "item_kNN": pd.Series(knn_hit).groupby(group, observed=True).m
```

```
item-kNN minus popularity: 0.099 (95% CI 0.082 to 0.116)
      customers  popularity  item_kNN
2-4         607       0.199     0.257
5-8         987       0.202     0.280
9-15        950       0.204     0.334
16+         456       0.246     0.382
```

The gain (0.099) has an interval far from zero, so it is real. The subgroup table says more than the average: customers with long histories gain most, and even customers with only 2 to 4 earlier purchases do better than with best sellers. That is one sentence for the card's Performance by subgroup section.

GUIDED LABS

- **Intervals:** `labs/module-13/l13_2-capstone/bootstrap_ci_starter.py`: a paired bootstrap on fraud, then rows vs whole weeks on demand.
- **Hand-in steps:** `labs/module-13/l13_2-capstone/model_card_starter.py`: results.json, a skops round trip, a card written from the results, a number check.
- **Review practice:** `labs/module-13/l13_2-capstone/debug_capstone.py`: a classmate's capstone that breaks four rules. Which, and which rubric row does each cost?

PREDICT, THEN RUN

P1

Predict the output, then run it. (the top 20%)

```python
import numpy as np
scores = np.array([0.9, 0.1, 0.4, 0.8, 0.2, 0.05, 0.3, 0.6, 0.7, 0.15])
held = scores >= np.quantile(scores, 0.8)
print(held.sum(), scores[held].tolist())
```

P2

Predict the output, then run it. (a mean and an interval end)

```
import numpy as np
diffs = np.array([0.02, -0.01, 0.03, 0.00, 0.05])
print(round(float(diffs.mean()), 3), bool(np.percentile(diffs, 2.5) < 0))
```

MY MILESTONE 6 NUMBERS

Test metric: _____ (95% CI _____ to _____) · baseline on the same test set: _____ · operating point: _____

WHERE MY MODEL FAILS (ONE FINDING)

RESPONSIBLE-ML CHECKLIST: ONE SPECIFIC ANSWER EACH

1

Leakage audit

2

One look at the test set

3

Baseline on the same test set

4

Subgroup performance

5

Who bears the errors

6

Sensitive attributes

7

Humans in the loop

8

Explanations are not causes

9

Privacy

10

Monitoring and fallback

SHOWCASE: MY THREE SENTENCES

Decision and recommendation · main number against the baseline, with its interval · one specific limitation

SELF-CHECK (PROJECT SKILLS)

Q1

When may you compute the test score of your capstone?

A) After every choice (features, model, hyperparameters, threshold or list size) is made on the training data, once

B) After each improvement, to see whether it helped

C) Before tuning, to know where you stand

D) Whenever, as long as you report the best one

Q2

What does this print?

```
import numpy as np
print(np.percentile(np.arange(1, 101), [2.5, 97.5]))
```

Q3

June average precision: tuned boosting 0.556, logistic regression 0.539; the paired bootstrap interval of the difference is -0.046 to +0.077. What goes in the card?

A) Boosting is better: 0.556 is higher

B) Logistic regression is better: it is simpler

C) The two cannot be told apart on 64 June frauds; choose on other grounds (simplicity, explainability, cost) and say so

D) Neither beats the baseline

Q4

Demand: resampling store-product ROWS gives a 95% interval of 4.90 to 5.53 units saved; resampling whole WEEKS gives 4.68 to 5.81. Which belongs in the card?

A) The row interval: it uses more data points

B) Neither: intervals are optional

C) The narrower one, whichever it is

D) The week interval: rows in the same week share the same conditions, so they are not independent

Q5

Review triage card, first line of Performance. Which is strongest?

A) The model is 87% accurate

B) On 774 held-out reviews the model finds 87% of the negative reviews (the baseline finds none), at accuracy 0.873 (95% CI 0.85 to 0.895) against 0.745

C) The model uses TF-IDF bigrams and balanced logistic regression

D) More data would improve the model

Q6

Your segment with the highest churn (about 32%) uses discounts on half its spending. Which sentence may go in the card?

A) Discounts cause these customers to leave

B) Stop discounting to this segment

C) This segment's model was trained on churn

D) Customers in this segment churned more often (about 32%); the segments describe behavior, not why customers leave

Q7

What does this print? (Errors by subgroup.)

```
import pandas as pd
err = pd.DataFrame({"store": ["Bend", "Bend", "Boise", "Boise", "Boise"],
                    "abs_error": [2.0, 4.0, 10.0, 0.0, 5.0]})
print(err.groupby("store")["abs_error"].mean().to_dict())
```

Q8

Your demand model's MAE is lowest for Snow (5.8 units) and highest for Footwear (12.5). What should the card say?

A) Snow is forecast best

B) Compare each group's error with its baseline and its typical volume: Snow sells about 12 units a week, Footwear about 46

C) Drop Footwear from the model

D) Nothing: subgroup errors are noise

Q9

Which figure may go into a public portfolio repository?

A) Hit rate at 10 by customer history length, for the model and the baseline

B) A table of ten customers with their recommended products

C) The twenty worst forecast errors with store and product IDs

D) Customer IDs and fraud scores for June

Q10

In Problem E, what should you do with `age_group`?

A) Add it to the clustering so segments are easier to target

B) Delete it from the data file

C) Keep it out of the clustering, then check whether the segments differ by age

D) Use it to name the segments

Q11

After a final rerun, your card says accuracy 0.877 but the notebook prints 0.873. What is the best fix for good?

A) Change the notebook's seed until it prints 0.877

B) Round both to 0.88

C) Delete the number from the card

D) Write the card from the results (results.json and an f-string), and check every card number against it

Q12

Your three-minute showcase. Which outline fits?

A) The decision and the recommendation, the main number against the baseline with its interval, one specific limitation, pointing at one figure

B) Every model you tried, in order

C) The code, cell by cell

D) The rubric scores you gave yourself

◆ JOURNAL (THE LESSON'S PROMPT)

Write down the one decision in your capstone you are proudest of: a leak you caught, a metric you chose over an easier one, a claim you refused to make. Explain it in three sentences as you would to an interviewer. Then reread your Lesson 1.1 entry: what would you tell that version of yourself about baselines?

THE RUBRIC (LESSON 13.2)

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Framing	"Predict X" with no decision or user.	Target and metric chosen, but the decision, the costs of errors or the prediction moment are vague.	Decision, user, target, prediction moment, split, metric and baseline written down before modeling, and followed.	Also states in advance what result would mean "do not deploy", and translates the metric into the business's units.
Data and leakage	Columns used as they come. No checks.	Shapes and missing values checked, but some feature's timing is never questioned.	Every feature's availability at prediction time is checked; transformations are fitted inside pipelines; the split matches the data's structure (time, groups).	Also demonstrates a leak caught (score before and after) and documents data quirks such as stockout zeros or mislabeled stars.
Evaluation	One score on training data, or accuracy on rare events.	Test score against a baseline, but the test set was used for choices, or the metric does not fit the decision.	Model selection with cross-validation on the training data, test used once, metric fits the decision, baseline beside it.	Also an interval (bootstrap) for the test metric and an operating point (threshold, top-k) chosen from costs or capacity.
Modeling	One model, default settings, no comparison.	Several models tried in a loop, without a reason for the order or the final pick.	A simple model first, then a stronger one; feature work and modest tuning that each earn their place in CV.	Also shows which changes did not help and stops when gains are smaller than the noise.
Error analysis and explanation	None.	A feature importance chart, unexplained.	Errors broken down by a meaningful subgroup; worst cases read; drivers explained as properties of the model.	Also turns an error finding into a fix (a feature, a rule, a caveat) and measures its effect.

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Responsible ML	Not considered.	A generic paragraph about fairness.	Every checklist item answered for this problem: who bears errors, subgroup results, sensitive attributes, privacy, human review, monitoring.	Also changes the system because of a finding (a threshold, a review step, a removed feature) and says why.
Communication	The notebook is the report.	A model card exists, but numbers lack baselines, or caveats are generic.	One to two pages that lead with a recommendation, with quantified, baseline-relative evidence, specific limitations and finding-first figures.	Also readable by a non-specialist, who could act on it correctly.
Reproducibility	Fails or gives different numbers when rerun.	Runs, but needs cells in a special order, absolute paths or unseeded randomness.	Restart and Run All works from the project folder; seeds set; model saved with skops; versions pinned; card numbers match the output.	Also a README, functions with clear names, and results logged (MLflow or results.json) for each experiment.

Score yourself on all eight rows before you submit and fix the lowest row first. Proficient in every row is a successful capstone.

EXIT TICKET

E1

Your main result in one sentence: the metric with its interval, against the baseline.

E2

One limitation specific to your data (not "more data").

E3

What is left, in order, before you submit?

NOTES

SUBMISSION CHECKLIST (WITHIN ONE WEEK)

- ☐ A written plan opens the notebook, and the work follows it (or says where and why it changed)
- ☐ Every feature passed the timing check; all preprocessing is inside a pipeline
- ☐ Choices made with cross-validation; the test set scored once, next to the baseline, with an interval
- ☐ Errors by at least one subgroup; every checklist item answered
- ☐ `model_card.md` leads with a recommendation; its numbers match the output
- ☐ Restart and Run All succeeds; `model.skops`, `results.json`, `requirements.txt` exist
- ☐ README with the fictional-data line and the help I used; my self-scored rubric

GLOSSARY

TERM	MEANING
paired bootstrap	Resample the same rows for two models and look at the difference
block bootstrap	Resample whole groups (weeks, customers) that are not independent
operating point	The threshold or list size used in practice
subgroup analysis	The metric by store, category, account age or segment, with the baseline beside it
model card	One to two pages: recommendation, use, performance, subgroups, risks, limits, monitoring, how to reproduce
fallback	What runs when the model is switched off (usually the baseline)

Complete Participant Workbook · all 13 sessions · v1.0 · © 2026 Ray de la Paz